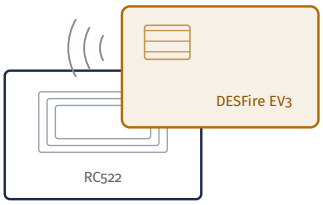
THE MIDNIGHT BREWER

Building a Contactless Payment Terminal From Scratch

From SPI bytes to AES mutual authentication — every layer written by hand, on a £3 reader and a Raspberry Pi.



A build-it-yourself book · Java · Dart · C-to-OOP

Revision July 26, 2026

The Midnight Brewer: Building a Contactless Payment Terminal From Scratch

A companion to the `DesfireVendingMachine` project. Every implementation described here exists, runs on a Raspberry Pi Zero 2 W with a £3 MFRC522 module, and debits a real MIFARE DESFire EV3 card.

On the code in this book. The listings are deliberately incomplete. They give you the class shape, the constants and the byte layouts, and leave the logic as `TODO`. That is the method, not an oversight — see *How to use this book* on page xiv.

Safety. Chapter 13 describes commands that can *permanently* destroy a contactless card. Use dedicated test cards. Never run provisioning against a card you did not buy for this project.

Security. The keys printed in this book are bench keys, published here on purpose. They protect nothing. Section 13.6 explains what a real deployment does instead.

Contents

Preface **xiii**

I Groundwork **1**

1 Setup: hardware, Pi, and an empty project **3**

1.1 Wiring the RC522 . . . 3

1.2 Pi OS and SPI . . . 4

1.3 Create the project . . . 5

1.4 Java for a C programmer . . . 6

1.5 Commit . . . 8

II Talking to the Chip **9**

2 The SPI seam **11**

2.1 The problem . . . 11

2.2 The idea: program against a description, not a thing . . . 12

2.3 Build it . . . 12

2.4 The concept, now that you have used it . . . 19

2.5 Traps . . . 19

2.6 Commit . . . 20

3 The RC522 driver **21**

3.1 The problem . . . 21

3.2 The idea: gather the mess in one place . . . 21
3.3 Build it . . . 22
3.4 Verify — with no hardware . . . 25
3.5 Now the real chip . . . 26
3.6 Traps . . . 27
3.7 Commit . . . 28

III Talking to the Card **29**

4 The reader base class **31**
4.1 The problem . . . 31
4.2 The idea: leave a hole in the middle of the algorithm . . . 31
4.3 Build it . . . 33
4.4 The two ports that do not survive translation . . . 37
4.5 Verify on hardware . . . 37
4.6 Traps . . . 38
4.7 Commit . . . 39

5 Protocol blocks **41**
5.1 The problem . . . 41
5.2 The idea: decide once, then let the object know what it is . . . 41
5.3 Build it . . . 42
5.4 Verify . . . 45
5.5 Inheritance versus composition . . . 46
5.6 Commit . . . 47

6 Activation **49**
6.1 The problem: the card's state machine . . . 49
6.2 Anticollision, and why it exists . . . 50

6.3 Build it . . . 51
6.4 Verify on hardware . . . 55
6.5 Traps . . . 56
6.6 Commit . . . 56

7 The APDU channel 57
7.1 The problem . . . 57
7.2 Build it . . . 58
7.3 The layer cake, complete . . . 61
7.4 Verify — your first conversation with a DESFire . . . 62
7.5 Traps . . . 63
7.6 Commit . . . 63

IV Cryptography 65

8 AES-CMAC from scratch 67
8.1 Read this before you write any cryptography . . . 67
8.2 What a MAC is . . . 67
8.3 The algorithm . . . 68
8.4 Build it . . . 69
8.5 Verify — RFC 4493 vectors . . . 73
8.6 Traps . . . 75
8.7 Commit . . . 75

9 Session keys 77
9.1 The problem: never use the long-term key directly . . . 77
9.2 The derivation . . . 78
9.3 Build it . . . 79
9.4 Two design notes worth absorbing . . . 80

9.5 Verify . . . 81
9.6 Traps . . . 82
9.7 Commit . . . 83

10 Mutual authentication **85**

10.1 The problem . . . 85
10.2 The idea: challenge–response, in both directions . . . 85
10.3 The dance . . . 86
10.4 The testability problem, and the seam that solves it . . . 87
10.5 Build it . . . 89
10.6 Verify — the full handshake, offline . . . 91
10.7 Then: on the real card . . . 93
10.8 Traps . . . 94
10.9 Commit . . . 94

11 Secure messaging and the debit **95**

11.1 The problem . . . 95
11.2 The command counter, and the ordering that will bite you . . . 96
11.3 The four building blocks . . . 96
11.4 Build it, part 1: grow `DesfireSession` . . . 97
11.5 Build it, part 2: the commands . . . 100
11.6 Why Debit and Commit are separate . . . 102
11.7 Verify — the full offline replay . . . 103
11.8 Then: the real thing . . . 104
11.9 Traps . . . 105
11.10 Commit . . . 105

V Product **107**

12 The kiosk **109**

12.1 The problem . . . 109
12.2 The idea: one interface, two implementations . . . 109
12.3 The contract . . . 110
12.4 The Adapter . . . 111
12.5 Threading: the rule you must not break . . . 114
12.6 Choosing the backend . . . 115
12.7 Verify . . . 116
12.8 Traps . . . 117
12.9 Commit . . . 117

13 The Android top-up app **119**

13.1 Why this chapter is different . . . 119
13.2 Setup . . . 120
13.3 Dart for a Java programmer . . . 121
13.4 Build it . . . 121
13.5 Provisioning: turning a blank card into a wallet . . . 125
13.6 The security reality of hardcoded keys . . . 126
13.7 Build order . . . 126
13.8 Traps . . . 127
13.9 Commit . . . 127

14 Deploying to the Pi **129**

14.1 Rotating the display . . . 129
14.2 GPU acceleration: the one flag that matters . . . 130
14.3 Autostart with systemd . . . 131
14.4 The deploy loop . . . 132
14.5 Field hygiene . . . 132

14.6 The finished machine . . . 133
14.7 Commit . . . 133

VI Appendices **135**

A Test vectors **137**
A.1 AES-CMAC — RFC 4493 (Chapter 8) . . . 137
A.2 The reference session (Chapters 9–11) . . . 138
A.3 The authentication replay (Chapter 10) . . . 138
A.4 Secure messaging (Chapter 11) . . . 139
A.5 Card personalisation . . . 141
A.6 DESFire command reference . . . 142
A.7 Status words . . . 142

B The bug museum **143**
B.1 How to debug this stack at all . . . 143
B.2 Hardware and SPI (Chapters 2–3) . . . 144
B.3 Card detection and activation (Chapters 4, 6) . . . 145
B.4 ISO-DEP and APDUs (Chapter 7) . . . 145
B.5 Cryptography (Chapters 8–11) . . . 145
B.6 Tests and tooling . . . 147
B.7 UI and deployment (Chapters 12, 14) . . . 147
B.8 Environment oddities from the real build . . . 148

C YouTube build plan **149**
C.1 The pitch . . . 149
C.2 Series structure . . . 149
C.3 Episode scripts . . . 150
C.4 Shot list . . . 154

C.5 Livestream pre-flight 154
C.6 The honest bits — say these on camera 155
C.7 Titles and thumbnails 156
C.8 Realistic schedule 156

Index **159**

Preface

There is a moment, somewhere around the middle of this book, when a small plastic card and a three-pound circuit board prove to each other that they both know the same sixteen-byte number — without either of them ever transmitting it, and while anyone within a few centimetres could be recording every bit that passes between them.

That moment is the reason this book exists. It is the same handshake that happens when you tap your bank card, open your office door, or board a train. It is not magic and it is not beyond you. It is about two hundred lines of code, and by the end of Chapter 10 you will have written all of them.

What you will build

A coffee kiosk that takes contactless payments. Touch a drink on a touchscreen, hold a card to the reader, and:

1. the reader wakes the card out of the radio field and reads its serial number;
2. your code and the card authenticate each other with AES-128;
3. your code reads the balance — encrypted, and authenticated, so a forged reply is rejected rather than believed;
4. it debits the price, commits the transaction, and the drink is dispensed.

Plus a phone app that puts credit back on.

Every layer is yours. No NFC library, no crypto library beyond a raw AES block cipher, no copied driver. You will write the SPI register encoding, the anticollision tree walk, the ISO-DEP block framing, AES-CMAC from the RFC, the session key derivation, the mutual authentication, and the secure messaging that carries the money.

Who this is for

You know C. You have blinked an LED, read a datasheet, and written `uint8_t buf[16]`. You have probably not written much Java, and you may never have seen an interface, an abstract class, or a line of cryptography.

That is exactly the intended reader. This book teaches object-oriented design and applied cryptography *as tools for a job you are actually doing*, in the order the job demands them.

> **CONCEPT No concept before the problem that needs it**
>
> You will not read "an interface is a contract between a class and its callers" as an abstract definition. You will hit a concrete wall — *I cannot test my driver without a Raspberry Pi physically plugged in* — and the interface will be the tool that gets you over it. Concepts learned that way stick, because they are attached to a memory of being stuck. Concepts learned as definitions evaporate before the exam.

Each chapter names the one design idea it introduces:

Chapter	Idea, and the problem that forces it
1	Interfaces — testing a driver with no hardware
2	Encapsulation — forty copies of a bit-twiddle
3	Abstract classes — one protocol, many reader chips
4	Polymorphism — three frame types, one call site
5	Value objects — three results that belong together
6	Layering — a seam that makes crypto testable
7	Immutability — an object that cannot be half-built
8	Enums over booleans — `derive(key, a, b, true)` means nothing
9	Injecting randomness — testing a function that is random
10	Growing a class — state and its operations belong together
11	The Adapter pattern — two fixed interfaces, made to fit
12	Porting a design — discovering what was incidental

How to use this book

Read the chapter, close the book, write the code. Every chapter gives you five things:

1. **The problem.** What does not work yet, concretely.

2. **The idea.** The design, with an analogy from outside programming.

3. **The boilerplate.** A skeleton with `TODO` comments.

4. **The verification gate.** A test with known-good values. *Do not continue until it passes.*

5. **The traps.** The specific bugs that will bite, with their symptoms.

Why the listings have holes in them

Because reading code teaches you very little and writing it teaches you almost everything.

The skeletons hand you the parts that are pure lookup — register addresses, class declarations, import lists, byte layouts — and withhold the parts that are actual reasoning: the algorithm, the control flow, the order of operations. You spend your thinking where thinking pays.

If you get genuinely stuck, a complete implementation of every file exists in the companion repository. Treat it as the back of a textbook: consulting it ends the exercise, so spend twenty minutes stuck first. *Being stuck is the learning.* The answer merely ends it.

The verification gate, and why it is not optional

Cryptographic code has a property that no code you have written before has: **wrong output is indistinguishable from right output.**

A broken checksum gives you a visible mismatch. A broken CMAC gives you sixteen bytes of perfect-looking randomness — exactly like a correct CMAC. You cannot eyeball it. You cannot infer it from a failed transaction. And if you build three more layers before testing, you will have no way to know which one is lying.

On real hardware it is worse: a card may count consecutive authentication failures and, after enough of them, lock a key permanently.

So every chapter in Part IV ends with **known-answer test vectors** — inputs with independently computed correct outputs. Reproduce them and your code is right. Fail them and it is wrong, and you fix it *now*.

The vectors in this book were generated with Python's `cryptography` library, deliberately *not* with the Java you are writing. Otherwise they would only prove your code agrees with itself.

What you need

Item	Notes	Approx.
Raspberry Pi Zero 2 W	Any Pi works; the Zero 2 W is the tight fit that teaches you to optimise	£18
MFRC522 reader module	The ubiquitous breakout board	£3
MIFARE DESFire EV2/EV3	**Not Classic, not NTAG. See below.**	£3 ea
7-inch touchscreen *(optional)*	1024×600, HDMI + USB touch	£45
Jumper wires, microSD, 5 V PSU		
An Android phone with NFC	For Part VI	

> **TRAP The card matters more than anything else on that list**
>
> It must say **DESFire EV2** or **EV3**. Not MIFARE Classic (broken crypto, entirely different protocol), not NTAG or Ultralight (no crypto at all, no value files), not "13.56 MHz RFID tag" from a bag of a hundred.
> If a listing says "compatible" or does not name the chip, it is not one. **Buy at least two** — one to develop against and one spare, because it is genuinely possible to destroy a card by changing a key and forgetting it.
> You can confirm what you have in Chapter 7: a real DESFire answers `GetVersion` with a hardware type byte of `0x33` for EV3.

Conventions

- Java 21 and Maven, package `com.midnightbrewer`. Keeping the same package as the reference repository means you can diff against it.
- Hexadecimal is lowercase and unspaced in code, spaced in prose: `91 00`.
- Coloured rules in the margin mark the four kinds of aside: concepts, analogies, traps, and verification gates.
- Every chapter ends with a suggested commit message, so your history becomes a record of the build.

One layer at a time.

PART I

Groundwork

CHAPTER 1

Setup: hardware, Pi, and an empty project

GOAL An RC522 wired to a Pi, a Java project that compiles, and one green test.

CONCEPT None yet — this chapter removes every excuse the environment can give you later.

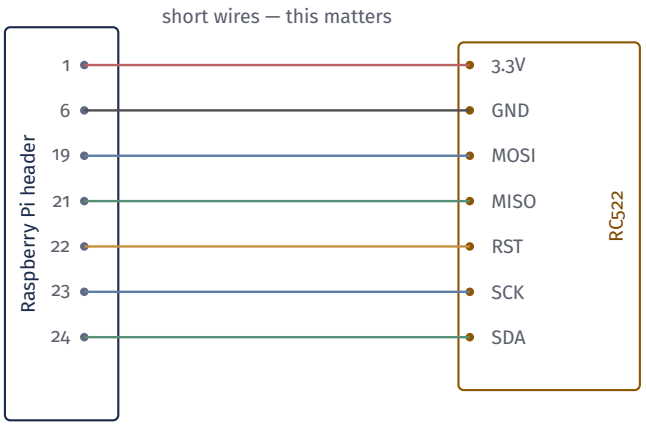
YOU END WITH `mvn compile` passing, and a chip you can talk to.

Nothing in this chapter is interesting, and that is the point. Every hour spent later wondering *"is this my code or my wiring?"* is an hour you can buy back now by making the environment boring and trustworthy.

1.1 Wiring the RC522

The RC522 speaks SPI: six signal wires plus power.

RC522 pin	Pi pin	Pi name	Why
3.3V	1	3V3	**Never 5 V.** The MFRC522 is a 3.3 V part.
RST	22	BCM 25	Hardware reset
GND	6	GND	
MISO	21	BCM 9 (MISO)	Chip → Pi
MOSI	19	BCM 10 (MOSI)	Pi → chip
SCK	23	BCM 11 (SCLK)	Clock
SDA	24	BCM 8 (CE0)	Chip select — confusingly labelled with an I^2C name
IRQ	—	not connected	We poll instead; simpler, and fast enough



> **TRAP Long wires are not a style question**
>
> At 1 MHz over 20 cm of loose DuPont jumper you get intermittent failures that look exactly like firmware bugs: a version register that reads `0x00` on one run and `0x92` on the next.
> If the chip is flaky, **shorten the wires before you debug a single line of code**. More people have lost an evening to this than to any bug in this book.

1.2 Pi OS and SPI

Flash Raspberry Pi OS (64-bit), then enable SPI:

```
sudo raspi-config       # Interface Options -> SPI -> Enable
sudo reboot
ls -l /dev/spidev0.0    # must exist after the reboot
```

`/dev/spidev0.0` is the kernel exposing "SPI bus 0, chip-select 0" as a file. That file is the entire hardware interface: everything in Part II is ultimately reads and writes to it.

Add your user to the hardware groups, or Pi4J cannot open the device:

```
sudo usermod -aG spi,gpio $USER    # log out and back in
sudo apt update
sudo apt install -y openjdk-21-jdk maven
java -version                      # expect 21.x
```

1.3 Create the project

Develop on a laptop and deploy to the Pi. Compiling on a Zero 2 W is slow enough to break your concentration.

```
mkdir midnight-brewer && cd midnight-brewer
git init
mkdir -p src/main/java/com/midnightbrewer/hardware
mkdir -p src/test/java/com/midnightbrewer/hardware
```

1.3.1 The `pom.xml`

Maven's project file. Three things matter: Java 21, JUnit 5, and Pi4J.

```
<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.midnightbrewer</groupId>
  <artifactId>MidnightBrewer</artifactId>
  <version>1.0-SNAPSHOT</version>

  <properties>
    <maven.compiler.source>21</maven.compiler.source>
    <maven.compiler.target>21</maven.compiler.target>
    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
    <pi4j.version>3.0.2</pi4j.version>
  </properties>

  <dependencies>
    <!-- Pi4J: Java access to the Pi's SPI and GPIO -->
    <dependency>
      <groupId>com.pi4j</groupId><artifactId>pi4j-core</artifactId>
      <version>${pi4j.version}</version>
    </dependency>
    <!-- the SPI provider: talks to /dev/spidev0.0 -->
    <dependency>
      <groupId>com.pi4j</groupId><artifactId>pi4j-plugin-linuxfs</artifactId>
      <version>${pi4j.version}</version>
    </dependency>
    <!-- the GPIO provider, for the reset pin -->
    <dependency>
      <groupId>com.pi4j</groupId><artifactId>pi4j-plugin-gpiod</artifactId>
      <version>${pi4j.version}</version>
    </dependency>
    <!-- Pi4J logs through SLF4J; without a binding it prints warnings -->
```

```
    <dependency>
      <groupId>org.slf4j</groupId><artifactId>slf4j-simple</artifactId>
      <version>2.0.13</version>
    </dependency>

    <dependency>
      <groupId>org.junit.jupiter</groupId><artifactId>junit-jupiter</artifactId>
      <version>5.10.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build><plugins>
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
    </plugin>
  </plugins></build>
</project>
```

> **CONCEPT `<scope>test</scope>` is a boundary, not a detail**
>
> It means "this library exists when compiling and running tests, but is not shipped with the application." Get it wrong and your kiosk drags a test framework onto the Pi. More usefully, it *enforces* the boundary: if you accidentally `import org.junit` in main code, the build fails. A dependency scope is the build system letting you state a design rule and have it checked.

```
mvn -q compile     # should print nothing and exit 0
```

There is nothing to compile yet. You have proven that Maven, Java and the network can all do their jobs — which is exactly one fewer thing that can be wrong tomorrow.

1.4 Java for a C programmer

You will meet the rest as you go, but four things trip up every C programmer on the first day.

1.4.1 Every function lives inside a class

There are no free functions. `Math.abs()` is a function `abs` inside a class `Math`. A file `Foo.java` contains `class Foo`. This feels bureaucratic, and for small programs it is — but it is also why the language can make guarantees C cannot, and Part II is where those guarantees start paying for themselves.

1.4.2 `byte` is signed, and there is no `unsigned`

This will bite you within the hour.

```
byte b = (byte) 0x92;
System.out.println(b);          // -110   <-- the same bits, read as signed
System.out.println(b & 0xFF);   // 146    <-- what you meant
```

`0x92` is `1001 0010`. With the top bit set, Java reads it as negative. The fix is the idiom `& 0xFF`, which promotes to `int` and masks off the sign extension.

> **TRAP The `& 0xFF` habit**
>
> **Any time you turn a `byte` into a number, write `& 0xFF`.** You will forget this at least once and lose an hour to a register that reads −110 instead of `0x92`. Everyone does. Now you have been warned, and you will still do it.

1.4.3 Arrays know their own length, and are bounds-checked

`arr.length` — no parentheses, it is a field. Reading past the end throws an exception naming the line, rather than silently returning whatever was next in memory. This converts C's worst class of bug into a stack trace.

1.4.4 No manual memory management

No `malloc`, no `free`. `new byte[16]` gives you sixteen *zeroed* bytes, and they vanish when nothing refers to them. The zeroing matters more than you would think: several structures in this book are "a 16-byte buffer that is mostly zeros", and you get that for free.

One C habit transfers perfectly: bit manipulation. «, », &, |, ^ and ~ all mean exactly what they mean in C. The register-poking half of this project will feel like home.

1.5 Commit

```
git add -A
git commit -m "chore: empty Maven project, Java 21, Pi4J + JUnit"
```

PART II

Talking to the Chip

CHAPTER 2

The SPI seam

GOAL Define what "an SPI bus" means to your program, build a fake one, and test against it with no hardware attached.

CONCEPT Interfaces, and dependency injection.

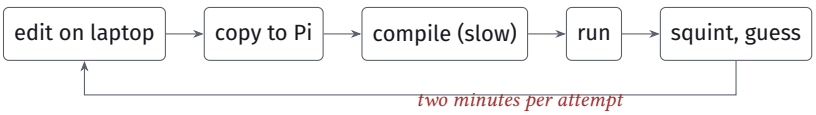
YOU END WITH A green test that runs with the reader unplugged.

2.1 The problem

You are about to write a driver that pokes RC522 registers. Before you write a line of it, ask an uncomfortable question:

> *How will I know the driver is correct?*

The obvious answer — *plug in the Pi and see* — will cost you the project. Consider the loop it commits you to:



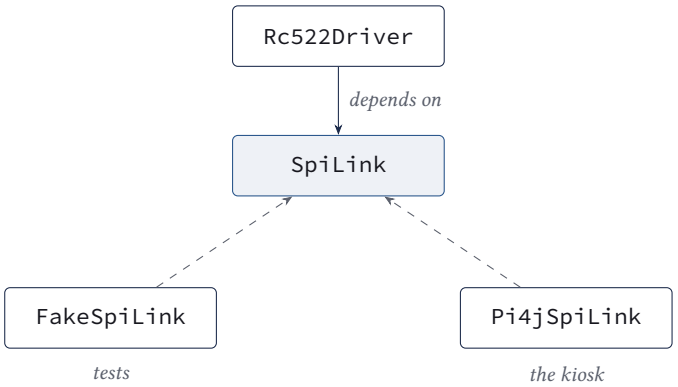
And every failure has three possible causes — your logic, your wiring, the chip — with no way to tell them apart. You want a loop that takes 200 milliseconds and has exactly *one* possible cause: your logic.

2.2 The idea: program against a description, not a thing

Your driver does not actually need "a Raspberry Pi SPI bus". Look at what it truly needs: *something that takes bytes out and hands bytes back.* That is the entire requirement. A real Pi satisfies it. So does a lookup table you write in ten lines.

So rather than have the driver depend on a concrete Pi4J class, write that *requirement* down as a type of its own — an **interface** — and have the driver depend on the requirement.

> **ANALOGY The wall socket**
>
> A wall socket is an interface. Your kettle does not depend on "the Hinkley Point C nuclear reactor"; it depends on "230 V at 50 Hz through two holes shaped like this". That is why the same kettle works on a generator, a solar inverter, or a test rig in the factory. The socket standard is a *contract*: it says nothing about how the electricity was made, only what shows up at the holes. Every appliance ever built works with every power source ever built, because both sides code against the socket rather than against each other.
>
> An interface is a socket. `SpiLink` is the socket shape, `Pi4jSpiLink` is the power station, and `FakeSpiLink` is the bench supply you use to test the kettle without burning coal.



In C you would get halfway there with a struct of function pointers, and the idea is genuinely the same. Java's version is checked by the compiler: a class that claims to satisfy the contract but forgets a method does not compile.

2.3 Build it

2.3.1 `SpiException` — the failure channel

SPI can fail: the device file may not exist, a transfer can error. In C you would return −1 and hope the caller checks.

```
package com.midnightbrewer.hardware;

/**
 * Something went wrong talking to the SPI bus.
 *
 * Deliberately a CHECKED exception (extends Exception, not RuntimeException):
 * the compiler forces every caller to either handle it or declare that it
 * throws too. Hardware failure is not an edge case you get to forget about,
 * so we make the compiler nag.
 */
public class SpiException extends Exception {

    public SpiException(String message) {
        super(message);
    }

    public SpiException(String message, Throwable cause) {
        super(message, cause);   // keeps the original error's stack trace
    }
}
```

> **CONCEPT Checked exceptions are a design statement**
>
> Java has two flavours of exception. *Unchecked* ones (`RuntimeException`) can be ignored silently. *Checked* ones must be acknowledged — a caller either catches it or adds `throws SpiException` to its own signature.
>
> That distinction lets you encode a rule: *the bus can fail, and you may not forget that.* The cost is that `throws SpiException` propagates up your call stack, which is a little noisy. But that noise is an accurate map of everything in your program that can fail because hardware exists, and that map turns out to be useful.

The two-argument constructor matters more than it looks. `super(message, cause)` preserves the underlying error, so when Pi4J throws some `IOException` deep inside, you wrap it with your own context while keeping the original stack trace. Debug output becomes "my error, *caused by* their error" rather than one or the other.

2.3.2 `SpiLink` — the contract

```java
package com.midnightbrewer.hardware;

/**
 * The seam between this program and an SPI bus.
 *
 * An interface is a PROMISE, not code: any class that says "implements
 * SpiLink" must supply these methods, and the compiler enforces it.
 *
 * Everything above this line -- the driver, the reader, the whole card stack
 * -- depends only on this promise. Never on Pi4J, never on a device file.
 *
 * Extends AutoCloseable so callers can use try-with-resources and the RF
 * field is never left energised by a crash.
 */
public interface SpiLink extends AutoCloseable {

    /**
     * Full-duplex SPI transfer: shift out txBuffer, shift in rxBuffer.
     *
     * SPI is inherently full duplex -- every clock pulse sends a bit AND
     * receives a bit. So this is ONE method, not a send() and a receive().
     * The API mirrors the hardware rather than pretending it is a stream.
     */
    void transfer(byte[] txBuffer, byte[] rxBuffer, int length) throws SpiException;

    /** Release the bus. Overridden to narrow the exception type. */
    @Override
    void close() throws SpiException;
}
```

Notice there is **no code here at all** — no fields, no bodies. An interface is pure vocabulary, which is exactly why it makes a good seam: there is nothing in it that could accidentally depend on a Pi.

> **GOING DEEPER The signature is a design decision**
>
> `transfer(tx, rx, length)` mirrors SPI's full-duplex reality: one call, both directions. Had it been `send()` and `receive()`, every caller would need to know that a "read" is really *write the address, then clock out a dummy byte to shift the answer in* — a hardware detail leaking into every call site.
> Design the interface around what the hardware genuinely *is*, and the code above it

> stays honest. Design it around what feels convenient today, and you spend next year translating.

2.3.3 `FakeSpiLink` — the test double

This lives in `src/test/java/`, so it never ships.

```
package com.midnightbrewer.hardware;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

/**
 * A fake SPI bus: answers from a canned table, records what it was sent.
 *
 * This is the payoff of the interface. The driver cannot tell this from a real
 * Pi -- it only ever saw the SpiLink promise, and this keeps that promise.
 */
class FakeSpiLink implements SpiLink {

    /** Register address -> value the fake chip should report. */
    private final Map<Integer, Integer> registers = new HashMap<>();

    /** Every tx buffer we were handed, so a test can assert on it. */
    final List<byte[]> sent = new ArrayList<>();

    private boolean closed = false;

    void setRegister(int address, int value) { registers.put(address, value); }

    boolean isClosed() { return closed; }

    @Override
    public void transfer(byte[] txBuffer, byte[] rxBuffer, int length) {
        sent.add(txBuffer.clone());

        // TODO 1: decode the address byte the driver sent.
        //   A READ is encoded as ((addr << 1) & 0x7E) | 0x80, so:
        //     address = (txBuffer[0] & 0x7E) >> 1
        //     isRead  = (txBuffer[0] & 0x80) != 0

        // TODO 2: if it was a read, put the canned value in rxBuffer[1].
```

```
        //   The answer arrives on the SECOND byte: the first byte out is the
        //   address, and while that goes out the chip has nothing to say yet.
        //   Default to 0 for an address nobody configured.

        // TODO 3: if it was a write, there is nothing to answer.
    }

    @Override
    public void close() { closed = true; }
}
```

TRAP A fake that returns nothing

If you leave `transfer()` empty, `rxBuffer` stays all zeros, your driver dutifully returns `0`, and your test asserting `version() == 0x92` fails — so you go hunting for a bug in the driver that is not there.
Worse is the opposite: a test that *passes* because both sides are trivially empty.
A fake must be a plausible liar, not a mute. It has to behave enough like the real thing that the code under test is genuinely exercised.

2.3.4 The test

```
package com.midnightbrewer.hardware;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class FakeSpiLinkTest {

    @Test
    void readReturnsTheCannedValue() throws SpiException {
        FakeSpiLink link = new FakeSpiLink();
        link.setRegister(0x37, 0x92);          // VersionReg answers 0x92

        // Hand-encode a READ of register 0x37, exactly as the driver will.
        byte[] tx = { (byte) (((0x37 << 1) & 0x7E) | 0x80), 0x00 };
        byte[] rx = new byte[2];
        link.transfer(tx, rx, 2);

        assertEquals(0x92, rx[1] & 0xFF);      // & 0xFF -- the signed-byte thing
        assertEquals(1, link.sent.size());
    }
```

```
    @Test
    void closeIsRecorded() throws Exception {
        FakeSpiLink link = new FakeSpiLink();
        assertTrue(!link.isClosed());
        link.close();
        assertTrue(link.isClosed());
    }
}
```

VERIFICATION GATE

```
mvn test -Dtest=FakeSpiLinkTest
```

Green — *with the reader unplugged.* Pause on that: you have just tested hardware-facing code with no hardware in the room.

2.3.5 `Pi4jSpiLink` — the real one

```
package com.midnightbrewer.hardware;

import com.pi4j.Pi4J;
import com.pi4j.context.Context;
import com.pi4j.io.spi.Spi;
import com.pi4j.io.spi.SpiMode;

/**
 * The real SPI bus, via Pi4J and /dev/spidev0.0.
 *
 * The ONLY class in the project that knows Pi4J exists. Everything else sees
 * the SpiLink interface. Contain your dependencies in one place and swapping
 * them later is a one-file job.
 */
public class Pi4jSpiLink implements SpiLink {

    private static final int SPI_CHANNEL = 0;         // CE0
    private static final int SPI_BAUD    = 1_000_000; // 1 MHz

    private final Context pi4j;
    private final Spi spi;

    public Pi4jSpiLink() {
```

```
        this.pi4j = Pi4J.newAutoContext();
        this.spi = pi4j.create(Spi.newConfigBuilder(pi4j)
                .id("rc522")
                .address(SPI_CHANNEL)
                .baud(SPI_BAUD)
                .mode(SpiMode.MODE_0)   // RC522 is mode 0: CPOL=0, CPHA=0
                .build());
    }

    @Override
    public void transfer(byte[] txBuffer, byte[] rxBuffer, int length)
            throws SpiException {
        try {
            spi.transfer(txBuffer, 0, rxBuffer, 0, length);
        } catch (RuntimeException e) {
            throw new SpiException("SPI transfer failed", e);
        }
    }

    @Override
    public void close() throws SpiException {
        try {
            spi.close();
            pi4j.shutdown();
        } catch (RuntimeException e) {
            throw new SpiException("failed to close SPI", e);
        }
    }
}
```

Why 1 MHz? The MFRC522 handles 10 MHz on a clean PCB. On loose jumper wires it does not. 1 MHz is fast enough that SPI is never your bottleneck — the card's RF timing dominates by orders of magnitude — and slow enough to tolerate imperfect wiring. Optimise the thing that is actually slow, not the thing that is easy to measure.

Why mode 0? CPOL $= 0$, CPHA $= 0$, from the datasheet: clock idles low, data sampled on the rising edge. Get this wrong and you read plausible-looking garbage, which is the worst kind of wrong.

2.4 The concept, now that you have used it

Dependency injection is the name for what you just did: `Rc522Driver` will receive its `SpiLink` from outside, through its constructor, rather than constructing one itself.

```
// If the driver did this, it would be welded to the Pi forever:
class Rc522Driver {
    private final SpiLink link = new Pi4jSpiLink();   // untestable
}

// Handing it in from outside means the CALLER decides:
class Rc522Driver {
    private final SpiLink link;
    Rc522Driver(SpiLink link) { this.link = link; }   // testable
}
```

Same code, two power sources. The kettle does not own the power station.

> **CONCEPT A rule worth internalising**
>
> **A class should not construct the things it depends on. It should be handed them.** "Who decides?" moves up to whoever assembles the program, and testability, flexibility and clear ownership all follow from that one move. When you meet a class that is hard to test, look first for a `new` in its constructor.

2.5 Traps

You see	Cause / fix
`version()` returns −110	Missing `& 0xFF`. `0x92` as a signed byte is −110.
Fake test passes, hardware returns 0	The fake answers unconditionally; make it decode the address.
`NoClassDefFoundError: com/sun/jna/...`	JNA missing from the classpath. Run `mvn dependency:copy-dependencies`.
Works once, fails on the second run	You did not `close()`; the device file stays locked. Use try-with-resources.
`Permission denied` on `/dev/spidev0.0`	Not in the `spi` group. `sudo usermod -aG spi,gpio $USER`.

2.6 Commit

```
git add -A
git commit -m "M1: SpiLink interface + fake and Pi4J implementations"
```

CHAPTER 3

The RC522 driver

GOAL Turn “shift bytes over SPI” into “read and write named registers”.

CONCEPT Encapsulation, and composition (HAS-A).

YOU END WITH A real chip reporting `version() == 0x92`.

3.1 The problem

Reading one RC522 register over raw SPI looks like this:

```
byte[] tx = { (byte) (((0x37 << 1) & 0x7E) | 0x80), 0x00 };
byte[] rx = new byte[2];
link.transfer(tx, rx, 2);
int version = rx[1] & 0xFF;
```

Five lines, three magic numbers, and a bit-twiddle you must get exactly right. Now picture that at every one of the roughly forty register accesses in a card transaction: two hundred lines of identical incantation, each an opportunity to type `0x7F` where you meant `0x7E`.

3.2 The idea: gather the mess in one place

That is what a class is, at its most useful: **a place to put the mess so the rest of the program does not have to know about it.** After this chapter the rest of your program says

```
int version = driver.version();
driver.antennaOn();
```

and the address encoding exists in exactly one place, written once, correct once.

> **ANALOGY The steering wheel**
>
> A car's steering wheel is an interface to a mess: a rack, a pinion, hydraulic assist, tie rods, geometry that changes with speed. You turn a wheel.
> You are not permitted to reach into the steering column mid-corner and adjust the rack — not because it is secret, but because the mechanism only works if it is driven through the one sanctioned control.
> `private` is that housing. It is not about hiding secrets; it is about **guaranteeing that the only way to affect this thing is through the paths you have thought about.**

3.2.1 The address encoding

From the MFRC522 datasheet, the first SPI byte is an address byte:



A register number must be shifted left one place to land in bits 6–1:

```
((register << 1) & 0x7E) | 0x80    // read
 (register << 1) & 0x7E            // write
```

`& 0x7E` is `0111 1110`: it masks off anything that overflowed past bit 6 and forces bit 0 to zero. `| 0x80` sets the read flag. That is the whole protocol, and after this chapter you never write it again.

3.3 Build it

```
package com.midnightbrewer.hardware;

/**
 * The RC522 register-level driver.
 *
 * Encapsulation earns its keep here: the address encoding, the read/write bit
 * and the two-byte SPI dance all live IN HERE. The rest of the program calls
 * readRegister(0x37) and never writes ((reg << 1) & 0x7E) | 0x80 again.
```

```
 */
public class Rc522Driver {

    // ---- register addresses (MFRC522 datasheet, section 9) ----
    private static final int COMMAND_REG    = 0x01;  // CommandReg
    private static final int TX_CONTROL_REG = 0x14;  // TxControlReg (antenna)
    private static final int VERSION_REG    = 0x37;  // VersionReg (0x91 / 0x92)

    private static final int PCD_SOFT_RESET = 0x0F;  // the SoftReset command

    // ---- composition: the driver HAS-A SpiLink ----
    //
    // 'private' = nobody outside can reach past the driver and touch the bus.
    // 'final'   = once set in the constructor, this never points elsewhere.
    // The type is the INTERFACE, so the driver cannot tell real from fake.
    private final SpiLink link;

    // Dependency injection: the bus is handed in, not constructed here.
    public Rc522Driver(SpiLink link) {
        this.link = link;
    }

    // ======================================================================
    // readRegister
    //
    //   C:  uchar Read_MFRC522(uchar addr) {
    //           SPI_Transfer( ((addr<<1)&0x7E) | 0x80 );  // address, read bit
    //           val = SPI_Transfer(0x00);                 // 2nd byte = answer
    //           return val;
    //       }
    //
    //   In Java ONE transfer() moves both bytes: build a 2-byte tx (encoded
    //   address, then 0x00), a 2-byte rx, call link.transfer, answer in rx[1].
    //   Return it masked with & 0xFF so 0x92 comes back as 146, not -110.
    // ======================================================================
    public int readRegister(int register) throws SpiException {
        // TODO
        return 0;
    }

    // ======================================================================
    // writeRegister
    //
    //   Same shape, two differences: the address byte does NOT set 0x80
    //   (that is what makes it a write), and the second byte is the value
    //   instead of 0x00. Nothing to read back.
    // ======================================================================
```

```
    public void writeRegister(int register, int value) throws SpiException {
        // TODO
    }

    // ================================================================
    // setBitMask -- turn bits ON without disturbing the others.
    //
    //   C:  tmp = Read(reg);  Write(reg, tmp | mask);
    //
    //   "Read-modify-write". Build it from the two methods you just wrote,
    //   NOT from link.transfer. Each layer uses only the layer below it.
    // ================================================================
    public void setBitMask(int register, int mask) throws SpiException {
        // TODO
    }

    /** Turn bits OFF: same shape, but AND with ~mask. */
    public void clearBitMask(int register, int mask) throws SpiException {
        // TODO
    }

    /** Write the SoftReset command to CommandReg. One line. */
    public void softReset() throws SpiException {
        // TODO
    }

    // ================================================================
    // antennaOn -- power up the RF field.
    //
    //   C:  Read_MFRC522(TxControlReg);      // the C reads and ignores it
    //       SetBitMask(TxControlReg, 0x03);  // Tx1RFEn | Tx2RFEn
    //
    //   The two low bits of TxControlReg are the antenna drivers. Watch the
    //   register go 0x80 -> 0x83; that is read-modify-write doing its job.
    // ================================================================
    public void antennaOn() throws SpiException {
        // TODO
    }

    /** The chip's version byte: 0x91 (v1.0) or 0x92 (v2.0). */
    public int version() throws SpiException {
        // TODO
        return 0;
    }
}
```

> **CONCEPT Each layer uses only the layer beneath it**
>
> `setBitMask` could call `link.transfer` directly and it would work. Do not.
> If you shortcut a layer you have created a second path to the same hardware — and the day you need to add logging, retries, or a mutex to register access, one path gets it and the other silently does not. That is not a hypothetical; it is precisely how "I fixed the bug but it still happens sometimes" bugs are born.

3.4 Verify — with no hardware

```
package com.midnightbrewer.hardware;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;

class Rc522DriverTest {

    @Test
    void readsTheVersionRegister() throws SpiException {
        FakeSpiLink link = new FakeSpiLink();
        link.setRegister(0x37, 0x92);

        Rc522Driver driver = new Rc522Driver(link);

        assertEquals(0x92, driver.version());
    }

    @Test
    void writeEncodesTheAddressWithoutTheReadBit() throws SpiException {
        FakeSpiLink link = new FakeSpiLink();
        Rc522Driver driver = new Rc522Driver(link);

        driver.writeRegister(0x14, 0x03);

        byte[] tx = link.sent.get(0);
        assertEquals((0x14 << 1) & 0x7E, tx[0] & 0xFF);   // read bit CLEAR
        assertEquals(0x03, tx[1] & 0xFF);
    }
}
```

The second test is the interesting one: it asserts on **what went out on the wire**, which is the only way to catch an encoding bug. A test that checked only return values would happily

pass with the read bit set on a write.

> **VERIFICATION GATE**
>
> ```
> mvn test -Dtest=Rc522DriverTest
> ```

3.5 Now the real chip

```
package com.midnightbrewer.hardware;

public final class HardwareCheck {

    private HardwareCheck() { }

    public static void main(String[] args) throws Exception {
        System.out.println("Opening the reader...");

        try (Pi4jSpiLink link = new Pi4jSpiLink()) {
            Rc522Driver driver = new Rc522Driver(link);

            driver.softReset();
            Thread.sleep(50);              // the chip needs a moment after reset
            driver.antennaOn();

            int version = driver.version();
            System.out.printf("version() = 0x%02X%n", version);
            if (version == 0x91 || version == 0x92) {
                System.out.println("  -> the driver works on silicon.");
            } else {
                System.out.println("  -> chip not answering. Check wiring/power.");
            }
        }
    }
}
```

```
# on your laptop
rsync -av --exclude target/ ./ martin@raspberrypi.local:~/midnight-brewer/

# on the Pi
cd ~/midnight-brewer
```

```
mvn -q -DskipTests compile dependency:copy-dependencies -DincludeScope=runtime
java -cp "target/classes:target/dependency/*" \
     com.midnightbrewer.hardware.HardwareCheck
```

VERIFICATION GATE

```
version() = 0x92
  -> the driver works on silicon.
```

3.5.1 If you get `0x00` or `0xFF`

Those are the two "nobody home" values, and they are diagnostic:

- `0x00` — MISO stuck low. Usually MISO not connected, chip unpowered, or CS on the wrong pin.
- `0xFF` — MISO stuck high, i.e. floating. Usually MISO connected to nothing, or a dead chip.

Check power first (3.3 V, *not* 5 V), then wiring, then shorten your wires. Do not debug the code: it is ten lines and your test already proved it.

> **GOING DEEPER try-with-resources, and why `AutoCloseable` mattered**
>
> `try (Pi4jSpiLink link = ...)` calls `close()` when the block exits — normally, by exception, or by early return. That is why `SpiLink extends AutoCloseable` back in Chapter 2.
> On a kiosk it means a crash can never leave the RF field energised, which is the sort of thing that quietly cooks a reader module overnight. Resource cleanup that depends on remembering to write it is resource cleanup that eventually does not happen.

3.6 Traps

You see	Cause / fix
`version()` $= -110$	Missing `& 0xFF`.
`version()` $=$ `0x00`	MISO low: power, wiring, or wrong CS pin.

You see	Cause / fix
`version()` = `0xFF`	MISO floating: not connected.
Reads work, writes do not	Read bit `0x80` accidentally set on writes.
Fine on the bench, flaky when mounted	Wire length. Shorten them.
`0x92` in the test, `0x00` on the Pi	Stale `target/`. Run `mvn clean compile`.

3.7 Commit

```
git add -A
git commit -m "M2: Rc522Driver -- register read/write, bitmasks, reset, antenna"
```

PART III

Talking to the Card

CHAPTER 4

The reader base class

GOAL Send a frame to a card and get its answer back.

CONCEPT Abstract classes, and the Template Method pattern.

YOU END WITH `isCardPresent()` detecting a real DESFire.

4.1 The problem

You can read registers. Now you need to *talk to a card*, which means a fixed sequence: tell the chip how many bits are in the last byte, flush the FIFO, clear old interrupt flags, load the outgoing bytes, start the transceive command, wait for a reply or a timeout, check the error register, and read the reply out.

Here is the thing: **those steps are RC522-specific, but the protocol above them is not.** "Send REQA, expect a two-byte ATQA" is ISO-14443 — identical on a PN532, a PN5180, or an ACR122. Write the protocol logic *inside* an RC522 class and you must rewrite all of it to support a second chip.

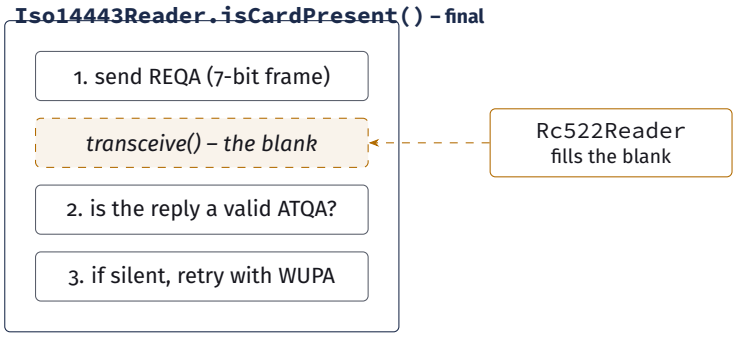
4.2 The idea: leave a hole in the middle of the algorithm

Write the protocol once, in a base class, and leave the chip-specific step **blank** — a method with a name and a signature but no body. Subclasses fill the blank. The base class calls the blank as if it were already written.

That is the **Template Method** pattern: the parent owns the *shape* of the algorithm, the children supply the *steps*.

ANALOGY The recipe with one variable step

A stir-fry recipe: heat the oil, **cook the protein**, add vegetables, add sauce, serve. The sequence is fixed — you never add sauce before the oil is hot.
But "cook the protein" is a hole. Chicken takes six minutes, prawns two, tofu wants pressing first. The cookbook writes the sequence; the specific dish fills the hole.
Critically, the recipe *calls* the hole without knowing what goes in it. `isCardPresent()` calls `transceive()`, and at runtime that call lands in `Rc522Reader`, or `Pn532Reader`, or a fake. The base class never learns which.



4.2.1 Why an abstract class and not an interface

An interface holds method signatures but no state. Here you need *both*: real shared code (the protocol) *and* a blank (the chip step). That mix is exactly what an abstract class is for.

	interface	abstract class
Method signatures	yes	yes
Real method bodies	mostly no	yes
Fields / state	no	yes
Can be instantiated	no	no
A class can have many	yes	no (one parent)

CONCEPT Which one to reach for

Interface = "can do this" — a capability, like `SpiLink`.
Abstract class = "is a kind of this" — a family sharing machinery, like `Iso14443Reader`.

4.3 Build it

4.3.1 The base

```
package com.midnightbrewer.hardware;

/**
 * An abstract base for any ISO-14443 reader.
 *
 * The protocol logic is a standard: identical for an RC522, a PN532, anything.
 * That logic lives HERE, written once. What differs per chip is only how you
 * physically push a frame out and read the reply -- left BLANK for each chip's
 * subclass to fill.
 */
public abstract class Iso14443Reader {

    // ---- ISO 14443-3 constants (chip-independent) ----
    private static final byte REQA = 0x26; // "any IDLE card, announce yourself"
    private static final byte WUPA = 0x52; // "any IDLE *or* HALTed card, wake up"

    /** A valid ATQA answer is exactly two bytes. */
    private static final int ATQA_LENGTH = 2;

    /** REQA/WUPA are 7-bit short frames, not whole bytes. */
    private static final int REQA_BITS = 7;

    // ================================================================
    // THE BLANK. 'abstract' = declared, no body. A subclass MUST supply
    // one or it will not compile.
    //
    //   sendData       : bytes to transmit
    //   bitsInLastByte : valid bits in the final byte (7 for REQA, 0 = all 8)
    //   returns        : bytes received, or an EMPTY array if nothing answered.
    //
    // Note "empty array" rather than an exception: a timeout is NORMAL when no
    // card is on the antenna. Exceptions are for the exceptional.
    // ================================================================
    protected abstract byte[] transceive(byte[] sendData, int bitsInLastByte)
            throws SpiException;

    /**
     * The second blank: compute the 2-byte CRC_A over data.
     *
     * SELECT and RATS need a CRC appended; anticollision does not. The RC522
     * has a hardware CRC unit, another chip might not -- so, like transceive,
     * it is left for the subclass. An abstract class may carry as many blanks
```

```
     * as the design needs.
     *
     * @return two CRC bytes, low first, ready to append.
     */
    protected abstract byte[] calculateCrc(byte[] data) throws SpiException;

    // ================================================================
    // THE TEMPLATE METHOD -- the fixed algorithm, shared by all chips.
    //
    // 'final' on purpose: a subclass may fill the blanks, but may NOT rewrite
    // the sequence. The protocol is the protocol.
    // ================================================================
    public final boolean isCardPresent() throws SpiException {
        // TODO 1: try REQA; if that finds a card, return true.
        // TODO 2: otherwise try WUPA and return that result.
        //
        //   Why both? REQA only wakes cards in IDLE. A card you already talked
        //   to sits in HALT and ignores REQA forever -- WUPA wakes those too.
        //   Without this, the reader works ONCE and then goes blind until the
        //   card is physically removed and replaced.
        return false;
    }

    /** Sends one REQA/WUPA and reports whether a well-formed ATQA came back. */
    private boolean request(byte mode) throws SpiException {
        // TODO: transceive a single-byte frame { mode } with REQA_BITS bits,
        //       and return whether the reply length == ATQA_LENGTH.
        return false;
    }
}
```

CONCEPT `public`, `protected`, `private`

`protected` means "visible to this class and its subclasses, but not the outside world". `transceive` is an implementation detail of the reader family — the kiosk should never call it.

`public` for the vocabulary, `protected` for the family, `private` for the class alone. Choosing these deliberately is most of what designing a class *is*.

And `final` on `isCardPresent()` encodes a design decision in the type system: *chips vary, the standard does not.* Deciding what subclasses may and may not change is the real work of writing a base class.

4.3.2 The RC522 subclass

```
package com.midnightbrewer.hardware;

/**
 * The RC522's answer to the two blanks in Iso14443Reader.
 *
 * 'extends' = inheritance: this IS-A Iso14443Reader and gets all its protocol
 * code for free. It supplies only the two chip-specific steps.
 */
public class Rc522Reader extends Iso14443Reader {

    // ---- registers this needs ----
    private static final int COMMAND_REG     = 0x01;
    private static final int COMM_IEN_REG    = 0x02; // which IRQs are enabled
    private static final int COMM_IRQ_REG    = 0x04; // which IRQs have fired
    private static final int ERROR_REG       = 0x06;
    private static final int FIFO_DATA_REG   = 0x09; // the FIFO, in and out
    private static final int FIFO_LEVEL_REG  = 0x0A; // bytes currently in it
    private static final int BIT_FRAMING_REG = 0x0D; // TxLastBits + StartSend

    private static final int PCD_IDLE       = 0x00; // "cancel / do nothing"
    private static final int PCD_TRANSCEIVE = 0x0C; // "send the FIFO, receive"

    // ---- CRC coprocessor ----
    private static final int DIV_IRQ_REG      = 0x05; // CRCIrq is bit 2
    private static final int CRC_RESULT_REG_L = 0x22;
    private static final int CRC_RESULT_REG_H = 0x21;
    private static final int PCD_CALCCRC      = 0x03;

    // COMPOSITION again: the reader HAS-A driver. Note it does NOT extend
    // Rc522Driver -- a reader is not a kind of driver, it USES one.
    private final Rc522Driver driver;

    public Rc522Reader(Rc522Driver driver) {
        this.driver = driver;
    }

    @Override
    protected byte[] transceive(byte[] sendData, int bitsInLastByte)
            throws SpiException {

        // ---- 1. how many bits are valid in the last byte ----
        //   C: Write(BitFramingReg, 0x07);   // 7 for REQA
        // TODO: writeRegister(BIT_FRAMING_REG, bitsInLastByte & 0x07);

        // ---- 2. arm the chip and empty the FIFO ----
```

```
        //   C: Write(CommIEnReg, 0x77|0x80);   // enable IRQs
        //      ClearBitMask(CommIrqReg, 0x80); // clear old flags
        //      SetBitMask(FIFOLevelReg, 0x80); // flush FIFO
        //      Write(CommandReg, PCD_IDLE);    // cancel running command
        // TODO: those four calls.

        // ---- 3. load the outgoing bytes into the FIFO ----
        // TODO: a loop writing each byte of sendData to FIFO_DATA_REG
        //       (as an int: sendData[i] & 0xFF).

        // ---- 4. run it, and press "start sending" ----
        //   C: Write(CommandReg, PCD_TRANSCEIVE);
        //      SetBitMask(BitFramingReg, 0x80);   // StartSend = bit 7
        // TODO: those two.

        // ---- 5. wait for the answer, or a timeout ----
        //   The C spins a large counter. That does NOT port -- see the note
        //   below. Use a WALL-CLOCK deadline:
        //
        //   long deadline = System.nanoTime() + 50_000_000L;   // ~50 ms
        //   int irq;
        //   do {
        //       irq = driver.readRegister(COMM_IRQ_REG);
        //   } while (System.nanoTime() < deadline
        //            && (irq & 0x01) == 0      // TimerIRq -> nobody answered
        //            && (irq & 0x30) == 0);    // RxIRq|IdleIRq -> done
        // TODO: the loop. Keep 'irq' -- step 6 needs it.

        // ---- 6. stop sending, then check for trouble ----
        //   C: ClearBitMask(BitFramingReg, 0x80);
        //      if (Read(ErrorReg) & 0x1B) ...   // overflow, parity, CRC, protocol
        //   Also: if TimerIRq fired, nobody answered.
        //   Either way return an EMPTY array -- "no card" is not an error.
        // TODO.

        // ---- 7. read the reply out of the FIFO ----
        //   C: n = Read(FIFOLevelReg);
        //      for (i=0;i<n;i++) backData[i] = Read(FIFODataReg);
        // TODO: read the level into n, allocate byte[n], loop reading
        //       FIFO_DATA_REG into each slot, return it.
        //
        //   Note: you read the SAME register n times. It is a FIFO -- each
        //   read pops the next byte. This looks wrong and is right.
        return new byte[0];
    }

    @Override
```

```
    protected byte[] calculateCrc(byte[] data) throws SpiException {
        //   C: Write(CommandReg, PCD_IDLE);
        //      ClearBitMask(DivIrqReg, 0x04);   // clear CRCIrq
        //      SetBitMask(FIFOLevelReg, 0x80);  // flush FIFO
        //      for (i..) Write(FIFODataReg, data[i]);
        //      Write(CommandReg, PCD_CALCCRC);
        //      wait for DivIrqReg bit 2
        //      out[0] = Read(CRCResultRegL);  out[1] = Read(CRCResultRegH);
        //
        // TODO all of it. Same wall-clock trick (~20 ms is plenty).
        // Return { low, high }: ISO-14443 puts the CRC on the wire low byte
        // first, and returning it wire-ready means callers just append it.
        return new byte[2];
    }
}
```

4.4 The two ports that do not survive translation

> **TRAP Iteration counts become deadlines**
>
> The single most important lesson in porting embedded C to Linux.
> `for (i = 0; i < 2000; i++)` on bare metal is microseconds. Here, each iteration is a `readRegister`, which is a syscall through the kernel — on the order of 50 µs. The same loop count runs for *minutes*.
> Always time out on `System.nanoTime()`, never on a loop counter. If your program appears to hang, this is the first thing to check.

Second: **chip-select is no longer yours**. The C toggles CS around each byte. Your `SpiLink.transfer()` already performs one complete, atomic transaction. Try to manage CS yourself and you will fight the kernel driver and lose.

4.5 Verify on hardware

Before polling, the chip needs a full init sequence — the datasheet defaults are not usable:

```
private static void initChip(Rc522Driver driver) throws SpiException {
    driver.writeRegister(0x2A, 0x8D); // TModeReg     } response timer,
    driver.writeRegister(0x2B, 0x3E); // TPrescalerReg} ~300 ms at 6.78 MHz
    driver.writeRegister(0x2C, 0x02); // TReloadRegH  } (0x0258 = 600 ticks)
    driver.writeRegister(0x2D, 0x58); // TReloadRegL  }
```

```
    driver.writeRegister(0x15, 0x40); // TxASKReg  -> force 100% ASK modulation
    driver.writeRegister(0x11, 0x3D); // ModeReg   -> CRC preset 0x6363
    driver.writeRegister(0x26, 0x08); // RFCfgReg  -> receiver gain
    driver.antennaOn();
}
```

Two of those matter enormously:

- `TxASKReg` = `0x40` forces 100 % ASK modulation, which Type A cards require. Without it the card never sees a valid signal, and you get a silence that looks exactly like a wiring fault.
- `ModeReg` = `0x3D` sets the CRC preset to `0x6363`, the ISO-14443-A value. Wrong preset means every CRC you compute is wrong, so every SELECT is rejected — in Chapter 6, not here, which makes it maddening to trace.

```
Rc522Reader reader = new Rc522Reader(driver);
System.out.println("Polling 15x (put a card on the antenna)...");
int hits = 0;
for (int i = 0; i < 15; i++) {
    if (reader.isCardPresent()) hits++;
    Thread.sleep(40);
}
System.out.printf("card detected in %d / 15 polls%n", hits);
```

VERIFICATION GATE

```
card detected in 15 / 15 polls
```

If you get `0/15` with a card sitting on the antenna, check the init sequence first — especially those two registers — then confirm the card is actually 13.56 MHz.

4.6 Traps

You see	Cause / fix
Runs for minutes, no output	Ported the C's iteration counter. Use a `nanoTime` deadline.
`0/15` polls, chip version fine	Missing `TxASKReg`=`0x40` or `ModeReg`=`0x3D`.

You see	Cause / fix
Detects once, then never again	Only sending REQA. Halted cards need WUPA.
Detection works about half the time	Wire length, or card not flat on the antenna.
`NegativeArraySizeException`	FIFO level read before the reply arrived — check the wait loop.

4.7 Commit

```
git add -A
git commit -m "M3: Iso14443Reader template method + Rc522Reader transceive/CRC"
```

CHAPTER 5

Protocol blocks

GOAL Model the three ISO-14443-4 frame types as a small class hierarchy.

CONCEPT Inheritance, polymorphism, and the factory method.

YOU END WITH A three-class hierarchy with tests, and one place where type is decided.

5.1 The problem

Once a card is activated you exchange **T=CL blocks**. Every frame starts with a PCB (Protocol Control Byte) whose top bits say which of three kinds it is:

PCB bits	Type	Carries	Purpose
`0xxxxxxx`	**I-block** (Information)	your APDU	the actual data
`10xxxxxx`	**R-block** (Receive ready)	nothing	acknowledgement
`11xxxxxx`	**S-block** (Supervisory)	control info	“wait”, “deselect”

The C approach is a switch statement, and it works fine — *once*. The problem is what happens over time. You need `describe()` for logging: one switch. Then `expectsResponse()`: a second. Then `encode()`: a third. Now three switches must be kept in agreement, and when a fourth block type appears you must find all of them.

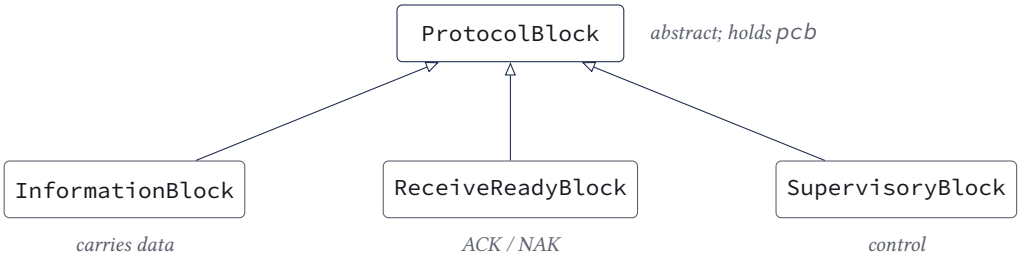
That is the smell: **the type of a thing is being re-decided at every place you use it.**

5.2 The idea: decide once, then let the object know what it is

Make each block type a class. Decide which one you are holding *exactly once*, at the moment the byte arrives. Afterwards you hold an object that already knows what it is, and you just

call methods on it.

> **ANALOGY The parcel depot**
>
> A parcel arrives. You could sort it at every stage — check "is this fragile?" at the truck, again at the belt, again at the van. Or you sort *once* at intake, put it in the right bin, and afterwards every stage handles "whatever is in this bin" by that bin's rules.
> One decision point, made once, near the door. The factory method is the intake desk; polymorphism is the bin's rules applying automatically.



> **CONCEPT Polymorphism**
>
> Literally "many shapes". You call `block.describe()`. Java looks at what the object *actually is* at runtime — I, R or S — and runs that class's version.
> One call site, three behaviours, and no `if` anywhere.

5.3 Build it

5.3.1 The base

```
package com.midnightbrewer.hardware;

/**
 * One ISO-14443-4 (T=CL) block.
 *
 * All three types share the PCB, so that state lives here in the base -- which
 * is exactly why this is an abstract CLASS and not an interface (an interface
 * cannot hold the field).
 *
 * What DIFFERS is behaviour, so describe() is abstract: each subclass answers
 * it its own way.
```

```
 */
public abstract class ProtocolBlock {

    private final int pcb;

    // 'protected' -- only subclasses construct blocks, via the factory below.
    protected ProtocolBlock(int pcb) {
        this.pcb = pcb & 0xFF;
    }

    /** The raw PCB byte (0..255). */
    public int pcb() {
        return pcb;
    }

    /** Declared here, no body -- every subclass MUST answer. */
    public abstract String describe();

    // ================================================================
    // THE FACTORY -- the ONE place the type is decided.
    //
    // This switch exists once, here. Everywhere else you hold a
    // ProtocolBlock and call methods on it. No type checks, no instanceof.
    //
    //   b8 = 0          -> I-block
    //   b8 = 1, b7 = 0 -> R-block
    //   b8 = 1, b7 = 1 -> S-block
    // ================================================================
    public static ProtocolBlock from(int pcb) {
        // TODO 1: if bit 7 (0x80) is clear      -> new InformationBlock(pcb)
        // TODO 2: else if bit 6 (0x40) is clear -> new ReceiveReadyBlock(pcb)
        // TODO 3: else                          -> new SupervisoryBlock(pcb)
        return null;
    }
}
```

CONCEPT Why a static factory instead of `new`

Because the caller does not know which class it wants — that is precisely what it is asking. `ProtocolBlock.from(0x02)` says "give me whatever block this byte represents". The caller states intent; the factory decides the type.

This is the **Open/Closed Principle** in miniature: to add a block type you add a class and one line in the factory. Every existing call site keeps working, untouched. Compare the switch-everywhere version, where you must hunt through the codebase and hope

> you found them all.

5.3.2 The three subclasses

```
package com.midnightbrewer.hardware;

public class InformationBlock extends ProtocolBlock {

    public InformationBlock(int pcb) {
        super(pcb);          // hand the PCB up to the base to store
    }

    @Override
    public String describe() {
        // TODO: "I-block (data), block number N"
        //       The block number is the low bit: pcb() & 0x01
        return null;
    }
}
```

```
public class ReceiveReadyBlock extends ProtocolBlock {

    public ReceiveReadyBlock(int pcb) { super(pcb); }

    @Override
    public String describe() {
        // TODO: "R-block (ACK)" or "R-block (NAK)" -- bit 4 (0x10) set = NAK.
        return null;
    }
}
```

```
public class SupervisoryBlock extends ProtocolBlock {

    public SupervisoryBlock(int pcb) { super(pcb); }

    @Override
    public String describe() {
        // TODO: bits 5..4 (mask 0x30) say which:
        //         0x00 -> "S-block (DESELECT)"
        //         0x30 -> "S-block (WTX -- card asking for more time)"
        //         else -> "S-block (unknown)"
```

```
        return null;
    }
}
```

`super(pcb)` calls the parent's constructor, and it must be the first statement: the base must be fully built before the subclass adds to it. Foundations before walls.

> **CONCEPT Always write `@Override`**
>
> It is optional, and you should write it anyway. It tells the compiler "I mean to be replacing a parent method". If you typo `describ()`, you get a compile error instead of a method that silently never gets called. Free insurance against a genuinely baffling class of bug.

5.4 Verify

```
package com.midnightbrewer.hardware;

import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
import static org.junit.jupiter.api.Assertions.assertTrue;

class ProtocolBlockTest {

    @Test
    void factoryPicksTheRightType() {
        assertTrue(ProtocolBlock.from(0x02) instanceof InformationBlock);
        assertTrue(ProtocolBlock.from(0x03) instanceof InformationBlock);
        assertTrue(ProtocolBlock.from(0xA2) instanceof ReceiveReadyBlock);
        assertTrue(ProtocolBlock.from(0xF2) instanceof SupervisoryBlock);
    }

    @Test
    void polymorphicDescribe() {
        // Same variable type, same method call, three different answers.
        ProtocolBlock[] blocks = {
            ProtocolBlock.from(0x02),
            ProtocolBlock.from(0xA2),
            ProtocolBlock.from(0xF2),
        };
        assertTrue(blocks[0].describe().startsWith("I-block"));
        assertTrue(blocks[1].describe().startsWith("R-block"));
        assertTrue(blocks[2].describe().startsWith("S-block"));
```

```
        assertEquals(0, ProtocolBlock.from(0x02).pcb() & 0x01);
        assertEquals(1, ProtocolBlock.from(0x03).pcb() & 0x01);
    }
}
```

Look closely at `polymorphicDescribe`: the array's type is `ProtocolBlock[]`. The code calling `describe()` has **no idea** which subclass each element is, and does not need to. That is the entire point.

> **GOING DEEPER** `instanceof` **in tests versus in production**
>
> Using `instanceof` in the first test is fine — a test's job is to verify the factory picks correctly.
> `instanceof` in *production* code is usually a signal that you should have added a method to the base instead. If you find yourself writing `if (b instanceof InformationBlock) { ... }`, ask what question you are really asking, and give the base class a method that answers it.

> **VERIFICATION GATE**
>
> ```
> mvn test -Dtest=ProtocolBlockTest
> ```

5.5 Inheritance versus composition

You have now used both ways one class can relate to another:

- **Composition (HAS-A)** — `Rc522Reader` *has a* `Rc522Driver`; `Rc522Driver` *has a* `SpiLink`.
- **Inheritance (IS-A)** — `Rc522Reader` *is a* `Iso14443Reader`; `InformationBlock` *is a* `ProtocolBlock`.

The test is whether "IS-A" is literally true. An I-block *is a* protocol block — yes. Is a reader *a kind of* driver? No: a reader *uses* a driver. Different relationships entirely.

> **TRAP** The `extends Pi4jSpiLink` disaster
>
> Writing `class Rc522Driver extends Pi4jSpiLink` compiles fine and looks like a shortcut — now the driver can call `transfer` directly!

> Then your unit test constructs a driver, which constructs a Pi4J context, which hunts for a GPIO chip that does not exist on your laptop, and the test dies with `No gpio chip found`. You have welded the driver to the hardware and destroyed the seam you built in Chapter 2.
> **Default to composition.** Reach for inheritance only when you genuinely have a family sharing an algorithm — like these blocks, or the reader base.

5.6 Commit

```
git add -A
git commit -m "M4: ProtocolBlock hierarchy -- I/R/S blocks with a factory"
```

CHAPTER 6

Activation

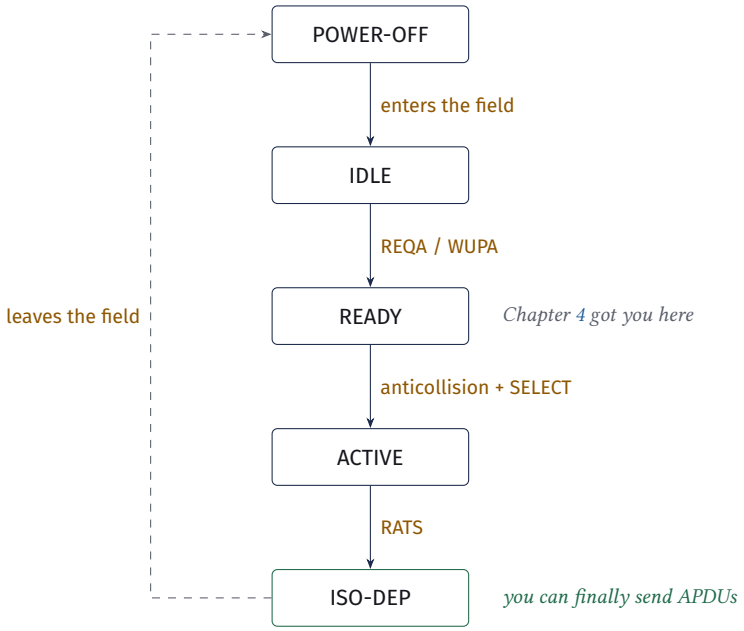
GOAL Take a card from “something is on the antenna” to “I know its UID and it is ready for commands”.

CONCEPT Value objects and immutability.

YOU END WITH A real UID, SAK and ATS printed from your card.

6.1 The problem: the card’s state machine

A contactless card is a state machine, and you must walk it in order.



6.2 Anticollision, and why it exists

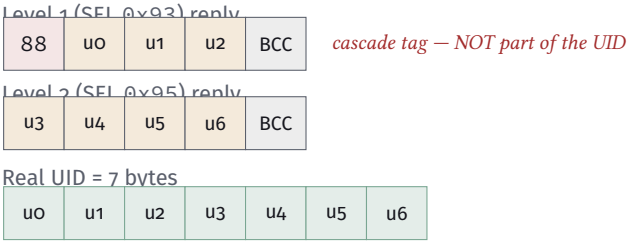
Put two cards on the antenna and both answer at once — the radio signals collide and you get mush. ISO-14443 solves this with a bit-by-bit tree walk: the reader asks "everyone whose UID starts with these bits, answer", and narrows until exactly one card remains.

We implement the **simple case**: ask for the whole UID at once, and if the reply's checksum is intact, exactly one card answered. If two cards collide, the checksum fails, we return empty, and the user is told to present one card. That is honest engineering for a coffee machine.

BCC. The block check character is `uid[0] ^ uid[1] ^ uid[2] ^ uid[3]`. XOR the four bytes; the result must equal the fifth. A cheap integrity check that catches both RF corruption and undetected collisions.

6.2.1 Cascade levels, and the `0x88` tag

UIDs come in 4-byte and 7-byte flavours, but an anticollision reply is fixed at five bytes (four data + one checksum). A 7-byte UID therefore needs two rounds:



The SAK from level 1 comes back with **bit 2 (`0x04`) set**, confirming "UID incomplete, do the next level". DESFire EV2/EV3 always has a 7-byte UID, so you *will* exercise this path.

> **TRAP Forgetting to skip the `0x88`**
>
> Keep the cascade tag and you get a 7-byte UID that is wrong in its first byte — and it looks completely plausible. You will only notice when it does not match what your phone reports, which might be days later.

RATS. Request Answer To Select (`0xE0 0x50`) is the final handshake: "let us speak ISO-14443-4". The card replies with its **ATS**, describing its capabilities and maximum frame size. The `0x50` parameter is FSDI $= 5$ (we accept up to 64-byte frames) and CID $= 0$.

6.3 Build it

6.3.1 `ActivatedCard` — a value object

```
package com.midnightbrewer.hardware;

/**
 * What activation produced: a UID, a SAK and an ATS.
 *
 * A VALUE OBJECT -- it holds data, has no behaviour that changes it, and is
 * IMMUTABLE: every field is final and there is no setter. Once built, it can
 * never disagree with itself.
 *
 * Why bother, instead of returning byte[][] or three out-parameters?
 *   - the three fields arrive together and belong together
 *   - the compiler enforces that you cannot forget one
 *   - it has a NAME, so signatures read as intent:
 *       ActivatedCard activate()      vs      byte[][] activate()
 */
```

```java
public final class ActivatedCard {

    private final byte[] uid;
    private final int sak;
    private final byte[] ats;

    public ActivatedCard(byte[] uid, int sak, byte[] ats) {
        // DEFENSIVE COPY on the way in: if we stored the caller's array
        // directly, they could mutate our state afterwards and we would
        // silently change. Copy, and we own it.
        this.uid = uid.clone();
        this.sak = sak;
        this.ats = ats.clone();
    }

    /** Defensive copy on the way OUT too, for the same reason. */
    public byte[] uid() { return uid.clone(); }

    public int sak() { return sak; }

    public byte[] ats() { return ats.clone(); }

    /** The UID as spaced uppercase hex, e.g. "04 76 18 62 7B 20 90". */
    public String uidHex() {
        // TODO: StringBuilder, String.format("%02X ", b) per byte, trim().
        return null;
    }
}
```

CONCEPT Three decisions, and why each one

`final class` — nobody may subclass it. For a value object that is right: there is no such thing as a "special kind of ActivatedCard", and allowing subclasses would let someone override `uid()` to return something else, breaking the guarantee.

Defensive copies both ways — the part people skip. Arrays in Java are references; handing out `this.uid` hands out a pointer to your internals, and the caller can scribble on it. `clone()` costs seven bytes and buys you a class that genuinely cannot be corrupted from outside.

Immutability — an `ActivatedCard` is *a fact about a moment*: "at 14:32 this card, with this UID, was activated". Facts do not change. Making it immutable means you can pass it anywhere, store it, log it, hand it to another thread, and never wonder whether something modified it.

6.3.2 The activation sequence

Add these constants to `Iso14443Reader`:

```
    /** Anticollision/SELECT command bytes for the two cascade levels. */
    private static final byte SEL_CL1 = (byte) 0x93;
    private static final byte SEL_CL2 = (byte) 0x95;

    /** NVB byte that turns an anticollision into a SELECT (full UID follows). */
    private static final byte SELECT_NVB = 0x70;

    /** SAK bit 3: "UID incomplete, do the next cascade level". */
    private static final int CASCADE_BIT = 0x04;

    /** RATS: request ISO-14443-4. PARAM 0x50 = FSDI 5, CID 0. */
    private static final byte RATS_CMD   = (byte) 0xE0;
    private static final byte RATS_PARAM = 0x50;

    /** The tag prefixing a 7-byte UID's first anticollision reply. */
    private static final byte CASCADE_TAG = (byte) 0x88;

    /** Anticollision reply is 4 UID/CT bytes + 1 BCC. */
    private static final int ANTICOLL_LENGTH = 5;
```

And the methods:

```
    // ================================================================
    // anticollision -- ask the card for (part of) its UID.
    //
    //   C: serNum[0] = 0x93; serNum[1] = 0x20;   // 0x20 NVB = "send it all"
    //      ToCard(PCD_TRANSCEIVE, serNum, 2, ...);
    //      // verify serNum[0]^[1]^[2]^[3] == serNum[4]
    //
    // Takes selCode so it works at BOTH levels. Full bytes, so
    // bitsInLastByte = 0. Returns the 5-byte reply, or empty if invalid.
    // ================================================================
    public byte[] anticollision(byte selCode) throws SpiException {
        // TODO 1: transceive { selCode, 0x20 } with 0 bits-in-last-byte.
        // TODO 2: a valid reply is exactly ANTICOLL_LENGTH bytes; else empty.
        // TODO 3: verify the BCC (XOR of the first four == the fifth);
        //         if it fails the read was garbled -> return empty.
        // TODO 4: return the reply.
        return new byte[0];
    }

    // ================================================================
```

```
    // select -- send the UID back, receive the SAK.
    //
    //   C: buffer[0]=selCode; buffer[1]=0x70;      // 0x70 NVB = SELECT
    //      memcpy(&buffer[2], anticoll, 5);        // 4 UID + BCC
    //      CalculateCRC(buffer, 7, &buffer[7]);    // -> 9 bytes
    //      ToCard(...);   // reply = SAK(1) + CRC(2)
    //
    // @return the SAK byte, or -1 if the reply was malformed.
    // ================================================================
    public int select(byte selCode, byte[] anticoll) throws SpiException {
        // TODO 1: build a 7-byte frame { selCode, SELECT_NVB, ...anticoll(5) }.
        // TODO 2: byte[] crc = calculateCrc(frame7);
        //         build a 9-byte frame: the 7, then crc[0], crc[1].
        // TODO 3: byte[] reply = transceive(frame9, 0);
        // TODO 4: if reply.length < 1 return -1, else return reply[0] & 0xFF.
        return -1;
    }

    // ================================================================
    // rats -- ask to speak ISO-14443-4; the card returns its ATS.
    //   C: req[0]=0xE0; req[1]=0x50; CalculateCRC(req,2,&req[2]); ToCard(...)
    // ================================================================
    public byte[] rats() throws SpiException {
        // TODO: header { RATS_CMD, RATS_PARAM }, append its CRC, transceive.
        return new byte[0];
    }

    // ================================================================
    // activate() -- the whole sequence, as another TEMPLATE METHOD.
    //
    // 'final' so a subclass cannot reorder the protocol.
    //   level 1: anticollision -> select -> SAK
    //   if the SAK's cascade bit is set, the UID is 7 bytes: do level 2
    //   then RATS
    // and assembles the real UID:
    //   - 4-byte card: ac1[0..3]
    //   - 7-byte card: ac1[1..3]  (SKIP the 0x88 tag)  +  ac2[0..3]
    // ================================================================
    public final ActivatedCard activate() throws SpiException {
        // TODO 1: ac1 = anticollision(SEL_CL1); if empty -> throw SpiException.
        //         sak1 = select(SEL_CL1, ac1);
        // TODO 2: if ((sak1 & CASCADE_BIT) == 0) -> 4-byte UID:
        //         uid = ac1[0..3], return new ActivatedCard(uid, sak1, rats());
        // TODO 3: else 7-byte UID: ac2 = anticollision(SEL_CL2);
        //         sak2 = select(SEL_CL2, ac2);
        //         uid = { ac1[1], ac1[2], ac1[3], ac2[0..3] }
        //         return new ActivatedCard(uid, sak2, rats());
```

```
        return null;
    }
```

6.4 Verify on hardware

```
if (reader.isCardPresent()) {              // wake the card into READY first
    ActivatedCard card = reader.activate();
    System.out.println("UID  = " + card.uidHex());
    System.out.printf("SAK  = 0x%02X%n", card.sak());
    System.out.println("ATS  = " + hex(card.ats()));
}
```

VERIFICATION GATE

For a DESFire EV3:

```
UID  = 04 76 18 62 7B 20 90
SAK  = 0x20
ATS  = 06 75 77 81 02 80 02 F0
```

How to read that output.

- **UID starts `04`** — NXP's manufacturer byte. Seven bytes total, so the cascade path ran and you skipped the tag correctly.
- **`SAK = 0x20`** — bit 5 set means "ISO-14443-4 compliant". This is the card telling you *I speak APDUs.* A MIFARE Classic answers `0x08`; if you see that, you have the wrong card and nothing after this chapter will work.
- **ATS** — `06` is its length, and the `75` encodes FSCI $= 5$, i.e. a 64-byte maximum frame.

TRAP The FSCI nibble

FSCI lives in one nibble of the ATS, and it is genuinely easy to read the wrong one — the original C in this project had exactly that bug.
Read the high nibble instead of the low and you conclude FSCI $= 7$, i.e. "128-byte frames". You then send a 100-byte APDU, and the card silently ignores it. Symptom: everything

> works until one particular command, which times out forever.

6.5 Traps

You see	Cause / fix
UID is 7 bytes but wrong in byte 0	Did not skip the `0x88` cascade tag.
`select()` returns −1	CRC wrong. Check `ModeReg`=`0x3D` (preset `0x6363`).
Anticollision always returns empty	Card not in READY — call `isCardPresent()` first.
`SAK = 0x08`	It is a MIFARE Classic, not a DESFire.
RATS returns empty	Card does not support 14443-4, or you sent RATS twice.
Works, then fails on retry	Card left in ACTIVE. Power-cycle the field per transaction.

6.6 Commit

```
git add -A
git commit -m "M5: activation -- anticollision, SELECT, cascade, RATS, ActivatedCard"
```

CHAPTER 7

The APDU channel

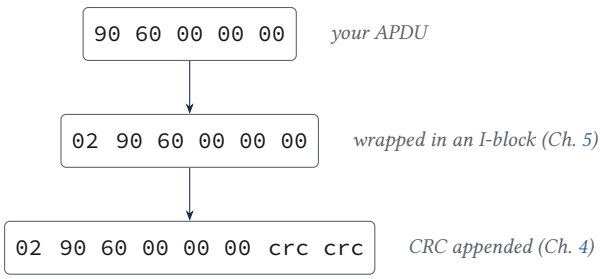
GOAL Send your first real DESFire command and get an answer.

CONCEPT Composition and layering — the last rung of the ladder.

YOU END WITH A real DESFire reporting its own chip type.

7.1 The problem

The card is activated. Now you want to send it commands. Those are **APDUs** (Application Protocol Data Units), and they need wrapping:



Someone has to own that wrapping — and the block-number bookkeeping that goes with it.

7.1.1 The block number: a one-bit conversation counter

ISO-DEP puts a single bit in the I-block PCB that **alternates 0, 1, 0, 1…** with each exchange. It exists to disambiguate retransmissions: if a reply is lost and you resend, the card can tell "this is the same command again" from "this is the next command".

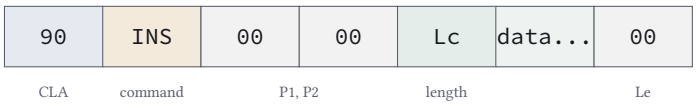
You must toggle it after every exchange. Get out of sync and the card ignores you — which

looks exactly like a dead card.

> **CONCEPT State that should be impossible to get wrong**
>
> If the block number were a parameter every caller had to pass, someone would eventually pass the wrong one.
> Hide it inside the channel as `private` state and toggle it automatically. The bug becomes *unrepresentable* rather than merely unlikely. That is the strongest form of encapsulation: not "callers should not do this" but "callers cannot".

7.1.2 DESFire's APDU wrapping

DESFire natively speaks its own short frames, but also accepts them wrapped in ISO-7816 APDUs, which is what we use:



`0x90` means "DESFire native command, wrapped". Replies end in a two-byte status word: `91 00` for OK, `91 AF` for "more frames follow", and various `91 xx` errors. Appendix A lists them all.

7.2 Build it

7.2.1 `ApduChannel` — another seam

```
package com.midnightbrewer.hardware;

/**
 * "Something that can carry an APDU to a card and bring back the reply."
 *
 * The same trick as SpiLink, one layer up. The DESFire logic you write in
 * Part IV will depend only on THIS -- so it can be tested against a fake
 * channel replaying canned card replies, with no reader and no card.
 *
 * That is not a nicety. Crypto is the one thing you cannot debug by staring
 * at it, and this interface is what makes it testable offline.
 */
public interface ApduChannel {
```

```
    /** Send one APDU, return the response (including its status word). */
    byte[] transceive(byte[] apdu) throws SpiException;
}
```

One method. That is the whole contract, and it is enough to make all of Part IV testable.

7.2.2 `InformationBlock.encode` — the block gains behaviour

Back in Chapter 5 an I-block could only `describe()` itself. Now it does real work:

```
    /**
     * Encode this I-block into a frame: its PCB, then the data it carries.
     */
    public byte[] encode(byte[] inf) {
        // TODO: byte[] frame of length inf.length + 1;
        //       frame[0] = (byte) pcb();
        //       copy inf in after it; return frame.
        return null;
    }
```

> **GOING DEEPER Designs pay off later than you would like**
>
> The class you built two chapters ago, which looked like an academic exercise in polymorphism, is now the thing that formats your wire frames.
> That is the normal shape of good design: the structure was right before you knew what you would need it for. It is also why "you aren't gonna need it" is advice to apply to *features*, not to *structure*.

7.2.3 `IsoDepChannel`

```
package com.midnightbrewer.hardware;

import java.util.Arrays;

/**
 * The ISO-14443-4 (ISO-DEP) channel: one APDU at a time, wrapped in an
 * I-block.
 *
 * The OOP on show:
 *   - COMPOSITION: it HAS-A reader (a channel is not a kind of reader).
```

```
 *   - ENCAPSULATED STATE: the block number toggles and nobody outside can
 *     touch it, or forget to.
 *   - it USES the Chapter 4 hierarchy: an InformationBlock encodes the frame.
 *
 * Simplifications, fine for DESFire's small frames: one I-block per APDU, no
 * chaining of large payloads, and no S-block "please wait" handling.
 */
public class IsoDepChannel implements ApduChannel {

    /** I-block PCB base; the low bit carries the block number. */
    private static final int IBLOCK_PCB = 0x02;

    private final Iso14443Reader reader;  // composition
    private int blockNumber = 0;          // private mutable state, owned here

    public IsoDepChannel(Iso14443Reader reader) {
        this.reader = reader;
    }

    /**
     *   1. wrap the APDU in an I-block for the current block number
     *   2. send it via the reader (which appends CRC and transceives)
     *   3. toggle the block number for next time
     *   4. the reply is [PCB, ...response...] -- strip the PCB
     */
    @Override
    public byte[] transceive(byte[] apdu) throws SpiException {
        // TODO 1: InformationBlock out =
        //             new InformationBlock(IBLOCK_PCB | blockNumber);
        //         byte[] frame = out.encode(apdu);
        // TODO 2: byte[] reply = reader.exchange(frame);
        // TODO 3: blockNumber ^= 1;    // toggle -- XOR with 1 flips a bit
        // TODO 4: if (reply.length < 1)
        //             throw new SpiException("empty ISO-DEP reply");
        //         return Arrays.copyOfRange(reply, 1, reply.length);
        return new byte[0];
    }
}
```

7.2.4 `Iso14443Reader.exchange` — the reusable primitive

You have written "append a CRC and transceive" twice already, inside `select()` and `rats()`. Pull it out:

```
    // ================================================================
```

```
    // exchange() -- send a payload as a CRC-protected T=CL frame, return the
    // reply with its CRC stripped.
    //
    // The ISO-DEP channel sits on top of this: it hands us [PCB + apdu], we
    // handle the CRC and the wire.
    // ================================================================
    public byte[] exchange(byte[] payload) throws SpiException {
        // TODO 1: byte[] crc = calculateCrc(payload);
        // TODO 2: build frame = payload followed by crc[0], crc[1].
        // TODO 3: byte[] reply = transceive(frame, 0);
        // TODO 4: if (reply.length < 2) return new byte[0];
        //         return Arrays.copyOf(reply, reply.length - 2);
        return new byte[0];
    }
```

7.3 The layer cake, complete

You have now built six layers, each speaking only to its neighbours.



Every arrow points down, and there are exactly two seams. The value of that shape: any layer can be tested by faking the layer below, and any layer can be replaced without touching

the others. Swap `Pi4jSpiLink` for an FT232H and nothing above changes. Swap the RC522 for a PN532 and you write one new `Iso14443Reader` subclass; nothing above changes.

> **CONCEPT What "good architecture" actually buys**
>
> It is not a matter of taste. It is the property that lets you **debug one layer at a time instead of debugging six at once.**
> When something breaks in Part IV, you will walk down this diagram asking each layer "are you still telling the truth?" — and the first *no* is your bug. Without the seams, every failure is a whole-system failure.

7.4 Verify — your first conversation with a DESFire

```
IsoDepChannel channel = new IsoDepChannel(reader);
// ISO-7816-wrapped DESFire GetVersion: 90 60 00 00 00
byte[] getVersion = { (byte) 0x90, 0x60, 0x00, 0x00, 0x00 };
byte[] resp = channel.transceive(getVersion);
System.out.println("GetVersion resp = " + hex(resp));
```

> **VERIFICATION GATE**
>
> ```
> GetVersion resp = 04 01 01 33 00 18 05 91 AF
> ```

Decode it — this is your card's identity document:

Byte	Value	Meaning
`04`	vendor	NXP
`01 01`	type/subtype	DESFire
`33`	**version**	**major 3 = EV3** (`0x12` = EV1, `0x22` = EV2)
`00 18`	storage	`0x18` → 8 KB
`05`	protocol	ISO-14443-2/3/4
`91 AF`	status	**additional frame available**

> **TRAP `91 AF` is not an error**
>
> It is the card saying "I have more to tell you". `GetVersion` chains three frames — hardware info, software info, then UID and batch number. You would send `90 AF 00 00 00` twice more to drain them.
>
> That matters practically: if you leave the card mid-chain and send a *different* command, some cards abort with an error. Either drain the chain or power-cycle the field before the next command. This will bite you in Chapter 10 if you do not.

If the `33` byte reads `0x12` or `0x22` you have an EV1 or EV2. EV2 works with everything in this book. **EV1 does not** — it has no `AuthenticateEV2First`, and needs the legacy `0x0A`/`0xAA` authentication, which is a different algorithm.

7.5 Traps

You see	Cause / fix
Empty reply	Block number out of sync, or RATS never ran.
Works exactly once per power-up	Not toggling the block number.
`91 AE` on a later command	Card was mid-chain from `91 AF`.
`6E 00` / `6D 00` instead of `91 xx`	ISO-7816 error — check the CLA `90` and the trailing `00`.
Reply is one byte too long	Forgot to strip the PCB.

7.6 Commit

```
git add -A
git commit -m "M6: ApduChannel seam + IsoDepChannel; GetVersion works on silicon"
```

PART IV

Cryptography

CHAPTER 8

AES-CMAC from scratch

GOAL Build the message authentication code DESFire EV2 runs on.

CONCEPT Immutable objects and constructor invariants.

YOU END WITH The RFC 4493 known-answer vectors passing.

8.1 Read this before you write any cryptography

Everything from here on is **verified against known-answer test vectors before it touches hardware**. This is not caution for its own sake.

Cryptographic code has a property no other code you have written has: **wrong output is indistinguishable from right output.** A broken CRC gives you a mismatch you can see. A broken CMAC gives you sixteen bytes of perfect-looking randomness — exactly like a correct CMAC. You cannot eyeball it, you cannot reason about it from a failed transaction, and if you build three more layers before testing you will have no idea which one is lying.

On real hardware it is worse. A bad MAC does not merely fail: the card may increment an internal error counter, and enough consecutive failures can permanently lock a key.

Vectors first. Silicon second.

8.2 What a MAC is

A **Message Authentication Code** answers one question: *was this message written by someone holding the secret key, and has it been altered?*

It is a keyed checksum. A CRC tells you a message survived the wire; a MAC tells you it survived an *adversary*. Without the key you cannot compute a valid MAC, so you cannot forge

or tamper.

> **ANALOGY The wax seal**
>
> A wax seal on a letter. Anyone can read the letter, but only the person with the signet ring can produce the seal, and breaking it to alter the contents is obvious.
> The ring is the key; the seal is the MAC. Crucially the seal does not *hide* the letter — MACs authenticate, they do not encrypt. Those are separate jobs, and DESFire uses both.

CMAC (RFC 4493 / NIST SP 800-38B) builds a MAC out of a block cipher you already have. Roughly: it is CBC-mode encryption where you throw away everything except the last block. That final block depends on every bit of the message and on the key, so it works as a fingerprint.

The subtlety CMAC adds — and the reason it is not simply "CBC-MAC" — is how it handles the **last block**, where naive CBC-MAC has a forgery weakness. CMAC derives two subkeys, K_1 and K_2, and XORs one of them into the final block depending on whether the message needed padding. That is the whole trick.

8.3 The algorithm

8.3.1 Part 1 — subkey generation (RFC 4493 §2.3)

```
L  = AES-ECB(K, 0^128)                encrypt an all-zero block
K1 = L << 1;   if MSB(L)  was set:  K1 ^= Rb
K2 = K1 << 1;  if MSB(K1) was set:  K2 ^= Rb
```

- $R_b = \texttt{0x87}$, the constant from AES's finite-field polynomial. "`^= Rb`" means XOR *the last byte* with `0x87`.
- "MSB set" means the top bit of the *first* byte: `(x[0] & 0x80) != 0`.
- « `1` shifts the whole 128-bit block left one bit — the bit that falls off byte $i{+}1$'s top becomes byte i's bottom.

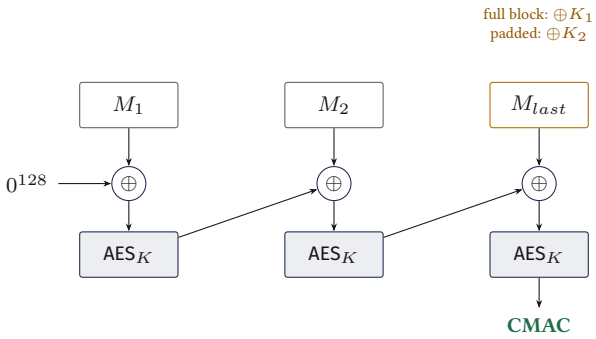
That last one is the only fiddly part, and in C you would write it identically:

```
byte[] out = new byte[16];
int carry = 0;
for (int i = 15; i >= 0; i--) {      // walk BACKWARDS, low byte first
    int b = in[i] & 0xFF;
    out[i] = (byte) ((b << 1) | carry);
```

```
    carry = (b >> 7) & 1;            // the bit shifted out of this byte
}
```

You walk from the last byte to the first because a carry propagates *upward*, toward the more significant end.

8.3.2 Part 2 — the MAC (RFC 4493 §2.4)




```
split the message into 16-byte blocks M_1 ... M_n
the LAST block is special:
    if it is exactly 16 bytes:   M_last = M_n XOR K1
    otherwise (incl. empty):     pad it (append 0x80 then 0x00s to 16)
                                 M_last = padded XOR K2
then CBC-MAC:
    X = 0^128
    for each block except the last:   X = AES(K, X XOR M_i)
    T = AES(K, X XOR M_last)          <- the CMAC
```

The K_1 versus K_2 choice is what makes "a 16-byte message" and "a 15-byte message padded to 16" produce different MACs. Without it, an attacker could exploit the ambiguity.

8.4 Build it

First, thin AES helpers. This is the one piece handed to you complete — reimplementing AES itself teaches you nothing here, and the JDK's is correct and fast.

```
package com.midnightbrewer.hardware.crypto;

import javax.crypto.Cipher;
import javax.crypto.spec.IvParameterSpec;
```

```
import javax.crypto.spec.SecretKeySpec;

/**
 * Thin AES-128 helpers over the JDK's crypto provider.
 *
 * NoPadding everywhere: DESFire does its own ISO 7816-4 padding, and letting
 * the JDK add PKCS#5 on top would corrupt every frame.
 */
public final class Aes {

    public static final int BLOCK = 16;

    private Aes() { }

    public static byte[] ecbEncrypt(byte[] key, byte[] block) {
        try {
            Cipher c = Cipher.getInstance("AES/ECB/NoPadding");
            c.init(Cipher.ENCRYPT_MODE, new SecretKeySpec(key, "AES"));
            return c.doFinal(block);
        } catch (Exception e) {
            throw new IllegalStateException("AES-ECB failed", e);
        }
    }

    public static byte[] cbcEncrypt(byte[] key, byte[] iv, byte[] data) {
        try {
            Cipher c = Cipher.getInstance("AES/CBC/NoPadding");
            c.init(Cipher.ENCRYPT_MODE, new SecretKeySpec(key, "AES"),
                   new IvParameterSpec(iv));
            return c.doFinal(data);
        } catch (Exception e) {
            throw new IllegalStateException("AES-CBC encrypt failed", e);
        }
    }

    public static byte[] cbcDecrypt(byte[] key, byte[] iv, byte[] data) {
        try {
            Cipher c = Cipher.getInstance("AES/CBC/NoPadding");
            c.init(Cipher.DECRYPT_MODE, new SecretKeySpec(key, "AES"),
                   new IvParameterSpec(iv));
            return c.doFinal(data);
        } catch (Exception e) {
            throw new IllegalStateException("AES-CBC decrypt failed", e);
        }
    }

    public static byte[] xor(byte[] a, byte[] b) {
```

```
        byte[] out = new byte[a.length];
        for (int i = 0; i < a.length; i++) {
            out[i] = (byte) (a[i] ^ b[i]);
        }
        return out;
    }
}
```

Now the part you write.

```
package com.midnightbrewer.hardware.crypto;

/**
 * AES-CMAC (RFC 4493) -- the MAC DESFire EV2 uses. The JDK has no CMAC, so
 * this is the from-scratch part; you are not reimplementing AES, you are
 * building the CMAC construction on top of Aes.ecbEncrypt.
 *
 * OOP on show: the key and the two derived subkeys are computed ONCE in the
 * constructor and kept private. compute() never mutates them. That makes this
 * an immutable object with an invariant established at birth: "if this object
 * exists, its subkeys are correct and match its key."
 */
public final class AesCmac {

    private static final int BLOCK = 16;
    private static final byte RB = (byte) 0x87;  // AES field polynomial

    private final byte[] key;
    private final byte[] k1;
    private final byte[] k2;

    public AesCmac(byte[] key) {
        if (key.length != BLOCK) {
            throw new IllegalArgumentException("AES-128 key must be 16 bytes");
        }
        this.key = key.clone();
        byte[][] subkeys = generateSubkeys(key);
        this.k1 = subkeys[0];
        this.k2 = subkeys[1];
    }

    // ---- RFC 4493 section 2.3 ----
    private static byte[][] generateSubkeys(byte[] key) {
        // TODO 1: byte[] l = Aes.ecbEncrypt(key, new byte[BLOCK]);
        // TODO 2: byte[] k1 = leftShift(l);
        //         if ((l[0] & 0x80) != 0) k1[15] ^= RB;
```

```
        // TODO 3: byte[] k2 = leftShift(k1);
        //         if ((k1[0] & 0x80) != 0) k2[15] ^= RB;
        // TODO 4: return new byte[][]{ k1, k2 };
        return new byte[][]{ new byte[BLOCK], new byte[BLOCK] };
    }

    /** Shift a 16-byte block left by ONE bit, as a single 128-bit number. */
    private static byte[] leftShift(byte[] in) {
        // TODO: walk from the LAST byte to the first, carrying the top bit
        //       of each byte down into the next. (See the chapter text.)
        return new byte[BLOCK];
    }

    // ---- RFC 4493 section 2.4 ----
    public byte[] compute(byte[] message) {
        // TODO 1: how many blocks, and is the last one full?
        //   if (message.length == 0) { n = 1; lastComplete = false; }
        //   else { n = ceil(message.length / 16.0);
        //          lastComplete = (message.length % 16 == 0); }

        // TODO 2: build M_last.
        //   if lastComplete: mLast = Aes.xor(lastBlock, k1)
        //   else: pad into a 16-byte buffer -- copy the leftover bytes,
        //         set padded[leftoverLen] = (byte) 0x80, rest stays zero;
        //         mLast = Aes.xor(padded, k2)

        // TODO 3: CBC-MAC through the blocks:
        //   byte[] x = new byte[BLOCK];
        //   for (int i = 0; i < n - 1; i++)
        //       x = Aes.ecbEncrypt(key, Aes.xor(x, i-th 16-byte slice));
        //   return Aes.ecbEncrypt(key, Aes.xor(x, mLast));
        return new byte[BLOCK];
    }
}
```

CONCEPT Establish invariants in the constructor

Why compute the subkeys in the constructor? Partly efficiency — they are a pure function of the key, and recomputing them per call would waste an AES operation on every MAC. But the deeper reason is **the invariant**. The constructor is the one place that can guarantee "this object is valid". It checks the key length and derives the subkeys; afterwards the object cannot be in a bad state, because nothing can change it.

Contrast a design with a `setKey()` method. Now every method must ask: has the

> key been set? Are the subkeys stale? Those questions do not exist if you make them impossible.
> **Establish invariants in the constructor, make fields `final`, expose no setters.** Half of "good OOP" is that one habit.

8.5 Verify — RFC 4493 vectors

These come from the RFC itself: independent of you, independent of this book, and definitive.

Key $K =$ `2b7e151628aed2a6abf7158809cf4f3c`

Message	Expected CMAC
(empty)	`bb1d6929e95937287fa37d129b756746`
`6bc1bee22e409f96e93d7e117393172a`	`070a16b46b4d4144f79bdd9dd04a287c`
`6bc1bee22e409f96e93d7e117393172a ae2d8a571e03ac9c9eb76fac45af8e51 30c81c46a35ce411` *(40 B)*	`dfa66747de9ae63030ca32611497c827`
the above + `e5fbc1191a0a52ef f69f2445df4f9b17ad2b417be66c3710` *(64 B)*	`51f0bebf7e3b9d92fc49741779363cfe`

Those four are chosen to hit every branch: empty (pad, K_2), exactly one block (K_1), a partial last block (pad, K_2), and multiple full blocks (K_1). If all four pass, your implementation is right.

The subkeys, useful mid-debug:

```
K1 = fbeed618357133667c85e08f7236a8de
K2 = f7ddac306ae266ccf90bc11ee46d513b
```

```java
package com.midnightbrewer.hardware.crypto;

import org.junit.jupiter.api.Test;
import java.util.HexFormat;
import static org.junit.jupiter.api.Assertions.assertEquals;

class AesCmacTest {

    private static final byte[] K =
```

```
            HexFormat.of().parseHex("2b7e151628aed2a6abf7158809cf4f3c");

    private static String mac(String messageHex) {
        byte[] m = HexFormat.of().parseHex(messageHex);
        return HexFormat.of().formatHex(new AesCmac(K).compute(m));
    }

    @Test
    void rfc4493_example1_empty() {
        assertEquals("bb1d6929e95937287fa37d129b756746", mac(""));
    }

    @Test
    void rfc4493_example2_oneBlock() {
        assertEquals("070a16b46b4d4144f79bdd9dd04a287c",
                mac("6bc1bee22e409f96e93d7e117393172a"));
    }

    @Test
    void rfc4493_example3_partialLastBlock() {
        assertEquals("dfa66747de9ae63030ca32611497c827",
                mac("6bc1bee22e409f96e93d7e117393172a"
                  + "ae2d8a571e03ac9c9eb76fac45af8e51"
                  + "30c81c46a35ce411"));
    }

    @Test
    void rfc4493_example4_fullBlocks() {
        assertEquals("51f0bebf7e3b9d92fc49741779363cfe",
                mac("6bc1bee22e409f96e93d7e117393172a"
                  + "ae2d8a571e03ac9c9eb76fac45af8e51"
                  + "30c81c46a35ce411e5fbc1191a0a52ef"
                  + "f69f2445df4f9b17ad2b417be66c3710"));
    }
}
```

VERIFICATION GATE

```
mvn test -Dtest=AesCmacTest      # all four must be green
```

Those four numbers were published in 2006 by people who have never seen your code. If your code reproduces them exactly, that is not “it seems to work” — that is proof.

`HexFormat` is a Java 17+ built-in, so no dependency is needed for hex parsing and printing —

which you will do constantly from here on.

8.6 Traps

You see	Cause / fix
Empty-message vector fails, others pass	Missing the `n=1, lastComplete=false` special case.
Partial-block vectors fail	Padding wrong: it is `0x80` then zeros, not all zeros.
Full-block vectors fail	Using K_2 where K_1 belongs, or vice versa.
All four fail, all plausible-looking	`leftShift` carry direction — you must walk backwards.
Sporadic wrong results	You mutated `key` or a subkey. They must be write-once.

8.7 Commit

```
git add -A
git commit -m "M7a: AES-CMAC (RFC 4493), verified against the RFC vectors"
```

CHAPTER 9

Session keys

GOAL Derive the two per-transaction AES keys from the nonces exchanged during authentication.

CONCEPT Private static helpers, and enums instead of boolean flags.

YOU END WITH EV2 key derivation verified against independent vectors.

9.1 The problem: never use the long-term key directly

Your card holds a permanent AES-128 key. It would be technically possible to encrypt every message with it. It would also be a bad idea, for three reasons worth understanding:

Replay.
If one key encrypts everything forever, a recorded "debit 1.50" frame stays valid. An attacker records one purchase and replays it — or replays a *credit*.

Traffic analysis.
The same plaintext under the same key gives the same ciphertext every time. An eavesdropper learns which commands are which without breaking anything.

Exposure.
Every message under the long-term key is another sample for cryptanalysis. Keys wear out with use.

So DESFire does what TLS does: use the long-term key *once*, to bootstrap a pair of throwaway keys valid only for this tap.

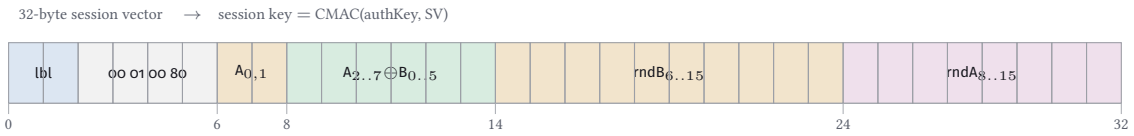
> **ANALOGY The hotel key card**
>
> A hotel does not hand guests the master key that opens every door forever. You present identification at the desk once — the long-term secret — and they programme a card that opens *your* room, *this* week. Lose it and the damage is bounded; the master key never left the safe.
>
> The nonces are what make each session unique. Both sides contribute randomness, so neither can force a repeat — which matters, because if the card alone chose, a compromised card could replay the same session forever.

Two keys, not one, because encryption and authentication are different jobs and reusing one key for both is a classic footgun:

- K_{enc} — encrypts data. *Confidentiality*: nobody can read the balance.
- K_{mac} — authenticates commands. *Integrity*: nobody can alter the debit.

9.2 The derivation

Both keys come from the same construction with a different label byte. The input is a 32-byte "session vector" (SV):




```
SV[0]      label            0xA5 for Kenc, 0x5A for Kmac
SV[1]      counter-label    the other one (0x5A when label is 0xA5)
SV[2..5]   00 01 00 80      fixed
SV[6..7]   rndA[0..1]
SV[8..13]  rndA[2..7] XOR rndB[0..5]      <- the interleave
SV[14..23] rndB[6..15]
SV[24..31] rndA[8..15]

session key = CMAC(authKey, SV)           32 bytes in, 16 out
```

The CMAC you built in Chapter 8 is doing duty here as a **key derivation function**: it mixes a lot of input down into sixteen bytes in a way that cannot be inverted. That is the same property that makes it a good MAC.

> **GOING DEEPER Why the strange interleave?**
>
> Both sides must derive identical keys, so the layout is ultimately just a specification both implement. But note *why it is mixed rather than concatenated*: bytes from both nonces are XOR'd together, so neither party can steer the result by choosing their nonce cleverly.
>
> If the layout were plain `rndA || rndB`, a malicious reader could pick `rndA` to force structure into the derived key. The interleave denies both sides that control.
>
> And `0xA5`/`0x5A` are bit-complements of each other, chosen to be maximally distinct so the two derivations cannot accidentally collide. The label sits at byte 0, so the very first AES block of the CMAC already differs and the two computations diverge immediately.

9.3 Build it

```
package com.midnightbrewer.hardware.crypto;

import java.util.Arrays;

/**
 * One authenticated session with a card: the two derived keys, and (from
 * Chapter 10) the secure-messaging state that goes with them.
 *
 * The keys are derived ONCE in the constructor and never change -- the same
 * invariant discipline as AesCmac. If a DesfireSession exists, its keys are
 * valid and belong together.
 */
public class DesfireSession {

    private final byte[] keyEnc;
    private final byte[] keyMac;

    /**
     * Which key we are deriving.
     *
     * An ENUM, not a boolean. deriveSessionKey(key, a, b, true) at a call site
     * tells you nothing -- true what? SessionKeyType.ENC is self-documenting,
     * the compiler rejects anything else, and adding a third key type later
     * does not change any existing signature.
     */
    private enum SessionKeyType { ENC, MAC }

    public DesfireSession(byte[] authKey, byte[] rndA, byte[] rndB) {
```

```
        if (authKey.length != 16) {
            throw new IllegalArgumentException("auth key must be 16 bytes");
        }
        this.keyEnc = deriveSessionKey(authKey, rndA, rndB, SessionKeyType.ENC);
        this.keyMac = deriveSessionKey(authKey, rndA, rndB, SessionKeyType.MAC);
    }

    // ================================================================
    // private static: it is a pure function (same inputs -> same output,
    // touches no instance state), so it does not need 'this'. Marking it
    // static says exactly that, and lets the constructor call it before the
    // object is finished being built.
    // ================================================================
    private static byte[] deriveSessionKey(byte[] authKey, byte[] rndA,
                                           byte[] rndB, SessionKeyType type) {
        if (rndA.length != 16 || rndB.length != 16) {
            throw new IllegalArgumentException("nonces must be 16 bytes");
        }
        byte[] sv = new byte[32];

        // TODO 1: sv[0] = 0xA5 for ENC, 0x5A for MAC.
        //         sv[1] = the opposite one.
        // TODO 2: sv[2..5] = 0x00, 0x01, 0x00, 0x80
        // TODO 3: sv[6] = rndA[0];  sv[7] = rndA[1];
        // TODO 4: sv[8..13] = rndA[2..7] XOR rndB[0..5]
        //         (Arrays.copyOfRange + Aes.xor is the tidy way)
        // TODO 5: sv[14..23] = rndB[6..15]     (10 bytes)
        // TODO 6: sv[24..31] = rndA[8..15]     (8 bytes)
        // TODO 7: return new AesCmac(authKey).compute(sv);
        return new byte[16];
    }

    /** Defensive copies -- callers must not be able to scribble on our keys. */
    public byte[] encKey() { return keyEnc.clone(); }

    public byte[] macKey() { return keyMac.clone(); }
}
```

9.4 Two design notes worth absorbing

9.4.1 Enum over boolean

Compare these call sites:

```
deriveSessionKey(key, rndA, rndB, true);                // true... what?
deriveSessionKey(key, rndA, rndB, SessionKeyType.ENC);  // obvious
```

A boolean parameter is a note-to-self that the reader does not get to see. This is sometimes called the **boolean trap**. Any time you are about to add a `boolean` parameter that selects *behaviour* rather than expressing *a yes/no fact*, an enum is better: self-documenting at the call site, extensible without changing signatures, and impossible to pass the wrong way round.

9.4.2 Do not keep scratch work as state

A tempting first draft stores the SV and the derived key as fields:

```
private byte[] SV;      // don't
private byte[] SK;      // don't
```

Now the object has fields that are only meaningful *during* construction. Anyone reading the class must work out whether they are still valid later, and a second call to a derive method would silently clobber them.

> **CONCEPT Fields versus locals**
>
> **Scratch work belongs in local variables. Fields are for state the object genuinely *has*.**
>
> `keyEnc` and `keyMac` are state. `sv` is a temporary. The test: *"if someone asked this object about this field a minute from now, would the answer be meaningful?"* If not, it is a local.

9.5 Verify

These vectors were generated independently with Python's `cryptography` library, so agreement means your Java is *right* — not merely self-consistent.

```
authKey = 00112233445566778899aabbccddeeff
rndA    = a0a1a2a3a4a5a6a7a8a9aaabacadaeaf
rndB    = b0b1b2b3b4b5b6b7b8b9babbbcbdbebf

-> Kenc  = d46c03528ecd3b1f1148e708d3e9729d
-> Kmac  = 116d156cc502da820eb57915f189ec74
```

```
package com.midnightbrewer.hardware.crypto;
```

```java
import org.junit.jupiter.api.Test;
import java.util.HexFormat;
import static org.junit.jupiter.api.Assertions.assertEquals;

class DesfireSessionTest {

    @Test
    void derivesBothSessionKeys() {
        byte[] authKey = HexFormat.of()
                .parseHex("00112233445566778899aabbccddeeff");
        byte[] rndA = HexFormat.of()
                .parseHex("a0a1a2a3a4a5a6a7a8a9aaabacadaeaf");
        byte[] rndB = HexFormat.of()
                .parseHex("b0b1b2b3b4b5b6b7b8b9babbbcbdbebf");

        DesfireSession session = new DesfireSession(authKey, rndA, rndB);

        assertEquals("d46c03528ecd3b1f1148e708d3e9729d",
                HexFormat.of().formatHex(session.encKey()));
        assertEquals("116d156cc502da820eb57915f189ec74",
                HexFormat.of().formatHex(session.macKey()));
    }
}
```

VERIFICATION GATE

```
mvn test -Dtest=DesfireSessionTest
```

If only one key is wrong, your `sv[0]`/`sv[1]` labels are swapped. **If both are wrong**, the SV layout is off — print your `sv` as hex and compare byte-by-byte against the diagram. The first eight bytes should read `a5 5a 00 01 00 80 a0 a1`, and eyeballing those finds most mistakes in seconds.

9.6 Traps

You see	Cause / fix
Both keys wrong	SV layout. Print it and compare against the spec.
Keys swapped	Label bytes `0xA5`/`0x5A` reversed.

You see	Cause / fix
`ArrayIndexOutOfBounds` in the XOR	Slice lengths: `rndA[2..7]` is 6 bytes, `rndB[6..15]` is 10, `rndA[8..15]` is 8.
Works in the test, fails on the card	You passed `rndB` where `rndA` belongs. The order matters.

9.7 Commit

```
git add -A
git commit -m "M7b: EV2 session key derivation, verified against independent vectors"
```

CHAPTER 10

Mutual authentication

GOAL Prove to the card you hold its key, make it prove the same to you, and establish a session — without either side ever transmitting the key.

CONCEPT Injecting randomness as a seam.

YOU END WITH A real card cryptographically proving it holds your key.

10.1 The problem

You and the card both — allegedly — hold the same sixteen-byte AES key. How do you each verify that about the other, over a radio link anyone can listen to, without sending the key?

You cannot send it. You cannot send a hash of it, because an eavesdropper simply replays the hash. You need each side to demonstrate *use* of the key on something new every time.

10.2 The idea: challenge–response, in both directions

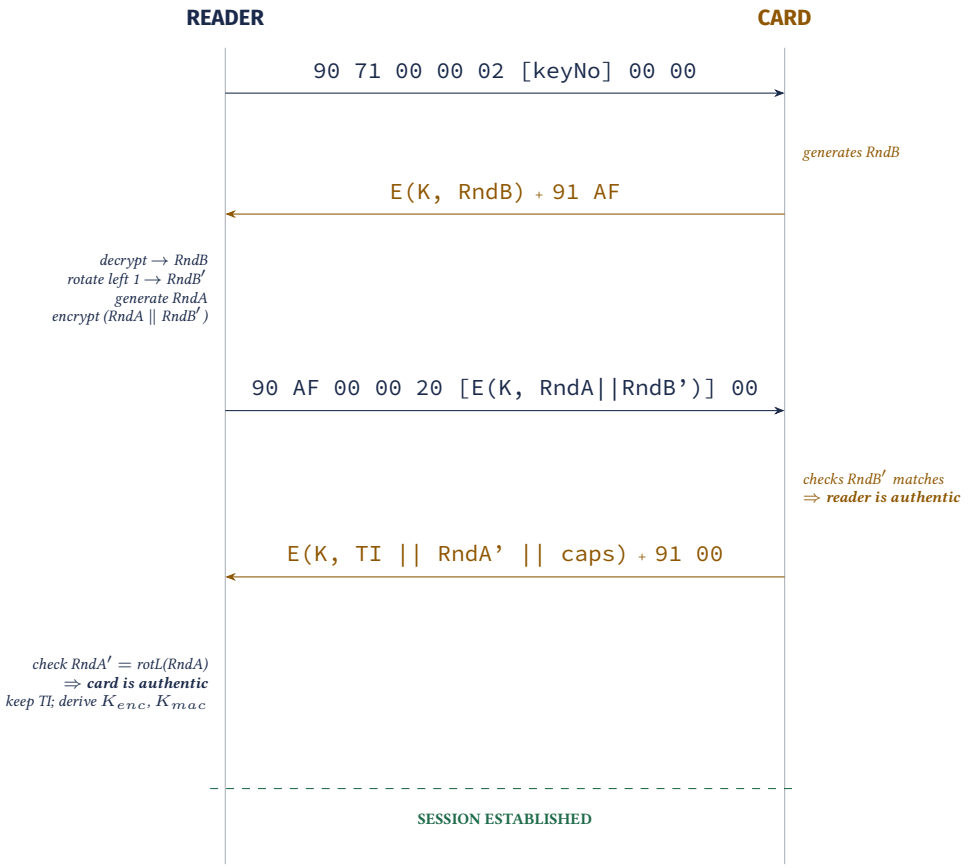
I send you a random number and ask you to encrypt it. Only someone with the key can produce the right answer. Because the number is new each time, a recorded answer is worthless later.

Do it in both directions and you have **mutual** authentication.

> **ANALOGY Two spies meeting**
>
> One says "the nightingale sings at midnight" — a line only someone briefed would recognise.
> If only *one* side gives a passphrase, an impostor can collect it and use it elsewhere. Both sides challenging each other means neither can be impersonated.

The rotation below is the detail that makes it work: you do not send back *the number you were given*, you send back a **transformation** of it, proving you actually decrypted and processed it rather than echoing bytes.

10.3 The dance



TRAP Every AES operation here uses an all-zero IV

Every single one. That is unusual — normally a fixed IV is a flaw — and it is safe *only* because every plaintext contains fresh randomness.
Do not carry the habit anywhere else. In Chapter 11 the IVs are derived per command,

> precisely because the plaintexts there are predictable.

Rotate left by one byte means `b[1], b[2], ... b[15], b[0]` — a *byte* rotation, not a bit rotation:

```
rotateLeft(b0b1b2b3b4b5b6b7b8b9babbbcbdbebf)
         = b1b2b3b4b5b6b7b8b9babbbcbdbebfb0
```

TI , the Transaction Identifier, is four bytes the card generates fresh for this session. From Chapter 11 on, it goes into every MAC and every IV, binding all subsequent traffic to *this* authentication. It is the session's fingerprint.

10.3.1 The step-2 reply, byte by byte

The card returns 32 encrypted bytes; decrypted they are:

Offset	Size	Contents
`[0..4)`	4	TI — the transaction identifier
`[4..20)`	16	RndA′ — your RndA, rotated left
`[20..26)`	6	PDcap2 — card capabilities
`[26..32)`	6	PCDcap2 — reader capabilities echoed back

You need the first two. The capabilities you can ignore.

10.4 The testability problem, and the seam that solves it

Here is a genuine engineering puzzle. This function calls a random number generator, so it **produces different output every run**. How do you write a known-answer test for it?

The answer is the same move as Chapter 2: **the thing that varies becomes a parameter.** Randomness is a dependency like any other. Inject it, and a test can inject a *fixed* "random" number, making the whole handshake deterministic and checkable.

```
package com.midnightbrewer.hardware;
```

```java
import java.security.SecureRandom;

/**
 * A source of random bytes -- the seam that makes the auth handshake testable.
 *
 * Production uses secure(). A test injects a fake returning a FIXED RndA, and
 * suddenly the whole handshake is a known-answer test.
 *
 * @FunctionalInterface = exactly one abstract method, so a fake can be written
 * as a one-line lambda:   RandomSource fixed = n -> myFixedBytes;
 */
@FunctionalInterface
public interface RandomSource {

    byte[] nextBytes(int n);

    /** The real one, backed by a cryptographically secure RNG. */
    static RandomSource secure() {
        SecureRandom rng = new SecureRandom();
        return n -> {
            byte[] out = new byte[n];
            rng.nextBytes(out);
            return out;
        };
    }
}
```

TRAP `SecureRandom`, never `java.util.Random`

`Random` is a linear congruential generator: observe a couple of outputs and you can predict every future one. For a nonce, that is the entire security of the protocol. `SecureRandom` draws from the operating system's entropy pool. This is a one-word difference with a catastrophic failure mode, and it is exactly the kind of thing that ships.

CONCEPT A seam is not a licence

The `RandomSource` seam exists so *tests* can pin randomness. If production code ever passes anything but `RandomSource.secure()`, the protocol is broken.
Test doubles live in `src/test` and that is not an accident: the build system enforces that a fixed-nonce implementation cannot reach the kiosk.

10.5 Build it

```
package com.midnightbrewer.hardware;

import com.midnightbrewer.hardware.crypto.Aes;
import com.midnightbrewer.hardware.crypto.DesfireSession;
import java.util.Arrays;

/**
 * A DESFire card, spoken to over an ApduChannel.
 *
 * COMPOSITION: it HAS-A channel and HAS-A random source, both injected. It
 * knows nothing about SPI, the RC522, or ISO-DEP framing -- only APDUs in and
 * responses out. That is why it can be tested with no hardware at all.
 */
public class DesfireCard {

    private final ApduChannel channel;
    private final RandomSource randomSource;

    private byte[] ti;                 // transaction identifier, set at auth
    private DesfireSession session;    // the derived keys, set at auth

    public DesfireCard(ApduChannel channel, RandomSource randomSource) {
        this.channel = channel;
        this.randomSource = randomSource;
    }

    /**
     * Wrap a DESFire command in an ISO-7816 APDU and send it.
     *   90 [cmd] 00 00 [Lc] [data...] 00
     */
    private byte[] wrapApdu(byte cmd, byte[] data) throws SpiException {
        byte[] apdu = new byte[data.length + 6];
        apdu[0] = (byte) 0x90;             // CLA: DESFire native, wrapped
        apdu[1] = cmd;                     // INS
        apdu[2] = 0x00;                    // P1
        apdu[3] = 0x00;                    // P2
        apdu[4] = (byte) data.length;      // Lc
        System.arraycopy(data, 0, apdu, 5, data.length);
        apdu[apdu.length - 1] = 0x00;      // Le
        return channel.transceive(apdu);
    }

    /** Rotate left by one BYTE: b[1..n-1] followed by b[0]. */
    private static byte[] rotateLeft(byte[] array) {
        // TODO
```

```
        return array;
    }

    /** SelectApplication (0x5A). The AID goes on the wire LSB-first. */
    public void selectApplication(byte[] aid) throws SpiException {
        if (aid.length != 3) {
            throw new IllegalArgumentException("AID must be 3 bytes");
        }
        byte[] response = wrapApdu((byte) 0x5A, aid);
        if (response.length < 2) {
            throw new SpiException("SelectApplication failed");
        }
    }

    // ================================================================
    // AuthenticateEV2First (0x71). All AES here uses an ALL-ZERO IV.
    // ================================================================
    public void authenticateEv2First(byte keyNo, byte[] key) throws SpiException {
        if (key.length != 16) {
            throw new IllegalArgumentException("key must be 16 bytes");
        }

        // ---- step 1: ask for the challenge ----
        // TODO 1: byte[] r1 = wrapApdu((byte) 0x71, new byte[]{ keyNo, 0x00 });
        //
        //   TWO data bytes: the key number AND a 0x00 "length of capabilities"
        //   byte. Sending only keyNo gets you 91 7E (length error) and it is a
        //   genuinely annoying hour to find.
        //
        // TODO 2: strip the 2 status bytes -> encRndB (16 bytes)
        // TODO 3: byte[] rndB = Aes.cbcDecrypt(key, new byte[16], encRndB);

        // ---- step 2: answer, and challenge back ----
        // TODO 4: byte[] rndBRot = rotateLeft(rndB);
        //         byte[] rndA    = randomSource.nextBytes(16);
        // TODO 5: build a 32-byte buffer: rndA then rndBRot.
        //         byte[] encAB = Aes.cbcEncrypt(key, new byte[16], that);
        //
        //   ONE 32-byte CBC run, not two 16-byte ones. In CBC the second block
        //   chains off the first's ciphertext -- encrypting them separately
        //   gives a different, wrong answer.
        //
        // TODO 6: byte[] r2 = wrapApdu((byte) 0xAF, encAB);
        //         strip the status bytes -> encResp (32 bytes)

        // ---- step 3: verify the card, keep the session ----
        // TODO 7: byte[] plain = Aes.cbcDecrypt(key, new byte[16], encResp);
```

```
        // TODO 8: byte[] rndARot = Arrays.copyOfRange(plain, 4, 20);
        //         if (!Arrays.equals(rndARot, rotateLeft(rndA)))
        //             throw new SpiException("authentication failed: RndA mismatch");
        //
        //   THIS CHECK IS THE WHOLE POINT. Skip it and you have not
        //   authenticated the card -- you have merely talked to something.
        //
        // TODO 9: this.ti = Arrays.copyOfRange(plain, 0, 4);
        //         this.session = new DesfireSession(key, rndA, rndB);
    }

    public byte[] getTi() { return ti; }
}
```

10.6 Verify — the full handshake, offline

The elegant part: your test class can **be** the fake card and the fake random source at once.

```
package com.midnightbrewer.hardware;

import java.util.Arrays;
import java.util.HexFormat;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertArrayEquals;

/**
 * The test IS its own fake card + fake RNG: it implements both seams, so
 * new DesfireCard(this, this) plugs the test straight into the card.
 */
public class DesfireCardTest implements RandomSource, ApduChannel {

    byte[] key   = HexFormat.of().parseHex("00112233445566778899aabbccddeeff");
    byte[] rndA  = HexFormat.of().parseHex("a0a1a2a3a4a5a6a7a8a9aaabacadaeaf");
    byte[] encAB = HexFormat.of().parseHex(
            "cf086a82c0b745a749daabb28a7a8db3"
          + "32272f9e5fdc37a4746064f25b7c2ca7");
    byte[] tiExpected = HexFormat.of().parseHex("11223344");

    // Replies MUST carry the trailing status word -- the card code strips the
    // last two bytes off every reply. Without them you chop real ciphertext
    // and decrypt garbage.
    byte[] selectReply = HexFormat.of().parseHex("9100");
    byte[] step1Reply  = HexFormat.of().parseHex(
            "eab6822fe368d5bc9895fb2558b38dde" + "91af");
```

```
    byte[] step2Reply  = HexFormat.of().parseHex(
            "4e08e447d74810052caa1c5b6c60343f"
          + "51f6e901ad171e677b100d3a34c925e4" + "9100");

    @Override
    public byte[] nextBytes(int n) {
        return rndA;                        // pin RndA -> deterministic
    }

    @Override
    public byte[] transceive(byte[] apdu) throws SpiException {
        switch (apdu[1]) {                  // dispatch on INS
            case (byte) 0x5A:
                return selectReply;
            case (byte) 0x71:
                return step1Reply;
            case (byte) 0xAF:
                // assert on what we SENT: encAB sits at [5 .. len-1)
                assertArrayEquals(encAB,
                        Arrays.copyOfRange(apdu, 5, apdu.length - 1),
                        "step-2 ciphertext is wrong");
                return step2Reply;
            default:
                throw new SpiException("unexpected APDU");
        }
    }

    @Test
    public void authenticateEv2First_matchesReplayVectors() throws SpiException {
        DesfireCard card = new DesfireCard(this, this);

        card.selectApplication(new byte[]{ 0x01, 0x02, 0x03 });
        card.authenticateEv2First((byte) 0x00, key);  // no throw == RndA' verified

        assertArrayEquals(tiExpected, card.getTi());
    }
}
```

CONCEPT Why this test is strong

It checks *both* directions:

- the `0xAF` case asserts the exact ciphertext **you sent** — proving your rotate, your concatenation and your CBC encryption are all correct;

- reaching the end without throwing proves your RndA′ verification passed;
- the TI assertion proves you sliced the reply correctly.

A test that only checked the return value would pass even if you had skipped the RndA′ comparison entirely.

VERIFICATION GATE

```
mvn test -Dtest=DesfireCardTest
```

The full vector set is in Appendix A.

10.7 Then: on the real card

```
DesfireCard desfire = new DesfireCard(channel, RandomSource.secure());
desfire.selectApplication(new byte[]{ 0x03, 0x02, 0x01 });  // AID 0x010203, LSB-first
byte[] appKey = new byte[16];
Arrays.fill(appKey, (byte) 0x22);                           // your app's user key
desfire.authenticateEv2First((byte) 0x02, appKey);          // key slot 2
System.out.println("MUTUAL AUTH OK. TI = " + hex(desfire.getTi()));
```

VERIFICATION GATE

```
MUTUAL AUTH OK. TI = 8071F582
```

Run it twice. **The TI will be different every time** — it is freshly generated per session. Seeing it change is your proof that this was a live authentication and not a replay of your test vectors.

TRAP A factory-blank card will fail here

Its default master key is DES, not AES, and there is no application yet. The card must be personalised first — that is Chapter 13's job.
If you do not have a provisioned card, use the top-up app's setup screen before attempting

> this chapter on hardware.

10.8 Traps

You see	Cause / fix
`91 7E` (length error) on step 1	Sent only `keyNo`. It must be `{ keyNo, 0x00 }`.
`91 AE` (auth error) on step 2	Wrong key, wrong key number, or wrong application selected.
RndA mismatch but vectors pass	Check you rotated *left*, and by a byte not a bit.
Test decrypts garbage	Fake replies missing their `91 xx` status word.
Ciphertext differs from the vector	You encrypted RndA and RndB′ as two separate CBC calls.
Auth “succeeds” always	You skipped the RndA′ comparison. Verify you actually throw.

10.9 Commit

```
git add -A
git commit -m "M7c: AuthenticateEV2First, verified offline against replay vectors"
```

CHAPTER 11

Secure messaging and the debit

GOAL Read the balance and spend money.

CONCEPT Growing a class as its responsibilities grow.

YOU END WITH **Money moving off a real card.**

11.1 The problem

You are authenticated. But the card will no longer accept a bare command — each one must now prove three things:

1. **Authenticity** — an 8-byte CMAC computed with K_{mac}.
2. **Freshness** — a command counter, so no frame can be replayed.
3. **Confidentiality** (for values) — the amount encrypted with K_{enc}.

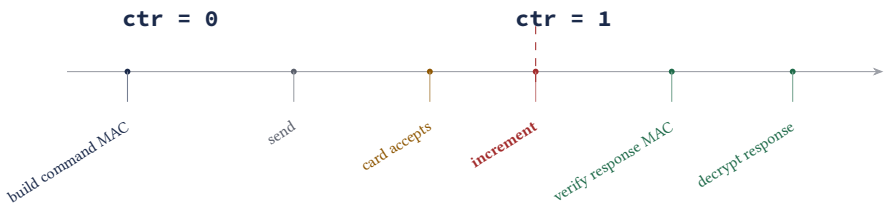
That is "CommMode.Full", and it is what a value file configured for full protection demands.

> **ANALOGY The cheque**
>
> The **amount** is written in the box — the encrypted data. The **signature** proves you authorised it — the MAC. And the **cheque number** stops the payee photocopying it and cashing it twice — the counter.
> Remove any one and the system fails: no signature, anyone writes cheques; no number, one cheque pays forever; no amount protection, someone alters 10 to 1000.

11.2 The command counter, and the ordering that will bite you

`CmdCtr` is a 16-bit counter that starts at **0** immediately after authentication and increments by one per accepted command. It goes into every MAC and every IV, so no two commands in a session look alike.

Here is the rule that costs people an evening:



> **TRAP The increment happens *before* you verify the response**
>
> The **command** MAC uses the pre-increment value; the **response** MAC and response IV use the post-increment value.
> Get this backwards and your command MACs are perfect while every response verification fails — which looks exactly like a broken MAC implementation, sending you back to debug code that is correct.

11.3 The four building blocks

11.3.1 Truncated MAC

DESFire transmits only 8 of the 16 CMAC bytes: the **odd-indexed** ones, `mac[1], mac[3], ... mac[15]`. It halves the frame overhead, and 64 bits is still ample against forgery. You can verify this one by eye:

```
truncate(00112233445566778899aabbccddeeff) = 1133557799bbddff
```

11.3.2 MAC input layout

Both directions have the same shape, differing only in byte 0:

```
command:   [cmd ][ctrLo][ctrHi][TI0..TI3][data...]
```

```
response:  [0x00][ctrLo][ctrHi][TI0..TI3][data...]
                  ^
             2 bytes, LITTLE-endian
```

Byte 0 of the response input is `0x00` — the success status code. You only check a response MAC after a successful status, so it is always zero.

11.3.3 IV derivation

IVs are computed, not chosen. Each is an ECB encryption of a block built from labels, TI and the counter:

```
command IV  = AES-ECB(Kenc, [0xA5, 0x5A, TI0..TI3, ctrLo, ctrHi, 00 x 8])
response IV = AES-ECB(Kenc, [0x5A, 0xA5, TI0..TI3, ctrLo, ctrHi, 00 x 8])
                             \____ the labels are SWAPPED ____/
```

Because the counter is in there, **every command gets a unique IV for free**. That is the property CBC needs, and it falls out of the counter rather than requiring a separate random value.

11.3.4 ISO 7816-4 padding

Append `0x80`, then zeros to the next 16-byte boundary.

> **TRAP Padding always adds at least one byte**
>
> A 16-byte payload becomes 32, not 16. That is what makes the padding unambiguously removable — if it sometimes added nothing, you could not tell a message ending in `80` from a padded one.
>
> ```
> int padded = (data.length | 0x0F) + 1;   // next multiple of 16, always > length
> ```

11.4 Build it, part 1: grow `DesfireSession`

The session started as “two keys”. It now becomes the full secure-messaging engine, because that is where the state and the keys already live.

```
    // ---- EV2 secure-messaging state ----
```

```
    private final byte[] ti;   // transaction id from auth, 4 bytes
    private int cmdCounter;    // starts at 0, one per command

    // Old constructor: keys only. TI defaults to zeros -- key derivation never
    // uses TI, so the Chapter 8 test keeps passing untouched.
    // (Constructor chaining: this(...) delegates to the fuller one.)
    public DesfireSession(byte[] authKey, byte[] rndA, byte[] rndB) {
        this(authKey, rndA, rndB, new byte[4]);
    }

    public DesfireSession(byte[] authKey, byte[] rndA, byte[] rndB, byte[] ti) {
        if (authKey.length != 16) throw new IllegalArgumentException("key: 16 bytes");
        if (ti.length != 4)       throw new IllegalArgumentException("TI: 4 bytes");
        this.keyEnc = deriveSessionKey(authKey, rndA, rndB, SessionKeyType.ENC);
        this.keyMac = deriveSessionKey(authKey, rndA, rndB, SessionKeyType.MAC);
        this.ti = ti.clone();
        this.cmdCounter = 0;
    }

    // ---- the MAC layer ----

    /** Tick the counter. Wraps at 16 bits. Ordering is load-bearing. */
    public void incrementCounter() {
        // TODO: cmdCounter = (cmdCounter + 1) & 0xFFFF;
    }

    /** Keep the ODD-indexed bytes: mac[1], mac[3], ... mac[15]. */
    private static byte[] truncateMac(byte[] mac16) {
        // TODO: byte[8]; out[i] = mac16[i * 2 + 1];
        return new byte[8];
    }

    /** MAC input = [cmd][ctrLo][ctrHi][TI 4][data...], then CMAC + truncate. */
    public byte[] commandMac(byte cmd, byte[] data) {
        // TODO: byte[] input = new byte[7 + data.length];
        //       input[0] = cmd;
        //       input[1] = (byte)(cmdCounter & 0xFF);
        //       input[2] = (byte)((cmdCounter >> 8) & 0xFF);
        //       copy ti into [3..7), data into [7..);
        //       return truncateMac(new AesCmac(keyMac).compute(input));
        return new byte[8];
    }

    /** Same shape, but input[0] = 0x00 (the success status code). */
    public byte[] responseMac(byte[] data) {
        // TODO
        return new byte[8];
```

```
    }

    public boolean verifyResponseMac(byte[] data, byte[] mac8) {
        // TODO: Arrays.equals(responseMac(data), mac8)
        return false;
    }

    // ---- the crypto layer ----

    public byte[] commandIv()  { /* TODO: calcIv(0xA5, 0x5A) */ return new byte[16]; }
    public byte[] responseIv() { /* TODO: calcIv(0x5A, 0xA5) */ return new byte[16]; }

    /** Only the two label bytes differ between the two IVs. */
    private byte[] calcIv(byte b0, byte b1) {
        // TODO: byte[16] block; block[0]=b0; block[1]=b1;
        //       ti into [2..6); block[6]=ctrLo; block[7]=ctrHi; rest zero;
        //       return Aes.ecbEncrypt(keyEnc, block);
        return new byte[16];
    }

    /** ISO 7816-4: append 0x80 then zeros. ALWAYS adds >= 1 byte. */
    private static byte[] padIso7816(byte[] data) {
        // TODO: int padded = (data.length | 0x0F) + 1; ...
        return data;
    }

    public byte[] encryptCommandData(byte[] plaintext) {
        // TODO: Aes.cbcEncrypt(keyEnc, commandIv(), padIso7816(plaintext))
        return new byte[0];
    }

    public byte[] decryptResponseData(byte[] ciphertext) {
        // TODO: Aes.cbcDecrypt(keyEnc, responseIv(), ciphertext)
        //   Padding is left in place -- the caller knows its payload length.
        return new byte[0];
    }
```

CONCEPT Put state and its operations in the same place

Note what just happened to this class. It began as a value object holding two keys. It is now a stateful engine with a counter, an identity, and a dozen operations.
That growth is *correct* — the new responsibilities all operate on state it already owned. Had you instead put `cmdCounter` in `DesfireCard` and passed it into every method, the counter would live in one class and the keys in another, and every call site would be

> responsible for keeping them in step. The bug where someone forgets to increment would be *possible*.
> Put the state and its operations together, and whole categories of bug become unrepresentable.

11.4.1 Verify the layers

Session built from Chapter 9's inputs, plus `TI = 11223344`:

Call	ctr	Expected
`truncateMac(0011...eeff)`	—	`1133557799bbddff`
`commandMac(0x6C, {01})`	0	`4d131ee17b482b16`
`responseMac({})`	1	`6b082e8062d1a89d`
`commandIv()`	0	`dd42134cd6970980d22066cb00ab1b16`
`encryptCommandData(96000000)`	0	`a2eb8f000bf3ddfe6e18e8859b284591`
`responseIv()`	1	`cb73ae774aafd81868062906135969b9`

```
DesfireSession s = new DesfireSession(authKey, rndA, rndB, h("11223344"));
assertEquals("4d131ee17b482b16", hex(s.commandMac((byte) 0x6C, h("01"))));
assertEquals("dd42134cd6970980d22066cb00ab1b16", hex(s.commandIv()));
assertEquals("a2eb8f000bf3ddfe6e18e8859b284591",
             hex(s.encryptCommandData(h("96000000"))));
s.incrementCounter();                                  // ctr -> 1
assertEquals("6b082e8062d1a89d", hex(s.responseMac(new byte[0])));
assertEquals("cb73ae774aafd81868062906135969b9", hex(s.responseIv()));
```

`96 00 00 00` is 150 little-endian — a €1.50 debit amount.

11.5 Build it, part 2: the commands

> **TRAP First, a one-line fix in Chapter 10's code**
>
> Your `authenticateEv2First` currently builds the session *without* the TI:
>
> ```
> session = new DesfireSession(key, rndA, rndB);        // TI = 0000 !
> ```

Change it to:

```
session = new DesfireSession(key, rndA, rndB, ti);
```

Miss this and every MAC from here on is computed against the wrong TI. The symptom is maddening: authentication succeeds, and then the very first `getValue` fails its response MAC — one command away from the cause.

11.5.1 `getValue` (`0x6C`) — read the balance

```
command : 90 6C 00 00 09 [fileNo][cmdMac 8] 00
response: [encrypted 16][respMac 8] 91 00
```

```
    public int getValue(byte fileNo) throws SpiException {
        // TODO 1: byte[] header = { fileNo };
        //         byte[] mac = session.commandMac((byte) 0x6C, header);  // ctr 0
        //         send header ++ mac (9 bytes) via wrapApdu(0x6C, ...)
        // TODO 2: session.incrementCounter();   // BEFORE reading the response
        // TODO 3: drop the 2 status bytes; body must be >= 24;
        //         encrypted = body[0..16), respMac = body[16..24)
        // TODO 4: if (!session.verifyResponseMac(encrypted, respMac))
        //             throw new SpiException("GetValue response MAC failed");
        //
        //   Do NOT skip this. An unverified balance is a number an attacker
        //   chose. The whole point of the MAC is refusing to trust it.
        //
        // TODO 5: byte[] plain = session.decryptResponseData(encrypted);
        //         return decodeValue(plain);    // first 4 bytes, LE signed
        return 0;
    }

    /** Signed 32-bit little-endian from the first 4 bytes. */
    private static int decodeValue(byte[] b) {
        return (b[0] & 0xFF) | ((b[1] & 0xFF) << 8)
             | ((b[2] & 0xFF) << 16) | ((b[3] & 0xFF) << 24);
    }
```

11.5.2 `debit` (`0xDC`) and `commitTransaction` (`0xC7`)

```
debit  : 90 DC 00 00 19 [fileNo][enc amount 16][cmdMac 8] 00  ->  [respMac 8] 91 00
commit : 90 C7 00 00 08 [cmdMac 8] 00                         ->  [respMac 8] 91 00
```

Note the asymmetry: **debit is Full** (the amount is encrypted), **commit is MAC-only** (it carries no data).

```
    /** Stage a debit. NOTHING MOVES until commitTransaction() succeeds. */
    public void debit(byte fileNo, int amountCents) throws SpiException {
        // TODO 1: byte[] header     = { fileNo };
        //         byte[] ciphertext =
        //             session.encryptCommandData(encodeValue(amountCents));
        //         byte[] macInput   = header ++ ciphertext;  // fileNo stays PLAIN
        //         byte[] mac        = session.commandMac((byte) 0xDC, macInput);
        //         send macInput ++ mac
        // TODO 2: requireOk(response); session.incrementCounter();
        // TODO 3: respMac = the last 8 bytes BEFORE the status word;
        //         verify over EMPTY data (the reply carries none).
    }

    /** Make the staged debit permanent. */
    public void commitTransaction() throws SpiException {
        // TODO: mac = session.commandMac((byte) 0xC7, new byte[0]);
        //       send it; requireOk; incrementCounter; verify respMac over empty.
    }

    private static byte[] encodeValue(int v) {
        return new byte[]{ (byte) v, (byte) (v >> 8),
                           (byte) (v >> 16), (byte) (v >> 24) };
    }

    /** Throw unless the status word is 91 00, naming the card's code. */
    private static void requireOk(byte[] response) throws SpiException {
        if (response.length < 2) throw new SpiException("response too short");
        int sw1 = response[response.length - 2] & 0xFF;
        int sw2 = response[response.length - 1] & 0xFF;
        if (sw1 != 0x91 || sw2 != 0x00) {
            throw new SpiException(String.format("card returned %02X%02X", sw1, sw2));
        }
    }
```

11.6 Why Debit and Commit are separate

This is the most important idea in the chapter, and it is a design lesson far beyond NFC.

`Debit` does **not** change the balance. It stages a delta in the card's transaction buffer. Only `CommitTransaction` makes it permanent.

Why? Because **a contactless card can leave the field at any instant**, mid-command, with no warning. Consider what a single atomic "debit" command would mean: the card is debited, the reply is lost as the user pulls the card away, the machine never learns it succeeded, and no coffee is dispensed. The customer has paid for nothing and you cannot prove otherwise.

With staging, the failure mode is safe: no commit means the card silently discards the delta and the balance is exactly what it was.

> **CONCEPT Make the ambiguous case the harmless one**
>
> **The worst case becomes "nothing happened" instead of "money vanished".** That is why the kiosk must dispense only *after* the commit is acknowledged, and why `PaymentReceipt` in Chapter 12 is constructed only after the commit returns. Whenever you design an operation that can be interrupted, ask: what does the interrupted state look like, and who pays for it? Then arrange for the answer to be "nobody".

11.7 Verify — the full offline replay

Fresh session, so debit runs at ctr 0:

Step	ctr	Bytes
`debit(0x01, 150)` sends	0	`01a2eb8f000bf3ddfe6e18e885 9b28459127e5ea2cb8acad7d`
card's debit reply MAC	1	`6b082e8062d1a89d`
`commitTransaction()` sends	1	`4dc9859e1a783ee0`
card's commit reply MAC	2	`febea1fcb55208ba`
`getValue` reply (balance 1000)	1	`7950692fc2f01e72b3cee290f7122b5c + a44a7f2359b2f512`

```
// extra cases in the fake's transceive switch
case (byte) 0x6C:                       // GetValue
    return getValueReply;
case (byte) 0xDC:                       // Debit
    assertArrayEquals(
        h("01a2eb8f000bf3ddfe6e18e8859b28459127e5ea2cb8acad7d"),
        Arrays.copyOfRange(apdu, 5, apdu.length - 1), "debit bytes wrong");
```

```
        return debitReply;
    case (byte) 0xC7:                       // CommitTransaction
        assertArrayEquals(h("4dc9859e1a783ee0"),
            Arrays.copyOfRange(apdu, 5, apdu.length - 1), "commit MAC wrong");
        return commitReply;

    @Test
    public void debitAndCommit_matchesVectors() throws SpiException {
        DesfireCard card = new DesfireCard(this, this);
        card.selectApplication(new byte[]{ 0x01, 0x02, 0x03 });
        card.authenticateEv2First((byte) 0x00, key);    // ctr 0
        card.debit((byte) 0x01, 150);
        card.commitTransaction();
    }
```

Put debit/commit in a **separate** `@Test` from the getValue one. Each needs a fresh session, because otherwise they disagree about the counter.

> **TRAP The green test that tests nothing**
>
> After wiring the `0x6C` case into your fake, it is entirely possible to have a passing test that never calls `getValue`. The test goes green, you feel good, and nothing was verified. The assertion must be `assertEquals(1000, card.getValue((byte) 0x01))` — not merely a `case` in a switch. After writing any test, ask: *what would I have to break to make this fail?* If the answer is "nothing", it is not a test.

11.8 Then: the real thing

```
int before = desfire.getValue((byte) 0x01);
System.out.printf("balance = %d cents%n", before);

desfire.debit((byte) 0x01, 10);          // stage 10 cents
desfire.commitTransaction();             // make it permanent
int after = desfire.getValue((byte) 0x01);
System.out.printf("%d -> %d cents%n", before, after);
```

> **VERIFICATION GATE**
>
> ```
> balance = 510 cents (5.10 EUR)
> ```

```
debiting 10 cents...
510 -> 500 cents (5.10 -> 5.00 EUR)
```

You have just moved real money with code you wrote from the SPI bytes up.

Every layer — register encoding, FIFO handling, anticollision, block framing, CMAC, key derivation, mutual authentication, secure messaging — is yours.

11.9 Traps

You see	Cause / fix
Auth OK, first `getValue` fails its MAC	The TI one-liner: session built without TI.
Command MACs right, response MACs wrong	Increment ordering. It happens *before* verifying the response.
`91 BE` (boundary error)	Debit would take the balance below the file's lower limit.
`91 9D` (permission denied)	Authenticated with the wrong key for that file's access rights.
`91 F0` (file not found)	The value file does not exist — the card needs provisioning.
Balance unchanged after debit	You forgot `commitTransaction()`. Working as designed.
First command works, second fails	Not incrementing, or incrementing twice.
Everything fails after one bad run	Card left mid-transaction. Power-cycle the field, re-authenticate.

11.10 Commit

```
git add -A
git commit -m "M7d: EV2 secure messaging + GetValue/Debit/Commit -- money moves"
```

PART V

Product

CHAPTER 12

The kiosk

GOAL A touchscreen that takes payments — without a single line of crypto in the UI.

CONCEPT The Adapter pattern, and threading discipline.

YOU END WITH Tap a drink, tap a card, watch it brew.

12.1 The problem

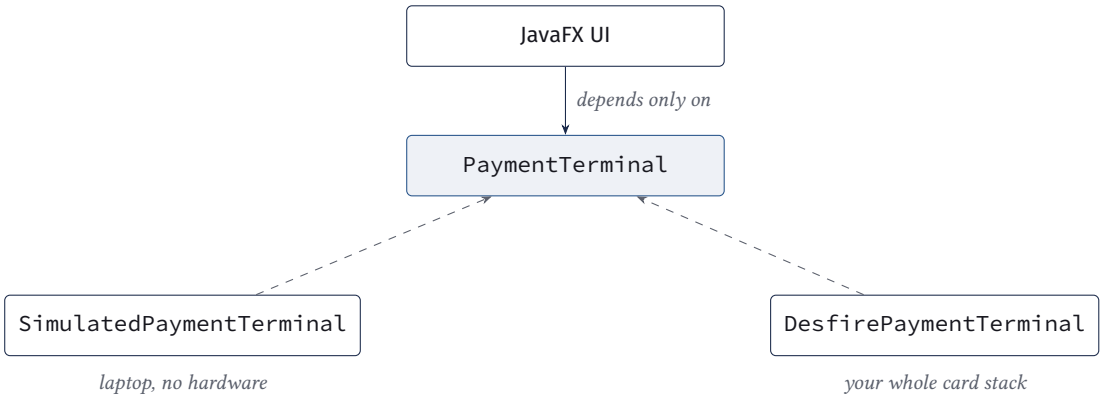
You have a working payment stack. Now you need a screen. Two constraints make this harder than it looks:

1. You want to build the UI *before* the hardware works — or at least be able to develop it on a laptop with no reader attached.

2. Card operations take *seconds* (polling, RF, crypto), and a UI that blocks for seconds is a broken UI.

12.2 The idea: one interface, two implementations

Define what the UI needs in the smallest possible terms, then satisfy it twice.



The same seam idea as `SpiLink` and `ApduChannel`, now at the top of the stack. It means the entire UI can be built and tested before a single card exists, and the kiosk degrades gracefully: no reader attached, so use the simulator, and the app still runs.

12.3 The contract

```
package com.midnightbrewer.payment;

/**
 * The seam between the kiosk UI and whatever actually takes the money.
 *
 * The UI depends only on this type, so it neither knows nor cares whether
 * payment is a simulated wallet or a real DESFire being debited over SPI.
 *
 * Extends AutoCloseable so the RF field is always powered down, even on a
 * crash.
 */
public interface PaymentTerminal extends AutoCloseable {

    /**
     * Begin waiting for a card and charge it for drink.
     *
     * MUST return immediately -- all polling happens on a background thread.
     * Exactly one terminal callback (approved or declined) is delivered per
     * call, unless cancel() intervenes.
     */
    void startTransaction(Drink drink, PaymentListener listener);

    /** Abort the in-flight transaction. Safe to call when nothing is running. */
```

```
    void cancel();

    /** Human-readable state for the status pill, e.g. "RC522 READY". */
    String getStatusText();

    @Override
    void close();
}
```

```
public interface PaymentListener {
    /** A card entered the field. Not yet authenticated. */
    void onCardDetected(String uid);
    /** Authenticated and balance read, before any debit. */
    void onBalanceRead(int balanceCents);
    /** Debit COMMITTED. The drink may now be dispensed. */
    void onApproved(PaymentReceipt receipt);
    /** Failed. The card was not charged, or the debit was rolled back. */
    void onDeclined(CardException cause);
}
```

CONCEPT Why callbacks, and why four of them

Why not a return value? Because `startTransaction` returns instantly and the result arrives seconds later. A blocking `boolean pay(Drink)` would freeze the UI thread: the animation stops, touches queue up, and the operating system offers to kill your app.
Why four callbacks? So the UI can narrate. "Card detected…" then "Balance €5.00, charging…" then approved. A payment terminal that shows nothing for four seconds feels broken even when it is working perfectly.

12.4 The Adapter

Now the interesting class: something that satisfies `PaymentTerminal` by driving the stack you built in Parts II–IV.

ANALOGY The travel plug adapter

Your laptop charger has one plug shape; the Italian wall has another. The adapter does not generate electricity or modify the laptop — it sits between two fixed interfaces and translates.

> Both sides were designed without knowledge of each other, and the adapter makes them fit. Here the kiosk speaks `PaymentTerminal` and `CardException`; your stack speaks `Rc522Reader`, `DesfireCard` and `SpiException`. `DesfirePaymentTerminal` translates — including the exceptions, so a `com.midnightbrewer.hardware` type never leaks into the UI.

```
package com.midnightbrewer.payment;

public class DesfirePaymentTerminal implements PaymentTerminal {

    // How the card was personalised (Chapter 12 / Appendix A).
    private static final byte[] APP_AID       = { 0x03, 0x02, 0x01 };
    private static final byte   USER_KEY_NO   = 0x02;
    private static final byte   VALUE_FILE_NO = 0x01;

    private final Pi4jSpiLink link;
    private final Rc522Driver driver;
    private final Rc522Reader reader;

    // ONE background thread, created once. Transactions are strictly
    // sequential -- two overlapping card conversations would interleave
    // command counters and corrupt both.
    private final ExecutorService worker =
            Executors.newSingleThreadExecutor(r -> {
                Thread t = new Thread(r, "desfire-terminal");
                t.setDaemon(true);      // never blocks JVM shutdown
                return t;
            });

    /** Written by the UI thread, read by the worker. */
    private final AtomicBoolean cancelled = new AtomicBoolean(false);
    private volatile boolean scanning = false;

    /** Opens the reader. THROWS if there is no RC522 -- see createTerminal(). */
    public DesfirePaymentTerminal() throws SpiException {
        this.link = new Pi4jSpiLink();
        this.driver = new Rc522Driver(link);
        try {
            initReader();
            int version = driver.version();
            if (version != 0x91 && version != 0x92) {
                throw new SpiException("RC522 not responding");
            }
        } catch (SpiException e) {
            quietClose();               // do not leak SPI on a failed construction
```

```
            throw e;
        }
        this.reader = new Rc522Reader(driver);
    }

    @Override
    public void startTransaction(Drink drink, PaymentListener listener) {
        cancelled.set(false);
        worker.submit(() -> runTransaction(drink, listener));   // returns instantly
    }

    private void runTransaction(Drink drink, PaymentListener listener) {
        scanning = true;
        try {
            // Fresh reader state each tap: softReset + antennaOn power-cycles
            // the field so the card is IDLE again and activate() runs cleanly
            // even after a previous purchase.
            initReader();

            if (!waitForCard()) return;         // cancelled; the UI moved on

            ActivatedCard card = reader.activate();
            listener.onCardDetected(card.uidHex());

            DesfireCard desfire = new DesfireCard(
                    new IsoDepChannel(reader), RandomSource.secure());
            desfire.selectApplication(APP_AID);
            desfire.authenticateEv2First(USER_KEY_NO, userKey());

            int balance = desfire.getValue(VALUE_FILE_NO);
            listener.onBalanceRead(balance);

            int price = drink.getPriceCents();
            if (balance < price) {
                listener.onDeclined(new InsufficientFundsException(balance, price));
                return;
            }

            // Provisional until the commit is acknowledged; only THEN may we
            // hand out a receipt, and only then may the UI dispense.
            desfire.debit(VALUE_FILE_NO, price);
            desfire.commitTransaction();
            int after = desfire.getValue(VALUE_FILE_NO);

            listener.onApproved(
                new PaymentReceipt(card.uidHex(), drink, balance, after));
```

```
        } catch (SpiException e) {
            listener.onDeclined(translate(e));  // stack vocabulary -> UI vocabulary
        } catch (RuntimeException e) {
            listener.onDeclined(new CardCommunicationException(
                    "unexpected reader error: " + e.getMessage(), e));
        } finally {
            scanning = false;
        }
    }

    private boolean waitForCard() throws SpiException {
        while (!cancelled.get()) {
            if (reader.isCardPresent()) return true;
            sleep(30);
        }
        return false;
    }
}
```

> **CONCEPT `catch (RuntimeException)` is not paranoia**
>
> A kiosk must never die on a bad tap. `SpiException` covers expected failures; the `RuntimeException` catch covers the unexpected — an array index, a null, a Pi4J internal error.
>
> Without it, one odd card kills the worker thread and the machine stops taking payments until someone reboots it.
>
> **At a process boundary, catch broadly and report. Inside your logic, catch narrowly.**

12.5 Threading: the rule you must not break

JavaFX has exactly one thread allowed to touch the UI. Your terminal calls back from a *different* thread. So every callback hops:

```
private final class UiPaymentListener implements PaymentListener {

    @Override
    public void onCardDetected(String uid) {
        Platform.runLater(() ->
            paymentHintLabel.setText("Card detected -- authenticating..."));
    }
```

```
    @Override
    public void onApproved(PaymentReceipt receipt) {
        Platform.runLater(() -> { stopPulse(); startBrewing(receipt); });
    }
}
```

> **TRAP Off-thread UI access does not reliably throw**
>
> Sometimes it works. Sometimes it corrupts rendering in ways that appear once an hour, on the real panel only, and never under a debugger.
> This is one of the nastiest bug classes in UI programming, and the fix is a habit, not a diagnosis: **every callback that touches the scene graph is wrapped in `Platform.runLater`.** No exceptions.
> The same rule exists everywhere: Android has `runOnUiThread`, Swing has `invokeLater`, Qt has queued connections. One thread owns the widgets.

12.6 Choosing the backend

The whole design collapses into one method:

```
private PaymentTerminal createTerminal() {
    if ("simulated".equalsIgnoreCase(System.getProperty("brewer.reader", ""))) {
        return new SimulatedPaymentTerminal();          // forced, for UI work
    }
    try {
        PaymentTerminal real = new DesfirePaymentTerminal();
        System.out.println("[UI] RC522 ready -- real card payment enabled");
        return real;
    } catch (Exception e) {
        System.out.println("[UI] no RC522 (" + e.getMessage()
                + ") -- using the simulator");
        return new SimulatedPaymentTerminal();
    }
}
```

Detection by attempting to open. On a laptop, constructing `Pi4jSpiLink` throws and you fall back. The same build runs everywhere: no flags, no config file, no `#ifdef`. This is precisely why the constructor *must* throw rather than returning a half-built object.

And because everything above depends on the interface, the UI needs **zero** changes to switch backends. The bench "simulate tap" button even hides itself:

```
devTapButton.setVisible(terminal instanceof SimulatedPaymentTerminal);
```

12.7 Verify

On a laptop (no hardware): `mvn javafx:run -Dbrewer.windowed=true`. The simulator's tap button drives the whole flow.

On the Pi, run a headless smoke test first — it exercises the exact path a tap takes, minus the screen:

```
public final class TerminalCheck {
    public static void main(String[] args) throws Exception {
        Drink testDrink = new Drink("test", "Test Shot", 10);   // 10 cents
        try (DesfirePaymentTerminal terminal = new DesfirePaymentTerminal()) {
            CountDownLatch done = new CountDownLatch(1);
            terminal.startTransaction(testDrink, new PaymentListener() {
                public void onCardDetected(String uid)   { print("card: " + uid); }
                public void onBalanceRead(int cents)     { print("balance: " + cents);
   ↪  }
                public void onApproved(PaymentReceipt r) { print("APPROVED"); done.
   ↪ countDown(); }
                public void onDeclined(CardException e)  { print("DECLINED"); done.
   ↪ countDown(); }
            });
            System.out.println("Tap a card...");
            done.await();
        }
    }
}
```

`CountDownLatch` is how `main` waits for a background thread: start at one, block on `await()`, and the callback counts down to zero to release it.

VERIFICATION GATE

```
card: 04 76 18 62 7B 20 90
balance: $5.00
APPROVED: charged $0.10  ($5.00 -> $4.90)
```

12.8 Traps

You see	Cause / fix
UI freezes on tap	Card work on the FX thread. It must be on the worker.
Random rendering corruption	A callback touching nodes without `Platform.runLater`.
Second tap always fails	Reader not re-initialised per transaction.
Kiosk dies on an odd card	Missing the broad `RuntimeException` catch.
Simulator used on the Pi	Constructor threw — read the printed message, it names the cause.
Drink dispensed but card not charged	Dispensing on `debit` instead of after `commitTransaction`.

12.9 Commit

```
git add -A
git commit -m "kiosk: PaymentTerminal seam + DesfirePaymentTerminal adapter"
```

CHAPTER 13

The Android top-up app

GOAL Put credit *onto* the card from a phone. Build it yourself, in Dart, from the design you already understand.

CONCEPT Porting a design across languages — and discovering how much of your stack was incidental.

YOU END WITH A phone that adds credit, and cross-language proof your crypto is right.

13.1 Why this chapter is different

You have built the whole reader stack once. The point of this chapter is not to learn NFC again. It is to discover something valuable:

Six of your ten chapters do not exist on Android.

The phone has an NFC controller with a driver already written. Android's `IsoDep` class hands you `transceive(byte[]) -> byte[]` directly. That single method is everything you built in Chapters 2–7.

Your Pi stack	Android
Ch. 2 `SpiLink`	*gone* — the OS owns the radio
Ch. 3 `Rc522Driver`	*gone*
Ch. 4 `Rc522Reader`	*gone*
Ch. 5 `ProtocolBlock`	*gone* — the OS frames I-blocks
Ch. 6 activation	*gone* — you are handed an activated tag
Ch. 7 `IsoDepChannel`	**`IsoDep.transceive()`**
Ch. 8–11 crypto	**you port all of it, unchanged**

CONCEPT The transport was replaceable; the protocol was the work

Look at the shape of your `ApduChannel` interface from Chapter 7 and Android's `IsoDep.transceive`. They are the same.

That is not luck — it is what happens when you draw a seam at the right place. Everything above `ApduChannel` ports across a language, an operating system, and a completely different radio, without changing.

If you want one test of whether an abstraction was well chosen, this is it: *how much survives when the thing underneath is replaced entirely?*

ANALOGY Posting the letter

You have been writing a letter and hand-delivering it. Now you are posting it. The envelope, the courier, the van — all different. The letter is identical.

13.2 Setup

```
flutter create --org com.midnightbrewer --platforms android topup_app
cd topup_app
```

```
dependencies:
  flutter:
    sdk: flutter
  # Android IsoDep (ISO 14443-4) transceive.
  # 4.x has a different API from 3.x -- pin the major version.
  nfc_manager: ^4.2.1
  # AES-ECB/CBC and AES-CMAC.
  pointycastle: ^4.0.0
```

And in `android/app/src/main/AndroidManifest.xml`:

```
<uses-permission android:name="android.permission.NFC" />
<uses-feature android:name="android.hardware.nfc" android:required="true" />
```

13.3 Dart for a Java programmer

The languages are close cousins. Four differences matter here.

`Uint8List`, not `byte[]`.
From `dart:typed_data`. Crucially its elements are **unsigned 0–255**: Dart has no signed-byte trap, and all those `& 0xFF` masks simply vanish.

`int` is 64-bit.
So `1 « 40` works fine, but when writing a 32-bit value you must mask deliberately: `(v » 24) & 0xFF`.

Async is explicit.
NFC calls are `Futures`. A function that awaits must be `async`, and its callers must `await` it — the colouring propagates upward, much as `throws SpiException` did in Java. Same discipline, different keyword.

No checked exceptions.
Dart has `throw`/`catch` but the compiler forces nothing. You lose the nagging, so you must be more deliberate about error paths.

13.4 Build it

13.4.1 The transport — one class replaces six chapters

```dart
import 'dart:typed_data';
import 'package:nfc_manager/nfc_manager.dart';

/// Sends one APDU to an activated card and returns the reply.
///
/// This is the ENTIRE transport layer -- the Android NFC stack has already
/// done activation, anticollision, RATS and ISO-DEP framing for us.
/// Compare Chapters 2-7 of the Pi build. Same seam, no work.
class IsoDepTransceiver {
  IsoDepTransceiver(this._isoDep);

  final IsoDep _isoDep;

  Future<Uint8List> transceive(Uint8List apdu) async {
    // TODO: return await _isoDep.transceive(data: apdu);
    //   Wrap failures in your own exception type so the UI never sees a
    //   platform exception -- the same translate() discipline as Chapter 12.
```

```
    throw UnimplementedError();
  }
}
```

13.4.2 The crypto — port it directly

Your Java from Chapters 8–11 translates almost line for line. `pointycastle` supplies AES *and* CMAC, so you do not even reimplement RFC 4493 — but you must verify it produces the same bytes.

```
import 'dart:typed_data';
import 'package:pointycastle/export.dart';

const int kAesBlockSize = 16;

Uint8List aesEcbEncryptBlock(Uint8List key, Uint8List block) {
  final engine = AESEngine()..init(true, KeyParameter(key));
  final out = Uint8List(kAesBlockSize);
  engine.processBlock(block, 0, out, 0);
  return out;
}

Uint8List aesCbcEncrypt(Uint8List key, Uint8List iv, Uint8List data) {
  // TODO: CBCBlockCipher(AESEngine())
  //         ..init(true, ParametersWithIV(KeyParameter(key), iv));
  //       process block by block; NO padding (DESFire pads itself).
  throw UnimplementedError();
}

Uint8List aesCmac(Uint8List key, Uint8List message) {
  final mac = CMac(AESEngine(), 128)..init(KeyParameter(key));
  return mac.process(message);
}

/// DESFire keeps the ODD-indexed bytes: mac[1], mac[3], ... mac[15].
Uint8List truncateMac(Uint8List mac) {
  // TODO: 8 bytes, out[i] = mac[i * 2 + 1];
  throw UnimplementedError();
}

/// The 32-byte derivation vector, exactly as in Chapter 9.
Uint8List deriveSessionKey(
    Uint8List key, Uint8List rndA, Uint8List rndB, int label) {
  // TODO: sv[0] = label (0xA5 enc / 0x5A mac); sv[1] = the other;
```

```
  //       sv[2..5] = 00 01 00 80;
  //       sv[6..7] = rndA[0..1];
  //       sv[8..13] = rndA[2..7] ^ rndB[0..5];
  //       sv[14..23] = rndB[6..15];
  //       sv[24..31] = rndA[8..15];
  //       return aesCmac(key, sv);
  throw UnimplementedError();
}

/// ISO 7816-4: append 0x80 then zeros. ALWAYS adds at least one byte.
Uint8List padIso7816(Uint8List data) {
  final paddedLength = (data.length | 0x0F) + 1;
  final out = Uint8List(paddedLength);
  out.setRange(0, data.length, data);
  out[data.length] = 0x80;
  return out;
}
```

VERIFICATION GATE

Verify with the same vectors. This is the elegant part: the vectors in Appendix A are language-agnostic. Your Dart must produce byte-for-byte what your Java produced.

```
void main() {
  test('derives the Chapter 9 session keys', () {
    final key  = fromHex('00112233445566778899aabbccddeeff');
    final rndA = fromHex('a0a1a2a3a4a5a6a7a8a9aaabacadaeaf');
    final rndB = fromHex('b0b1b2b3b4b5b6b7b8b9babbbcbdbebf');

    expect(toHex(deriveSessionKey(key, rndA, rndB, 0xA5)),
           'd46c03528ecd3b1f1148e708d3e9729d');
    expect(toHex(deriveSessionKey(key, rndA, rndB, 0x5A)),
           '116d156cc502da820eb57915f189ec74');
  });

  test('truncates a CMAC to the odd bytes', () {
    expect(toHex(truncateMac(fromHex('00112233445566778899aabbccddeeff'))),
           '1133557799bbddff');
  });
}
```

```
flutter test           # no phone, no card -- same discipline as the Pi build
```

> If your Dart disagrees with your Java on any vector, **one of them is wrong**, and you now know before either touches a card. Two independent implementations converging on the same bytes is much stronger evidence than either alone.

13.4.3 Credit — the mirror image of debit

```dart
/// Credit (0x0C) in CommMode.Full -- the mirror image of Debit.
///
/// Identical wire format; only the command byte differs. Like debit it only
/// STAGES the change: nothing moves until commitTransaction().
Future<void> credit(int fileNo, int amountCents) async {
  // TODO: same shape as Chapter 11's debit:
  //   header     = [fileNo]
  //   ciphertext = session.encryptCommandData(encodeValue(amountCents))
  //   macInput   = header + ciphertext
  //   mac        = session.commandMac(0x0C, macInput)
  //   send, check status, incrementCounter, verify response MAC over empty
  throw UnimplementedError();
}
```

And the top-up sequence, which reads the balance back:

```dart
Future<int> topUp(int fileNo, int amountCents) async {
  final before = await getValue(fileNo);
  await credit(fileNo, amountCents);
  await commitTransaction();
  final after = await getValue(fileNo);      // trust nothing; re-read

  if (after != before + amountCents) {
    throw StateError('balance did not move as expected');
  }
  return after;
}
```

Reading the balance back after the commit is not paranoia — it is the only way to *know*. If the phone loses the field between `credit` and `commitTransaction`, the card discards the delta and the balance is unchanged. Same safety property as Chapter 11, now protecting your customer's money rather than yours.

13.5 Provisioning: turning a blank card into a wallet

A factory-fresh DESFire has no application, no value file, and a DES master key. To make one work with your kiosk:

1. authenticate at PICC level with the master key (16 × `0x00` factory default);
2. `CreateApplication` (`0xCA`) — AID `0x010203`, 3 keys, AES;
3. select the application, authenticate with its master key (initially 16 × `0x00`);
4. `ChangeKey` (`0xC4`) — set app master key (`0x11…`) and user key (`0x22…`);
5. re-authenticate with the new app master key;
6. `CreateValueFile` (`0xCC`) — file `0x01`, CommMode.Full, access via key 2, limits 0…100000, initial value 0.

Wire format for `CreateValueFile`, 17 bytes:

```
[0]      fileNo
[1]      commSettings          0x03 = Full
[2..3]   accessRights          little-endian; nibbles: read|write|rw|changeRights
[4..7]   lowerLimit            signed LE
[8..11]  upperLimit            signed LE
[12..15] initialValue          signed LE
[16]     limitedCreditEnabled  0x00 / 0x01
```

The access-rights word packs four 4-bit key numbers: *read* (GetValue), *write* (Debit), *readWrite* (Credit too), and *changeAccessRights*. Using key 2 for all four gives `0x2222`. Nibble `0xE` means free access; `0xF` means never.

> **TRAP `ChangeKey` is irreversible**
>
> Set a key to a value you do not record and that card is scrap — permanently, by design. There is no recovery path; that *is* the security property.
> Before you run provisioning:
>
> - use **dedicated test cards** you bought for this project;
> - never change the PICC master key from its factory default (this project deliberately does not, so any other tool can re-personalise the card);
> - never point this at a card you did not buy for this — the DESFire in your wallet may be your office badge or transit pass, and it will not survive.

> And **never auto-provision on tap**. Require an explicit "Set up this card" button press. A member of the public tapping your app must never have their existing card silently rewritten.

13.6 The security reality of hardcoded keys

Your app will contain, in plaintext, the AES keys for every card in the fleet. Anyone who pulls the APK off a phone can extract them and mint unlimited balance on every card you have issued.

For a bench project with test cards this is fine. Say so out loud, in a comment, so nobody mistakes it for a design:

```
/// BENCH KEYS. These are compiled into the APK in plaintext. Anyone who pulls
/// the APK can read them and mint unlimited balance on every card, because the
/// same key is on every card. Acceptable for a bench rig with test cards.
/// NOT acceptable for anything representing real money.
```

> **CONCEPT What a real deployment does instead**
>
> Worth understanding even if you never build it.
> **1. Key diversification.** Each card gets a *different* key, derived from the card's UID and a master key (typically via CMAC). Compromise one card and you learn one card's key — not the fleet's.
> **2. A backend HSM or SAM.** The master key never leaves tamper-resistant hardware. The phone becomes a dumb pipe: it relays APDUs between the card and the server, and the *server* performs the cryptography.
> **3. Server-side transaction logs**, so the balance on the card is a cache rather than the source of truth, and double-spends are detectable after the fact.
> The gap between "works" and "safe to deploy" is mostly key management, and it is why payment systems have HSMs. Knowing exactly where your bench rig sits on that spectrum is the professional part.

13.7 Build order

Do it in this order:

1. `flutter create` plus dependencies; blank app runs on your phone.
2. **Crypto module and tests.** No NFC yet. All vectors green.
3. `IsoDepTransceiver` and a "read the card's version" screen. Prove `90 60 00 00 00` returns `...91AF`.
4. `authenticateEv2First`, on a card the kiosk already works with.
5. `getValue` — display the balance. Read-only, so mistakes are harmless.
6. `credit` + `commit` + read-back. Now money moves.
7. Provisioning screen, last, behind an explicit button.
8. UI polish — the NFC pulse animation, preset amounts, result screen.

Steps 2 and 5 are the natural stopping points if you are filming: "the crypto agrees across two languages" and "the phone reads the balance the Pi wrote".

13.8 Traps

You see	Cause / fix
`nfc_manager` API does not match tutorials	3.x and 4.x differ. Pin 4.x and use its docs.
Works on one phone, not another	NFC antenna placement varies enormously. Try the upper third of the back.
Tag lost mid-transaction	Phone moved. This is *normal* — it is why commit exists. Handle it.
Dart and Java disagree on a MAC	Almost always the counter or TI, not the AES. Print both inputs.
`91 AE` after provisioning	You changed a key and re-authenticated with the old one.
Card bricked	You changed a key you did not record. There is no recovery. This is why you buy spares.

13.9 Commit

```
git add -A
git commit -m "topup: Flutter app -- crypto ported, credit + commit, provisioning"
```

CHAPTER 14

Deploying to the Pi

GOAL The machine boots into the kiosk: fullscreen, correctly rotated, GPU-accelerated, no keyboard attached.

CONCEPT Operational discipline — the difference between a demo and a machine.

YOU END WITH Something you can hang on a wall and walk away from.

14.1 Rotating the display

A 7-inch panel mounted vertically needs the *compositor* rotated, not the app. Raspberry Pi OS (Bookworm and later) uses **labwc** on Wayland.

`~/.config/labwc/autostart`:

```
# 1024x600 panel mounted rotated -> the app sees a 600x1024 portrait screen.
wlr-randr --output HDMI-A-1 --transform 90 &
```

Check your output name with `wlr-randr`; it may be `HDMI-A-2`.

Touch must be rotated too, and this is the step everyone forgets. An unrotated touchscreen on a rotated display gives inverted axes: you press the top-left tile and the bottom-right one activates. Maddening, and it looks like a UI bug.

In `~/.config/labwc/rc.xml`, inside `<libinput>`:

```
<device category="touch">
  <transformationMatrix>0 -1 1 1 0 0</transformationMatrix>
</device>
```

That matrix is a 90° rotation. For 270° use `0 1 0 -1 0 1`. If touch ends up mirrored rather

than rotated, you have the wrong one.

> **CONCEPT Why the app does not rotate itself**
>
> If JavaFX rendered rotated, every animation would run through a transform, the compositor would still composite unrotated, and touch would *still* be wrong.
> Rotating at the compositor means everything below it — including input — is consistently transformed, and the app simply sees a portrait screen. Apply a transformation at the lowest layer that owns all the affected data.

14.2 GPU acceleration: the one flag that matters

JavaFX on a Pi will silently fall back to software rendering, and the difference is night and day: a fade that takes 16 ms on the GPU takes 400 ms on the CPU, and your kiosk feels broken.

The cause is specific. The VideoCore VI driver reports **OpenGL 2.1**, and JavaFX's Prism pipeline checks for a higher version and refuses. The GPU is perfectly capable; the version check is wrong.

```
java -Dprism.forceGPU=true \
     -Dprism.verbose=true \
     -cp "target/classes:target/dependency/*" com.midnightbrewer.ui.MainApp
```

`-Dprism.verbose=true` prints the chosen pipeline at startup. You want to see `es2` (hardware). If it says `sw`, you are on the CPU.

Also worth setting in `/boot/firmware/config.txt`:

```
dtoverlay=vc4-kms-v3d
gpu_mem=128
```

14.2.1 Other things that mattered on a Zero 2 W

- **Cache the pulse rings.** `ring.setCache(true); ring.setCacheHint(CacheHint.SPEED);` Without it, JavaFX re-rasterises the vector circle every frame while it scales. With it, you are scaling a bitmap, which is nearly free.
- **Hide the cursor** (`scene.setCursor(Cursor.NONE)`). Xwayland parks a pointer mid-screen; it sits on a tile and leaves it stuck in `:hover`, which reads as a pre-selected drink.

- **Scope hover styling** to a `.desktop` CSS class you enable only in windowed development mode. A touchscreen has no hover state to represent.

14.3 Autostart with systemd

`/etc/systemd/system/midnight-brewer.service`:

```
[Unit]
Description=The Midnight Brewer kiosk
After=graphical.target

[Service]
Type=simple
User=martin
Environment=WAYLAND_DISPLAY=wayland-0
Environment=XDG_RUNTIME_DIR=/run/user/1000
WorkingDirectory=/home/martin/midnight-brewer
ExecStart=/usr/bin/java -Dprism.forceGPU=true \
    -cp "target/classes:target/dependency/*" com.midnightbrewer.ui.MainApp
Restart=always
RestartSec=5

[Install]
WantedBy=graphical.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now midnight-brewer
journalctl -u midnight-brewer -f      # watch it
```

> **CONCEPT** **`Restart=always` is the important line**
>
> A kiosk that dies at three in the morning and stays dead is worse than one that restarts in five seconds.
> Combined with the broad `catch (RuntimeException)` from Chapter 12, you have two layers of resilience: bad taps do not kill the worker thread, and a fatal crash does not kill the machine.
> Design for the failure you cannot foresee by making recovery automatic.

Remember the group membership, or Pi4J cannot open the device:

```
sudo usermod -aG spi,gpio martin      # log out and back in
```

14.4 The deploy loop

Compiling on a Zero 2 W is slow enough to break your concentration. Two options.

Rsync the source, build on the Pi (simple, about 40 seconds):

```
rsync -av --exclude target/ ./ martin@raspberrypi.local:~/midnight-brewer/
ssh martin@raspberrypi.local 'cd ~/midnight-brewer && mvn -q -DskipTests compile'
```

Or build locally and ship the classes (about 5 seconds). Java bytecode is architecture-independent, so a laptop build runs unchanged on ARM:

```
mvn -q -DskipTests compile
rsync -av target/classes/ martin@raspberrypi.local:~/midnight-brewer/target/classes/
```

Do the dependency copy *once*:

```
ssh martin@raspberrypi.local \
  'cd ~/midnight-brewer && mvn dependency:copy-dependencies -DincludeScope=runtime'
```

> **TRAP Stale classes**
>
> If you change a file and the old `.class` survives, you will debug behaviour that no longer exists in your source.
> When something makes no sense *at all* — when the code in front of you cannot possibly produce the output you are seeing — run `mvn clean compile` before anything else.

14.5 Field hygiene

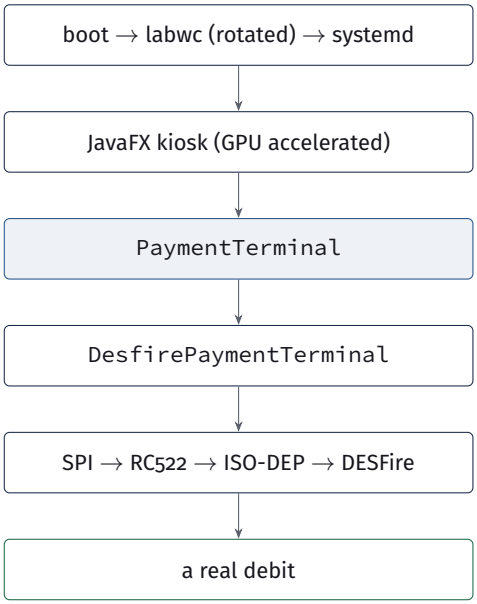
Three habits that keep a kiosk healthy.

Power-cycle the RF field per transaction.
`softReset()` + `antennaOn()` at the start of each tap puts any lingering card back to IDLE. Without it, a card left on the antenna after one purchase is in the wrong state for the next.

Always close the link.
try-with-resources on `Pi4jSpiLink`, and `close()` in the JavaFX `stop()` hook. An RF field left energised overnight cooks the module.

Never log key material.
It is tempting during debugging to print session keys. If that line survives into production, your logs are now a key store. Print the *TI* and the *counter* instead — they tell you everything you need for protocol debugging and nothing an attacker wants.

14.6 The finished machine



14.7 Commit

```
git add -A
git commit -m "deploy: rotation, GPU flag, systemd unit, deploy scripts"
```

PART VI

Appendices

CHAPTER A

Test vectors

Everything you need to verify the cryptography offline, in one place. All values are hexadecimal, lowercase, unspaced.

> **CONCEPT Why these are trustworthy**
>
> They were generated with Python's `cryptography` library, deliberately *not* with the Java you are writing. Agreement therefore means your implementation is **correct**, not merely self-consistent.
> Regenerate or extend them with `tools/m7d_vectors.py` in the companion repository.

A.1 AES-CMAC — RFC 4493 (Chapter 8)

Key K = `2b7e151628aed2a6abf7158809cf4f3c`

Subkeys:

```
K1 = fbeed618357133667c85e08f7236a8de
K2 = f7ddac306ae266ccf90bc11ee46d513b
```

#	Message	CMAC
1	*(empty)*	`bb1d6929e95937287fa37d129b756746`
2	`6bc1bee22e409f96e93d7e117393172a`	`070a16b46b4d4144f79bdd9dd04a287c`
3	`6bc1bee22e409f96e93d7e117393172a` `ae2d8a571e03ac9c9eb76fac45af8e51` `30c81c46a35ce411`	`dfa66747de9ae63030ca32611497c827`
4	message 3 + `e5fbc1191a0a52ef` `f69f2445df4f9b17ad2b417be66c3710`	`51f0bebf7e3b9d92fc49741779363cfe`

The four cases cover every branch: empty (pad, K_2), one full block (K_1), partial last block (pad, K_2), and multiple full blocks (K_1).

A.2 The reference session (Chapters 9–11)

Every vector below derives from these three inputs. They are chained deliberately: the same session runs through key derivation, the authentication replay, and secure messaging, so a failure anywhere points at one specific step.

```
authKey = 00112233445566778899aabbccddeeff
rndA    = a0a1a2a3a4a5a6a7a8a9aaabacadaeaf     (pin this in tests)
rndB    = b0b1b2b3b4b5b6b7b8b9babbbcbdbebf
```

A.2.1 Session keys

```
Kenc = d46c03528ecd3b1f1148e708d3e9729d      (label 0xA5)
Kmac = 116d156cc502da820eb57915f189ec74      (label 0x5A)
```

A.2.2 Byte rotation

```
rotateLeft(b0b1b2b3b4b5b6b7b8b9babbbcbdbebf)
         = b1b2b3b4b5b6b7b8b9babbbcbdbebfb0
```

A.3 The authentication replay (Chapter 10)

A complete `AuthenticateEV2First` exchange with `rndA` pinned. All AES uses an **all-zero IV**.

Direction	Bytes
card → you, step 1 $E(K, RndB)$	`eab6822fe368d5bc9895fb2558b38dde`
you → card, step 2 $E(K, RndA \| RndB')$	`cf086a82c0b745a749daabb28a7a8db3` `32272f9e5fdc37a4746064f25b7c2ca7`
card → you, step 2 $E(K, TI \| RndA' \| caps)$	`4e08e447d74810052caa1c5b6c60343f` `51f6e901ad171e677b100d3a34c925e4`
→ extracted TI	`11223344`

TRAP Fake replies must carry their status word

Your card code strips the last two bytes off every response, so a fake that omits them chops real ciphertext.

```
step-1 reply = eab6822fe368d5bc9895fb2558b38dde + 91af
step-2 reply = 4e08e447d74810052caa1c5b6c60343f
               51f6e901ad171e677b100d3a34c925e4 + 9100
select reply = 9100
```

Cross-check. Decrypting `eab6822f...` with `authKey` and a zero IV must yield exactly `b0b1b2b3b4b5b6b7b8b9babbbcbdbebf`. If it does, your authentication vectors and your session-key vectors are consistent with each other.

A.4 Secure messaging (Chapter 11)

Session as above, with **TI** = **`11223344`**.

A.4.1 The MAC layer

```
truncateMac(00112233445566778899aabbccddeeff) = 1133557799bbddff
```

(literally the odd-indexed bytes — verifiable by eye)

Call	ctr	MAC input	Result
`commandMac(0x6C, {01})`	0	`6c00001122334401`	`4d131ee17b482b16`
`responseMac({})`	1	`00010011223344`	`6b082e8062d1a89d`

A.4.2 The crypto layer

Call	ctr	Result
`commandIv()`	0	`dd42134cd6970980d22066cb00ab1b16`
`responseIv()`	1	`cb73ae774aafd81868062906135969b9`
`padIso7816(96000000)`	—	`96000000800000000000000000000000`
`encryptCommandData(150)`	0	`a2eb8f000bf3ddfe6e18e8859b284591`

`96 00 00 00` is 150 little-endian, a €1.50 amount.

A.4.3 GetValue (balance 1000 cents)

Item	ctr	Bytes
card's encrypted payload	1	`7950692fc2f01e72b3cee290f7122b5c`
card's response MAC	1	`a44a7f2359b2f512`
decrypts to		`e8030000` + `80` padding → **1000**

Full fake reply: `7950692fc2f01e72b3cee290f7122b5c` + `a44a7f2359b2f512` + `9100`

A.4.4 Debit 150 cents, then Commit

Item	ctr	Bytes
debit ciphertext	0	`a2eb8f000bf3ddfe6e18e8859b284591`
debit command MAC input	0	`dc00001122334401a2eb8f000b f3ddfe6e18e8859b284591`
debit command MAC	0	`27e5ea2cb8acad7d`
debit APDU data	0	`01a2eb8f000bf3ddfe6e18e885 9b28459127e5ea2cb8acad7d`
debit response MAC	1	`6b082e8062d1a89d`
commit command MAC input	1	`c7010011223344`
commit command MAC	1	`4dc9859e1a783ee0`
commit response MAC	2	`febea1fcb55208ba`

Watch the counter walk $0 \to 1 \to 2$. That progression *is* the increment-ordering rule made visible.

A.5 Card personalisation

The values the kiosk and the top-up app must agree on. Change them anywhere and you must change them everywhere.

Setting	Value	Notes
Application ID	`0x010203`	on the wire `03 02 01` (**LSB-first**)
App master key no.	0	
App **user** key no.	**2**	everyday operations
Value file no.	`0x01`	
Comm mode	Full (`0x03`)	encrypted + MAC'd
Access rights	key 2 for all four roles	word `0x2222`
Lower limit	0 cents	
Upper limit	100000 cents	€1000
PICC master key	16 × `0x00`	factory default, never changed
App master key	16 × `0x11`	bench key
App user key	16 × `0x22`	bench key

TRAP These are bench keys

Identical on every card, hardcoded in the app, and published in this book. They protect nothing. See Section 13.6 for what a real deployment does.

A.6 DESFire command reference

Cmd	Name	Comm mode	Notes
`0x5A`	SelectApplication	plain	AID LSB-first
`0x60`	GetVersion	plain	chains 3 frames
`0x71`	AuthenticateEV2First	—	data `{keyNo, 0x00}`
`0xAF`	*continue / additional frame*	—	both a command and a status
`0x6C`	GetValue	Full	
`0x0C`	Credit	Full	stages only
`0xDC`	Debit	Full	stages only
`0x1C`	LimitedCredit	Full	needs the file flag
`0xC7`	CommitTransaction	MAC	**makes it permanent**
`0xA7`	AbortTransaction	MAC	discards staged changes
`0xCA`	CreateApplication	plain	provisioning
`0xCC`	CreateValueFile	MAC	provisioning
`0xC4`	ChangeKey	Full	**irreversible**

A.7 Status words

Status	Meaning	Usual cause
`91 00`	OK	
`91 AF`	additional frame follows	not an error — drain the chain
`91 0C`	no changes	
`91 1C`	illegal command	wrong command for this file type
`91 1E`	integrity error	**your MAC or CRC is wrong**
`91 7E`	length error	wrong `Lc` — e.g. auth step 1 missing its `0x00`
`91 9D`	permission denied	authenticated with the wrong key
`91 AE`	authentication error	wrong key, or session invalidated
`91 BE`	boundary error	debit would breach the file's limits
`91 F0`	file not found	card not provisioned
`91 F1`	file integrity error	card-side corruption

`6E 00` and `6D 00` are **ISO-7816** errors, not DESFire ones. They mean your APDU wrapper is malformed — bad CLA or INS — not that the card rejected the command.

CHAPTER B

The bug museum

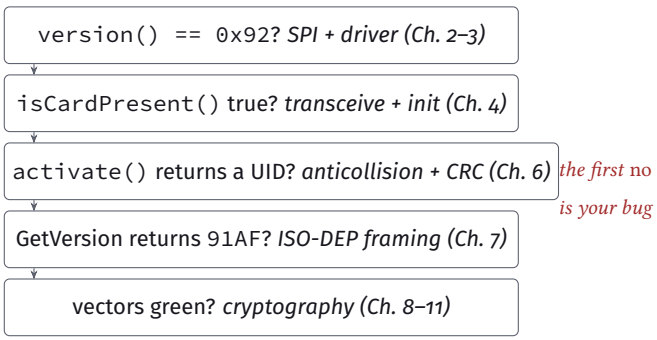
Every trap in this book, indexed by **what you actually see**. These are real bugs from the real build, not hypotheticals.

B.1 How to debug this stack at all

Before the tables — the method, which matters more than any individual fix.

B.1.1 Bisect by layer

The stack has six layers and two seams. Find which layer is lying:



Do not debug above a layer that has not answered "yes". Most long debugging sessions are someone investigating layer five while layer two is broken.

B.1.2 Three more rules

Crypto never gets debugged on hardware.
If a card operation fails, run the offline vectors first. Green vectors mean the bug is in framing,

state, or the card's configuration — not in your mathematics. That single distinction saves hours.

Print the MAC *input*, not the MAC.
When a MAC mismatches, the output tells you nothing — it is noise either way. The input tells you everything: you can see immediately whether the counter, the TI, or the data is wrong.

Suspect the counter first.
In secure messaging, roughly nine failures out of ten are the command counter being off by one, not the cryptography.

B.2 Hardware and SPI (Chapters 2–3)

You see	Cause / fix
`version()` = −110	Signed byte. Add `& 0xFF`.
`version()` = `0x00`	MISO stuck low: power (3.3 V!), wiring, CS pin.
`version()` = `0xFF`	MISO floating: not connected.
Version flickers between runs	Wire length / noise. **Shorten the jumpers.** Not a code bug.
Reads fine, writes ignored	Read bit `0x80` set on writes.
`NoClassDefFoundError: com/sun/jna`	JNA missing. `mvn dependency:copy-dependencies`.
`No gpio chip found` in a *unit test*	The test constructs real hardware — you inherited instead of composing.
Second run fails, first worked	Device file still locked. `close()` / try-with-resources.
`Permission denied` on `/dev/spidev0.0`	`sudo usermod -aG spi,gpio $USER`.

TRAP The `extends Pi4jSpiLink` disaster

Writing `class Rc522Driver extends Pi4jSpiLink` compiles and looks convenient. Then every unit test constructs a Pi4J context, hunts for a GPIO chip, and dies on your laptop.
You have welded the driver to the hardware and destroyed the seam. **A driver HAS-A link; it is not a kind of link.**

B.3 Card detection and activation (Chapters 4, 6)

You see	Cause / fix
Program runs for *minutes*	Ported the C's iteration counter. Use a `nanoTime` deadline.
`0/15` polls, version fine	Init incomplete: `TxASKReg`=`0x40`, `ModeReg`=`0x3D`.
Detects once, then blind forever	Card sits in HALT. Send WUPA (`0x52`) after REQA.
Detection about half the time	Card position or wires. Flat on the antenna, short wires.
7-byte UID, first byte wrong	Kept the `0x88` cascade tag. Use `ac1[1..3]`.
`select()` returns −1	CRC wrong. `ModeReg`=`0x3D` (preset `0x6363`).
`SAK = 0x08`	It is a MIFARE Classic. Wrong card.
RATS empty	Not 14443-4, or RATS sent twice.
Works, then fails on retry	Card left ACTIVE. `softReset()` + `antennaOn()` per tap.
`NegativeArraySizeException`	FIFO level read too early. Check the IRQ wait loop.

B.4 ISO-DEP and APDUs (Chapter 7)

You see	Cause / fix
Empty reply	Block number desynced. Toggle once per exchange.
Works exactly once per power-up	Never toggling. `blockNumber ^= 1`.
Reply one byte too long	PCB not stripped. `copyOfRange(reply, 1, ...)`.
`6E 00` / `6D 00`	ISO-7816 error. Check CLA `0x90` and the trailing `0x00`.
`91 AE` on a command that used to work	Card mid-chain from `91 AF`. Drain it or reset the field.
GetVersion byte 3 is `0x12`	It is an EV1 — no `AuthenticateEV2First`.

B.5 Cryptography (Chapters 8–11)

You see	Cause / fix
CMAC empty-message vector fails only	Missing the `n=1, lastComplete=false` case.
CMAC partial-block vectors fail	Padding: `0x80` then zeros, not all zeros.
CMAC full-block vectors fail	K_1/K_2 swapped. Full $= K_1$, padded $= K_2$.
All CMAC vectors fail plausibly	`leftShift` carry direction. Walk **backwards**.
Both session keys wrong	SV layout. Print the SV and compare to the spec.
Session keys swapped	Labels reversed. `0xA5` = enc, `0x5A` = mac.
`91 7E` on auth step 1	Sent only `keyNo`. Must be `{ keyNo, 0x00 }`.
Step-2 ciphertext $\neq$ vector	Two separate CBC calls. It is one **32-byte** run.
Auth "succeeds" always	You never compared RndA′. Actually throw on mismatch.
Fake test decrypts garbage	Replies lack `91 xx`. Append the status word.
Auth OK, first `getValue` MAC fails	**Session built without TI.**
Command MACs right, response MACs wrong	Increment ordering: increment *before* verifying.
First command OK, second fails	Counter not incremented, or incremented twice.
Balance unchanged after debit	No commit. Working as designed.
`91 BE`	Boundary error: debit breaches the file's lower limit.
`91 9D`	Permission denied: wrong key for that file's access rights.
`91 F0`	File not found: card not provisioned.
Everything fails after one bad run	Card mid-transaction. Power-cycle the field, re-authenticate.

B.5.1 The three crypto bugs that cost the most time

1. The TI one-liner.
`authenticateEv2First` builds the session; if you pass only `(key, rndA, rndB)`, the TI defaults to zeros. Authentication still *succeeds* — the TI is not used until secure messaging — so the failure appears *one command later*, in code that is completely correct.

2. Counter increment ordering.
The counter ticks after the card accepts the command and *before* you verify the response. Backwards, and every command MAC is right while every response check fails.

3. Fake replies without status words.
Your card code strips two bytes off every reply. A fake returning bare ciphertext loses two real

bytes, and the decrypt produces garbage that looks exactly like a crypto bug.

B.6 Tests and tooling

You see	Cause / fix
Test passes but proves nothing	The method is never called. Assert on a *return value*.
Test passes, hardware fails	Fake too permissive. Make it assert what it received.
Changed code, behaviour identical	Stale `.class` files. `mvn clean compile`.
Passes alone, fails in the suite	Shared mutable state. Fresh session per `@Test`.
Debit and getValue tests conflict	Same fake, different counters. Separate `@Test` methods.

TRAP The green test that tests nothing

The most dangerous bug in this book, because it *rewards* you. You add a `case 0x6C` to your fake, run the suite, see green — and never notice the test body does not call `getValue`.

Guard against it deliberately. After writing a test, ask: *what would I have to break to make this fail?* If the answer is "nothing", it is not a test.

B.7 UI and deployment (Chapters 12, 14)

You see	Cause / fix
UI freezes on tap	Card work on the FX thread. Use the background worker.
Intermittent rendering corruption	Node touched off-thread. `Platform.runLater` everywhere.
Animations crawl	Software rendering. `-Dprism.forceGPU=true`.
Touch inverted	Display rotated, input not. Rotate the input device too.
A tile stuck highlighted	Xwayland cursor sitting on it. `Cursor.NONE`.
Second tap always declines	Field not power-cycled per transaction.

You see	Cause / fix
Kiosk dies overnight	Unhandled exception. Broad catch + `Restart=always`.
Simulator used on the Pi	Constructor threw. Read the logged message.

B.8 Environment oddities from the real build

Not bugs in your code, but they will cost you time anyway.

- **The Pi's IP changes constantly** on DHCP. Set a static lease or use `raspberrypi.local`.
- **SSH host-key conflicts** after a DHCP reshuffle: *password authentication is disabled to avoid man-in-the-middle attacks.* Fix the known-hosts entry rather than disabling checking permanently.
- **sshd dies under memory pressure** on a Zero 2 W with a JavaFX app running. Add swap, or do not run the kiosk while developing.
- **`git commit` hangs for minutes** on a folder synced by iCloud or Dropbox — the index refresh stalls on the network filesystem. Keep repositories out of synced folders.
- **`timeout` is not on macOS** by default. Use `gtimeout` from coreutils, or a background job.

CHAPTER C

YouTube build plan

A production plan for filming this build: series structure, per-episode scripts, shot list, and the pre-flight checklist that stops a stream dying on air.

C.1 The pitch

> *"I built a coffee machine that takes contactless payments — and I wrote every byte myself, from the SPI bus up. No libraries. Here is how the cryptography in your bank card actually works."*

Why this works as content: almost every NFC tutorial online is `import nfc; print(tag.uid)`. Nobody shows the anticollision tree walk, the mutual authentication, or the reason a debit and a commit are separate commands. You have a genuinely rare artefact: a from-scratch payment stack that *works*, with a physical payoff at the end.

The hook is the money. Not "let us learn OOP" — it is *"this card has €5 on it, and by the end of this video my code will take €1.50 off it."* Every episode should end with something visibly working.

C.2 Series structure

Two formats, and you should decide up front which you are making.

	Edited series	**Livestream**
Length	10–25 min per episode	90–120 min
Audience	Discovery, evergreen	Existing community
Prep	High (script, retakes)	Low — but rehearse
Best for	Episodes 1, 6, 7	The grind chapters

Recommendation: livestream the build, then cut a highlights series from the footage. You get both, and the edited version benefits from having already explained everything once.

C.2.1 Episode map

Ep	Title	Chapters	Payoff on screen	Len
1	Talking to an NFC chip with zero libraries	1–3	`version() = 0x92`	18
2	Waking up a contactless card	4	`15/15 polls`	15
3	Reading a UID — the anticollision dance	5–6	UID on screen	20
4	First words with a DESFire	7	`...91AF`	12
5	Writing AES-CMAC from the RFC	8	4 vectors green	22
6	**Proving I know a secret without sending it**	9–10	`MUTUAL AUTH OK`	25
7	**Taking money off a real card**	11	`5.10 -> 5.00`	20
8	Building the kiosk	12, 14	Tap → brew	18
9	The phone that tops it up	13	Balance goes up	15
10	What I would do differently	—	Retrospective	12

Episodes 6 and 7 will carry the series. Put the most effort into episode 6: mutual authentication is genuinely beautiful and almost nobody explains it well on video.

C.3 Episode scripts

C.3.1 The structure for every episode

Time	Beat
0:00	**COLD OPEN** — the payoff, out of order. 15 seconds, no talking head.
0:15	**PROBLEM** — what does not work yet, concretely
1:30	**THEORY** — the idea + the analogy (whiteboard or slide)
4:00	**BUILD** — live coding, the hard part
12:00	**IT FAILS** — keep this. Always keep this.
14:00	**FIX** — the diagnosis, out loud
16:00	**PAYOFF** — it works, on hardware
17:00	**NEXT** — one sentence teasing the next episode

> **CONCEPT Keep the failures**
>
> The single most important production note in this appendix. **The debugging *is* the content** — it is what separates you from a tutorial that hides the mess.
> When your MAC fails and you reason your way to "the counter increments before the response", that is the most valuable ninety seconds in the episode. Nobody learns from watching code work the first time, and nobody believes it either.

C.3.2 Episode 1 — Talking to an NFC chip with zero libraries

Cold open: close-up of the card touching the reader, kiosk screen shows €5.10 → €5.00. Cut to black. *"By the end of this series, every byte of that transaction will be code I wrote. Today: making the chip say hello."*

1. Show the RC522. £3. Show the datasheet — the register map page. *"This is all the documentation there is, and it is enough."*
2. Wire it up on camera. **Say the 3.3 V warning out loud** — it is the one mistake that costs a viewer their module.
3. The problem: *"I could plug this in and print things until it works. Two minutes per attempt, and when it fails I will not know if it is my code, my wiring, or the chip."*
4. **Analogy: the wall socket.** Draw it.
5. Build `SpiLink`, `SpiException`, `FakeSpiLink`. Run the fake test green — **with no hardware plugged in.** Hold up the unplugged cable to camera. That visual sells the whole idea.

6. Build `Rc522Driver`. Explain the address encoding on screen, one bit at a time.
7. **Deliberate failure:** run it and get −110. *"That is the signed byte thing. Java has no unsigned types."* Fix with `& 0xFF`.
8. **Payoff:** on the Pi. `version() = 0x92`.

Line to land: *"That `0x92` means the chip is alive and I am speaking its language. Nothing between my code and the silicon but two bytes of SPI."*

C.3.3 Episode 5 — Writing AES-CMAC from the RFC

The episode that proves the series is serious.

Cold open: scroll RFC 4493 on screen. *"I am going to implement this. Not import it — implement it. And then prove it is correct."*

1. What a MAC is. **Analogy: the wax seal.** Emphasise: it authenticates, it does not hide. Different jobs.
2. Why CMAC exists — the last-block problem with naive CBC-MAC, in one sentence.
3. Subkey generation on the whiteboard.
4. Code `leftShift` live. **Get the carry direction wrong** — deliberately, or genuinely, it is a common slip. All four vectors fail with plausible-looking garbage.
5. *"And this is why we do not test crypto on hardware. Look at that output — it is sixteen bytes of noise. Correct output is also sixteen bytes of noise. I cannot tell them apart by looking."* **This is the thesis of the whole series.** Deliver it slowly.
6. Fix: walk backwards, carry propagates up.
7. **Payoff:** four RFC vectors, green.

Line to land: *"Those four numbers were published in 2006 by people who have never seen my code. My code reproduces them exactly. That is not 'it seems to work' — that is proof."*

C.3.4 Episode 6 — Proving I know a secret without sending it

The best episode. Take your time.

Cold open: *"This card and my Pi both know a sixteen-byte number. In the next twenty minutes they are going to prove it to each other — and if you record every bit of radio between them, you learn nothing."*

1. Set up the problem properly. *"I cannot send the key. I cannot send a hash — you would replay it. So how?"*
2. **Analogy: two spies.** Emphasise it must be **mutual** — one-way lets an impostor collect the passphrase.
3. Draw the full dance on the whiteboard. This is the money shot of the series; animate it in post if you can.
4. The testability problem: *"This function calls a random number generator. So it does something different every time. How do I write a known-answer test for that?"* Pause — let the viewer feel it.
5. **Reveal `RandomSource`.** *"Randomness is a dependency like any other. Inject it, and I can pin it."*
6. Show the test class implementing *both* seams. *"My test is the fake card AND the fake dice."*
7. **Payoff, in two stages** — the ordering matters: offline vectors green (*"provably correct with no card in the room"*), then the real card: `MUTUAL AUTH OK. TI = 8071F582`.
8. **Run it twice.** Different TI each time. *"That is not a replay of my test data. That is a live handshake with a real chip."* Best twenty seconds in the series.

C.3.5 Episode 7 — Taking money off a real card

Cold open: the balance dropping. Full screen.

1. Three requirements: authenticity, freshness, confidentiality. **Analogy: the cheque** — amount, signature, cheque number.
2. The counter, and the ordering rule. Say it three times, three ways. It is the bug everyone hits.
3. Build `getValue`. **Deliberately forget the TI one-liner.** Auth succeeds, `getValue` fails its MAC. Debug on camera — a great sequence, because the failure is one command away from its cause.
4. **The debit/commit split.** Stop and explain properly: *"Debit does not move the money. If the card leaves the field before the commit, nothing happened. That is not a limitation — that is the entire safety design."* Then **physically yank the card mid-transaction** and show the balance unchanged. Best demo in the series.
5. **Payoff:** `510 -> 500 cents`, then tap the phone app and show the balance really is 5.00 on an independent device.

Line to land: *"Every layer of that — the register writes, the anticollision, the CMAC, the session keys, the handshake, the secure messaging — I wrote. It took me [N] weeks and it moves real money."*

C.4 Shot list

Cameras.

- **A** — screen capture, 1080p60 (OBS; editor plus a terminal).
- **B** — overhead on the bench: hands, reader, card. A phone on a gooseneck is fine.
- **C** *(optional)* — talking head for intros and outros.

Recurring shots you will need repeatedly:

- Card approaching the reader, slow, close. *Get ten takes of this early* — you will reuse it constantly.
- Terminal output scrolling to a success line.
- Whiteboard or tablet diagrams.
- Insert of the datasheet register table.
- The kiosk screen, straight on, no glare.

Audio matters more than video. A lavalier or a USB condenser. Viewers forgive 720p; they leave over hiss.

Screen legibility: editor at **16–18 pt minimum**, high-contrast theme, and **zoom in on the specific lines you are discussing** in post. Nobody can read your normal font size on a phone.

C.5 Livestream pre-flight

Print this. Run it before every stream.

Hardware, 30 minutes before:

☐ Pi booted, on the network, IP noted (**static lease** — DHCP will move it mid-stream)

☐ `ssh` session open and tested

☐ RC522 responds: `version() = 0x92`

☐ Test card has a **known balance** — set it deliberately with the phone app

☐ **A second card, provisioned identically**, in a drawer

☐ Reader taped down. It *will* get nudged on camera.

Software:

☐ `mvn clean compile` passes

☐ `mvn test` all green

☐ Vectors file open in a tab — you *will* be asked to show them

☐ Git committed to a known-good point; `git stash` any half-finished work

The safety net — the most important item here:

☐ A branch with the chapter **already working**, ready to `git checkout` if you get stuck live

> **CONCEPT The ten-minute rule**
>
> That safety net is not cheating. Streaming with no escape hatch is how you spend forty minutes on a typo in front of an audience.
> Give yourself a hard rule: **stuck for ten minutes on air → check out the working branch, walk through it, and move on.** Debug it off-stream and cover the fix at the start of the next episode. Your audience will respect the discipline more than the struggle.

Stream ergonomics:

☐ Notifications off, phone silent, second monitor off-camera for chat

☐ Water within reach

☐ Say what you are about to do *before* doing it — silent typing is dead air

C.6 The honest bits — say these on camera

Credibility comes from what you are willing to admit.

1. The keys are hardcoded and that is not secure.
Say it plainly: *"Every card in my fleet has the same key, and that key is in the APK in plaintext. Anyone who pulls the APK can mint unlimited balance. For a bench project with test cards that is fine — I am telling you so nobody copies this into something real."* Then explain diversification

and HSMs briefly. Half your technical audience already knows; the ones who do not will learn the most important thing in the series.

2. Cards can be destroyed.
Warn viewers before the provisioning episode. Somebody *will* run it against their office badge.

3. What you would change.
Episode 10: the FSCI nibble bug that cost you an evening, the three-second dead time from a bad wait loop, choosing wall-clock deadlines over iteration counts. A retrospective on your own mistakes is unusually good content — and it proves you understand the thing rather than having merely typed it.

C.7 Titles and thumbnails

Titles that work:

- *I wrote a contactless payment system from scratch (no libraries)*
- *How your bank card proves it knows a secret — without sending it*
- *Taking real money off an NFC card with code I wrote myself*

Titles that do not: anything starting with "Learning", "Tutorial", or "Part 3". Lead with the artefact.

Thumbnails: the card mid-tap with the balance visible, or a hex dump with one line circled. Avoid a face plus arrow if you want the technical audience — they read it as low-signal.

Description boilerplate: link this book, the repository, the RFC, and the NXP datasheet. Timestamp the chapters. Pin a comment with the bill of materials.

C.8 Realistic schedule

Phase	Effort
Rebuild solo from this book (no camera)	3–5 evenings
Rehearse each episode's build once	1 evening each
Record	1 session each
Edit (for the edited series)	3–4 h per finished episode

Do the full rebuild before filming anything. Not because you will forget — because the

second time through you will know exactly where the interesting explanations live, and you will have already hit the bugs you want to keep in.

Index

abstract class, 32
Adapter pattern, 111
anticollision, 50

composition, 46

dependency injection, 19

encapsulation, 21
enum, 80

immutability, 51
inheritance, 46
interface, 12
invariant, 72

Java
    versus C, 6

polymorphism, 42

RC522
    wiring, 3

signed byte, 7
SPI
    wiring, 3

Template Method, 31

value object, 51